



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

School of Engineering

Department of Mechanical & Manufacturing Engineering

Final Report

4E4 Industrial Internship Project

In Partnership with ElectroRoute



ElectroRoute

Finn O'Connor | 22336740

Under Supervision of Dr. Amir Pakdel

July 24, 2026

A dissertation submitted in partial fulfilment of the degree:
M.A.I - Master of Mechanical and Manufacturing Engineering

Declaration

I hereby declare that this dissertation is entirely my own work and that it has not been submitted as an exercise for a degree at this or any other university.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at <https://www.tcd.ie/calendar/>.

I have completed the Online Tutorial on avoiding plagiarism 'Ready Steady Write', located at https://www.tcd.ie/library/support/plagiarism/story_html5.html.

I consent to the examiner retaining a copy of the thesis beyond the examining period, should they so wish (EU GDPR May 2018).

I agree that this thesis will not be publicly available but will be available to TCD staff and students in the University's open access institutional repository on the Trinity domain only, subject to Irish Copyright Legislation and Trinity College Library conditions of use and acknowledgement.

Signed:

Date:

Finn O'Connor

July 24, 2026

Abstract

This report presents the work undertaken during a seven-month industrial internship with ElectroRoute, an international energy trading and services company. The internship was based within the Trading Operations team, which is responsible for the real-time management of physical energy positions and the optimisation of renewable assets across the Irish and GB electricity markets.

The first project involved the design and development of a web-based KPI dashboard to track trading desk performance. Built using Flask as the backend framework and Plotly for interactive chart generation, the dashboard provides a comprehensive view of portfolio performance across three asset groups — DAM, RFIT, and SOL — with functionality to filter by date range and aggregation frequency. Key metrics including net income, capacity factor, dispatch down rates, day-ahead balancing costs, trader performance, and market analytics are displayed through a series of interactive charts and summary widgets. What began as a local prototype was later deployed to a dedicated internal server, making it accessible to the wider business, and documentation was produced to support its ongoing maintenance beyond the internship. The dashboard was presented to the CEO and wider team.

The second project involved a detailed analysis of solar trading strategy, conducted in collaboration with a senior trader. Forecast accuracy was evaluated across multiple auction stages and forecast providers using a dynamic ramp-based filtering approach, and alternative auction split strategies were backtested to assess whether reallocating volume from the day-ahead market to later intraday auctions could improve revenue outcomes. Key drivers of solar book performance were identified, including systematic forecast over-prediction, curtailment exposure during peak solar hours, and the influence of wind generation on electricity prices, and a framework for a predictive ensemble model to determine optimal daily auction splits was proposed.

The third project involved the development of a machine learning ensemble model to forecast system-wide electricity demand. Ridge regression was used to produce a weighted blend of two existing demand forecasts — one from EirGrid and one from an external provider — exploiting their complementary strengths. The blended model outperformed both individual forecasts on unseen test data, with training and test errors closely aligned, indicating that the model generalised well rather than overfitting to the training set.

The fourth project involved building a machine learning model, using XGBoost, to forecast net flow across the IE-GB interconnectors at 30-minute resolution for the day ahead of delivery. The model combined day-ahead price spreads, wind and demand forecasts, and lagged flow history as predictors, with hyperparameters tuned via Optuna under time-series cross-validation to prevent lookahead bias. A set of further improvements, including a per-interconnector ensemble architecture, was identified as a natural next step for the model.

In addition to these technical projects, a structured trading strategy framework was developed and applied through a paper trading exercise on the REFIT asset portfolio in the Day-Ahead Market. The framework codified the key factors influencing trading decisions — including weather conditions, system fundamentals, forecast confidence, interconnector flows, SNSP, and Net Imbalance Volume — into a set of guiding principles for position construction. Paper trading provided a daily feedback loop to iteratively refine this framework, developing practical intuition for how forecast uncertainty translates into financial risk and how to synthesise multiple conflicting market signals into a coherent trading position.

Across all projects and activities, the internship developed strong technical skills in Python, SQL, data visualisation, and the deployment of tools into production use by the wider team, as well as a deep understanding of renewable energy trading, electricity market structures, and the practical application of statistical and machine learning techniques to commercial decision-making.

Lay Abstract

This report describes the work completed during a seven-month internship at ElectroRoute, a company that trades electricity generated by wind farms and solar farms on behalf of its clients. The company operates in the Irish and British electricity markets, where electricity is bought and sold through a series of auctions held at different times before it is actually needed.

The first project involved building an interactive online dashboard that allows the trading team to monitor how well they are performing. The dashboard displays charts and statistics covering areas such as income, energy production, and trading costs. It was later moved from a version running on a single computer to one hosted on a company server, making it available to the wider business, and was designed so that new features can be added easily after the internship ends.

The second project analysed how the company trades its solar energy. Currently, most solar electricity is sold in the earliest auction, but the analysis investigated whether selling some of it in later auctions — when forecasts of how much electricity the solar farms will produce are more accurate — could lead to better financial outcomes. The analysis found that forecast errors and grid curtailment were the biggest factors affecting performance, and proposed a data-driven approach to deciding how to split volume across auctions each day.

The third project developed a regression model that combines two existing electricity demand forecasts to produce a single, more accurate prediction. By learning from historical data which forecast tends to be more accurate under different conditions, the model was able to reduce prediction errors compared to using either forecast on its own.

The fourth project built a machine learning model to predict the flow of electricity between Ireland and Britain through the undersea interconnector cables, up to a day in advance, using forecasted weather, demand, and price data. This gives the trading desk an earlier read on expected cross-border flow than was previously available, ahead of the interconnector nominations that are otherwise only published close to real time.

Alongside these technical projects, a practical trading strategy was developed and tested through simulated trading. Each day, weather forecasts, energy supply and demand data, and market conditions were reviewed to decide how much electricity to sell and when. The simulated trades were then compared against actual market outcomes, providing immediate feedback that was used to refine the strategy over time. This exercise built practical experience in making trading decisions under uncertainty — a skill that is difficult to develop through analysis alone.

Together, these projects and activities contributed practical tools, analysis, and market knowledge to support the company's trading operations, while providing hands-on experience in data analysis, programming, energy markets, and machine learning.

Acknowledgements

I would like to thank the team at ElectroRoute for their support and guidance throughout my placement. In particular, I would like to thank my manager Patrick Larkin, Operations Trading Manager, and Rowan Osborne, Shift Trading Team Manager, for providing me with the opportunity to work on meaningful projects and for taking the time to share their knowledge and experience throughout the year. I would also like to thank my academic supervisor Dr. Amir Pakdel for his support and feedback throughout the placement.

It should be noted that a number of details have been redacted from this report in order to protect ElectroRoute's intellectual property and confidential client data. This includes forecast provider names, asset site names, trader names, generation figures, and commercially sensitive revenue and balancing cost data.

Contents

List of Abbreviations	6
1 Introduction	7
1.1 Company Background	7
1.2 Internship Role and Project Area	7
1.3 The Irish Integrated Single Electricity Market	8
1.4 Report Structure	9
2 SQL Query Development	9
3 KPI Dashboard Build	12
3.1 KPI Dashboard Overview	12
3.2 Dashboard Architecture and Deployment	12
3.3 Key Metrics	14
3.4 Skills and Learning Outcomes	29
4 Solar Trading Strategy Development	30
4.1 Filtering the Data	31
4.2 Forecast Performance Analysis	33
4.3 Individual Solar Site Performance Analysis	34
4.4 Auction Split Strategy Backtest	36
4.5 Finding the Optimal Split	38
4.6 Influence of Wind and Dispatch Down	40
4.7 Next Steps	42
4.8 Skills and Learning Outcomes	44
5 IE-GB Interconnector Flow Prediction Model (XGBoost)	44
5.1 Approach	44
5.2 Key Concepts	44
5.3 Future Improvements	56
5.4 Skills and Learning Outcomes	56
6 Machine Learning Demand Forecasting Ensemble	57
6.1 Model Selection	57
6.2 Implementation	57
6.3 Model Output and Results	59
6.4 Next Steps	62
6.5 Skills and Learning Outcomes	62

7	Paper Trading and Trading Strategy	62
7.1	Underselling and Overselling	63
7.2	Weather-Driven Factors	63
7.3	System and Market Factors	65
7.4	System-Wide Forecasts	66
7.5	Combined Assessment	67
7.6	Paper Trading Results and Key Learnings	68
8	Certifications and Professional Development	70
8.1	Dataquest: Data Scientist in Python	70
8.2	Energy Markets Training	70
8.3	Regulatory Compliance	70
9	Conclusion	71

List of Abbreviations

API	Application Programming Interface
BM	Balancing Market
CEO	Chief Executive Officer
CPPA	Corporate Power Purchase Agreement
CSS	Cascading Style Sheets
DA	Day-Ahead
DAM	Day-Ahead Market
EWIC	East-West Interconnector
GB	Great Britain
GHI	Global Horizontal Irradiance
GW	Gigawatt
HTML	HyperText Markup Language
I-SEM	Integrated Single Electricity Market
IDA	Intraday Auction
IDA1	Intraday Auction 1
IDA2	Intraday Auction 2
IDA3	Intraday Auction 3
IDC	Intraday Continuous
KPI	Key Performance Indicator
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MCV	Market Clearing Volume
MW	Megawatt
MWh	Megawatt hour
NBP	National Balancing Point
NIV	Net Imbalance Volume
NSI	Normalised Solar Index
OLS	Ordinary Least Squares
PnL	Profit and Loss
PPA	Power Purchase Agreement
REFIT	Renewable Energy Feed-in Tariff
REMIT	Regulation on Energy Market Integrity and Transparency
RMSE	Root Mean Square Error
SCADA	Supervisory Control and Data Acquisition
SEM	Single Electricity Market
SEMO	Single Electricity Market Operator
SNSP	System Non-Synchronous Penetration
SOL	Solar
SONI	System Operator for Northern Ireland
SQL	Structured Query Language
TSO	Transmission System Operator

1 Introduction

1.1 Company Background

ElectroRoute Holdings Limited is an international energy trading and services company owned by the Mitsubishi Corporation. The company's mission is "to solve the commercial mechanics of a decarbonised energy system." By combining deep expertise in energy markets with cutting-edge digital infrastructure, ElectroRoute helps clients maximise the value of their renewable assets, navigate market volatility, and contribute to system-wide decarbonisation.

The company operates both proprietary trading and physical shift-based trading functions. Proprietary trading involves trading the firm's own capital to take positions in energy markets based on price signals, forecasts, and market fundamentals, with the objective of generating profit. Shift trading primarily relates to the physical trading and real-time management of electricity and gas positions. This involves the continuous, 24/7/365, execution of trades to balance physical supply and demand, manage forecast deviations, and respond to real-time system conditions. Shift traders are responsible for ensuring that physical delivery obligations are met, while managing imbalance risk and maintaining compliance with market rules.

As at March 2025, ElectroRoute had a turnover of €753.7 million and 124 employees. The company has offices in Dublin (headquarters), Letterkenny, London, and Tokyo. For the 2024/25 financial year, ElectroRoute managed over 2 GW of client assets and executed on average 5,200 trades per day. ElectroRoute currently holds a 3.5 GW portfolio of physical assets, and it is set to grow further, with 79 MW of renewables currently operating in the GB market expanding to 531 MW by FY26/27.

ElectroRoute offers a comprehensive range of energy trading and market services, including power, green certificate, and gas trading across wholesale markets; the structuring of Corporate Power Purchase Agreements (CPPAs) between renewable generators and corporate off-takers; fixed-price revenue guarantee structures that provide generators with revenue certainty; renewable balancing and market access services encompassing forecasting, nomination, and imbalance management; and battery and flexible asset optimisation across energy, balancing, and ancillary service markets.

1.2 Internship Role and Project Area

The internship was based within ElectroRoute's Trading Operations team, which sits at the core of the company's commercial activity. This team is responsible for the real-time and forward management of physical energy positions across electricity and gas markets, as well as the analysis and optimisation of renewable and flexible assets. The role operated at the intersection of energy systems, financial markets, and data-driven decision-making. The internship duration was 6 months long, from January to June 2026.

A key focus of the project area was physical trading and asset optimisation. This involved forecasting renewable generation and demand, monitoring system conditions, managing imbalances, and executing trades to ensure that physical delivery obligations were met in real time. Due to the variable nature of renewable generation, effective forecasting, monitoring, and rapid response to market conditions are critical to minimising imbalance costs and maximising asset value.

The internship also sat heavily within the domain of data analysis and automation. ElectroRoute maintains large databases populated with live and historical market, weather, and asset data. These datasets are used to produce dashboards, Key Performance Indicators (KPIs), trading lookaheads, and analytical tools that support both shift traders and proprietary trading teams. As a result, strong technical skills in Python, SQL, and Excel were essential within the project area.

Day-to-day responsibilities included performing daily market analysis on Irish and GB power markets, confirming SCADA availability and following up on outages, compiling and presenting weekly KPIs, preparing

trading lookaheads, reviewing constraint and outage reports, and attending SEMO market calls. Beyond these operational duties, the internship involved the completion of the pre go-live components of the ElectroRoute Academy trader training programme, covering electricity market structures, power systems, weather and forecasting, risk and compliance, and the operational responsibilities of the trading desk.

1.3 The Irish Integrated Single Electricity Market

The Integrated Single Electricity Market (I-SEM) is the wholesale electricity market for the island of Ireland, operated jointly by EirGrid (the Republic of Ireland's Transmission System Operator, or TSO) and SONI (the System Operator for Northern Ireland) through the Single Electricity Market Operator (SEMO). The I-SEM replaced the previous SEM arrangements in October 2018, introducing a series of sequential market timeframes designed to align with European electricity market coupling frameworks.

The market operates through a sequence of trading stages, each progressively closer to real-time delivery:

- **Day-Ahead Market (DAM):** The primary forward market, operated as a local auction through SEMOpx. Market participants submit buy and sell orders for each trading period of the following day, and prices are determined through market coupling. The DAM is the most liquid auction stage and typically where the majority of volume is traded.
- **Intraday Auction 1 (IDA1):** A coupled SEM–GB auction. The order book opens at 23:00 on D-19 and closes at 17:30 on D-1, covering trading periods from 23:00 to 23:00 (a full 24-hour delivery window). Each trading period is 30 minutes in duration.
- **Intraday Auction 2 (IDA2):** A coupled SEM–GB auction. The order book opens at 23:00 on D-19 and closes at 08:00 on the delivery day, covering trading periods from 11:00 to 23:00 (a 12-hour delivery window).
- **Intraday Auction 3 (IDA3):** A local auction. The order book opens at 23:00 on D-19 and closes at 14:00 on the delivery day, covering trading periods from 17:00 to 23:00 (a 6-hour delivery window).
- **Intraday Continuous (IDC):** A continuous trading platform operated through the European ECC clearing framework, allowing bilateral trades to be executed on a pay-as-bid basis up to one hour before delivery.
- **Balancing Market (BM):** The real-time balancing mechanism operated by the TSO to resolve any residual imbalance between supply and demand. Any difference between a participant's final traded position and their metered output is settled at the imbalance price, which can be significantly more volatile than day-ahead or intraday prices.

Auction	Order Book Opening	Order Book Closure	Trading Periods	Duration
IDA1	23:00 (D-19)	17:30 (D-1)	23:00 – 23:00	24 hours
IDA2	23:00 (D-19)	08:00 (D)	11:00 – 23:00	12 hours
IDA3	23:00 (D-19)	14:00 (D)	17:00 – 23:00	6 hours

Table 2: Intraday auction timelines in the I-SEM. All times are in Irish local time. Unlike the DAM, the trading period duration for each intraday auction is 30 minutes.

For renewable generators, the sequential structure of the I-SEM creates both opportunities and risks. Wind and solar output is inherently uncertain, and forecast accuracy typically improves as delivery approaches. Traders must therefore decide how to allocate volume across auction stages, balancing the liquidity and

price certainty of the DAM against the improved forecast accuracy available at later intraday auctions. Any residual mismatch between the final traded position and actual metered generation is settled in the balancing market, where the imbalance price can result in either a cost or a gain depending on the direction of the system imbalance.

The System Net Imbalance Volume (NIV) represents the net volume the TSO must procure through the balancing mechanism to balance the grid. Its direction determines whether a generator's residual imbalance is rewarded or penalised at settlement. Understanding and managing this imbalance exposure is central to the work of ElectroRoute's trading operations team and underpins much of the analysis presented in this report.

1.4 Report Structure

Section 2 describes the development of a reusable SQL query interface. Section 3 covers the design of a KPI dashboard for tracking trading desk performance. Section 4 presents analysis of solar trading strategy, including forecast accuracy and auction split backtesting. Section 6 details the development of a machine learning demand forecasting ensemble. Section 7 describes the trading strategy framework developed through a paper trading exercise. Section 8 outlines professional certifications and development activities completed during the internship.

2 SQL Query Development

A foundational task completed early in the internship was the development of a reusable SQL query function to efficiently retrieve data from ElectroRoute's market intelligence database. The database follows a star schema architecture, with a central fact table (`fact`) storing timestamped numerical values and multiple dimension tables describing the attributes of each record. Querying this database directly requires joining the fact table with up to six dimension tables and filtering on specific attribute values, resulting in verbose and error-prone SQL statements.

To address this, a Python wrapper function was developed that abstracts the query complexity into a simple interface. The function accepts descriptive parameters (category, country, source, zone, segment, date range, and optional time interval), dynamically constructs the appropriate SQL query, and returns results as a time-indexed Pandas DataFrame ready for analysis.

```
def sql_query(category, country, source, zone, segment, start, end,
              interval=None):
    # Convert datetime objects to SQL-compatible string format
    if isinstance(start, (dt.date, dt.datetime)):
        start = start.strftime("%Y-%m-%d %H:%M:%S")
    if isinstance(end, (dt.date, dt.datetime)):
        end = end.strftime("%Y-%m-%d %H:%M:%S")

    # Base query structure with joins to all dimension tables
    base_query = """
        SELECT
            f.F.UTC_Datetime AS Start_Time,
            f.F.Value AS quantity
        FROM fact f
        JOIN dim dim
            ON f.F_D_ID = dim.D_ID
            AND dim.D_Category = '{category}'
        JOIN dimCountry countryFrom
            ON countryFrom.CN_ID = dim.D_CN_ID_CountryFrom
```

```

        AND countryFrom.CN_TwoLetterName = '{country}'
    JOIN dimSource s
        ON s.SC_ID = dim.D_SC_ID_Source
        AND s.SC_Name = '{source}'
    JOIN dimTimeInterval timeInterval
        ON timeInterval.VI_ID = dim.D_VI_TimeInterval
    JOIN dimZone zoneFrom
        ON zoneFrom.ZN_ID = dim.D_ZN_ID_ZoneFrom
        AND zoneFrom.ZN_Name = '{zone}'
    JOIN dimSegment segment
        ON segment.SG_ID = dim.D_SG_ID_Segment
        AND segment.SG_Segment = '{segment}'
    """

    # Add interval filter if specified
    if interval:
        interval_filter = f"AND timeInterval.VI_TimeInterval = '{interval}'"
    else:
        interval_filter = ""

    # Complete query with date filters and ordering
    query = base_query + f"""
        {interval_filter}
        WHERE f.F_UTC_Datetime >= '{start}'
            AND f.F_UTC_Datetime < '{end}'
        ORDER BY f.F_UTC_Datetime
    """
    query = query.format(category=category, country=country, source=source,
                        zone=zone, segment=segment)

    # Execute query and return as DataFrame
    data = dbConnect.select(query, "Vor")
    df = pd.DataFrame(data, columns=['Start_Time', 'quantity'])
    if not df.empty:
        df['Start_Time'] = pd.to_datetime(df['Start_Time'])
        df = df.set_index('Start_Time')

    return df

```

The function operates as follows:

1. **Date conversion:** Python datetime objects are automatically converted to SQL-compatible string format (YYYY-MM-DD HH:MM:SS), allowing users to pass dates in their native Python format.
2. **Dynamic query construction:** The function builds a SQL query by joining the fact table with all relevant dimension tables. Each join includes both the foreign key relationship and a filter condition on the dimension attribute (e.g., `D_Category = 'Market|DayAhead|Price'`).
3. **Optional interval filtering:** When the interval parameter is provided (e.g., `'30 Minute'`), an additional filter is applied to the time interval dimension. When omitted, all available time intervals are returned, allowing the function to handle both settlement-period data and aggregated hourly or daily values.
4. **Date range filtering:** The `WHERE` clause restricts results to the specified date range using inclusive start and exclusive end semantics.
5. **DataFrame conversion:** Query results are converted to a Pandas DataFrame with the timestamp column set as a `DatetimeIndex`, making the output immediately compatible with time series analysis and visualization libraries.

The underlying database connection is managed by a separate `select` function that handles MySQL connection pooling, automatic retries on connection loss, and credential management:

```
def select(query, database, retries=2):
    """
    Execute a SQL query against the specified database with automatic retry
    ↪ logic.

    Args:
        query: The SQL query string to execute
        database: Database identifier for connection routing
        retries: Number of retry attempts on connection failure

    Returns:
        numpy.ndarray: Query results as a 2D array

    Raises:
        MySQLdb.OperationalError: If all retry attempts fail
    """
    cnxn, cursor = connect(database)

    # Attempt query execution with retry logic
    for i in range(retries - 1):
        try:
            cursor.execute(query)
            break
        except MySQLdb.OperationalError:
            continue
    else: # Final attempt without exception handling
        cursor.execute(query)

    data = np.array(cursor.fetchall())
    cursor.close()
    cnxn.close()
    return data
```

This two-layer architecture separates query construction (operational logic) from database connection management (infrastructure), improving code maintainability and testability.

The `sql_query` function was used extensively throughout all three projects presented in this report, serving as the primary data retrieval interface for the KPI dashboard, solar trading analysis, and demand forecasting ensemble. By standardizing data access, it reduced query construction errors, improved code readability, and significantly accelerated development. An example call to retrieve half-hourly IDA1 prices for the Irish Single Electricity Market is shown below:

```
da_prices = sql_query(
    category = "Market|IntraDay1|Price",
    country = "IE",
    source = "SEMOpX",
    zone = "SEM",
    segment = "Power",
    start = "2025-04-01",
    end = "2026-04-01",
    interval = "30 Minute"
)
```

This single function call replaces what would otherwise require manually writing a 20+ line SQL query with multiple joins and filters, demonstrating the practical value of abstraction in data engineering workflows.

3 KPI Dashboard Build

At the start of the internship, the trading desk had no single, consolidated way to track its own performance — balancing costs, dispatch down, forecast accuracy, and trader PnL were each assessed separately, if at all, through manual spreadsheet pulls and ad hoc queries. I was tasked with building a KPI script to close this gap and give the desk a repeatable way to monitor how the portfolio was performing. What began as a relatively straightforward scripting task gradually grew in scope, eventually evolving into a fully interactive web-based dashboard, as it became clear that a static script would not be enough to serve the range of questions the desk and wider business wanted answered on an ongoing basis.

3.1 KPI Dashboard Overview

The KPI dashboard provides a consolidated, interactive view of portfolio performance across the DAM, RFIT, and SOL asset groups. It brings together balancing cost analysis, dispatch down and curtailment tracking, forecast accuracy, intraday trading performance (by both auction and individual trader), and wider market analytics (demand, interconnector flows, and price behaviour across ISEM and GB) into a single Flask-based application. The full dashboard, generated for the reporting period 13–19 July 2026, is reproduced in full below.

3.2 Dashboard Architecture and Deployment

The dashboard began life as a Jupyter notebook, was then converted into a Flask application running locally on my machine, and has since evolved further — it is now hosted on a dedicated internal server, making it accessible to anyone in the company via the internal network rather than a single local instance.

The application itself remains a Flask-based single-page application with a REST API backend. Data is loaded on demand and persisted in a global state object for the lifetime of the server process, rather than being re-queried on every request.

```
dashboard_state = {
    'results'      : None,
    'start_date'   : None,
    'end_date'     : None,
    'agg_freq'     : 'D',
    'last_updated' : None
}
```

The user selects a date range and aggregation frequency (Daily/Monthly) via the frontend, which triggers a POST request to `/api/load_data`. This loads all data into the global state, making it available to all subsequent API calls without re-querying the database.

The application exposes a series of REST API endpoints, each serving a specific function:

Route	Method	Purpose
<code>/api/load_data</code>	POST	Loads data for the selected date range
<code>/api/chart/<chart_id></code>	GET	Generates and returns a specific chart
<code>/api/stats/<group></code>	GET	Returns KPI summary statistics

Table 3: Flask REST API endpoints.

Charts are generated server-side using Plotly, serialised to JSON and passed to the frontend for rendering. The chart route handles both single and multiple figure responses dynamically, as some dashboard sections return more than one chart simultaneously.

Project Structure:

```
1 kpi_dashboard/
2 |─ app.py           # Flask app (main entry point)
3 |─ data_loader.py   # Data loading functions
4 |─ templates/
5 |   └─ index.html   # Dashboard UI
6 |─ static/
7 |   └─ *.png        # Logos
8 |─ charts/
9 |   └─ __init__.py   # Chart registry
10 |    └─ colors.py    # Color theme
11 |    └─ *.py         # Chart modules
```

Figure 1: Dashboard project directory structure.

The project is split into four main components: `app.py` handles all Flask routing and API logic, `data_loader.py` contains all data loading and processing, and the `charts/` directory holds all chart modules, with each individual file representing a self-contained chart that can be added or removed without touching the core application logic. The frontend lives in the `templates/` and `static/` directories, and discovers available charts dynamically at runtime rather than requiring manual wiring on the frontend side.

Hosting. The dashboard now runs on a dedicated internal server, served using Waitress (a production-ready WSGI server) rather than Flask’s local development server. It is started via a `.vbs` launcher script that runs the underlying batch file silently in the background, so that the process is not accidentally terminated by someone closing a terminal window — a deliberate choice given multiple people rely on the same running instance. As the server holds all loaded data in memory rather than persisting it, a restart requires users to reload their data before continuing.

Deploying changes. Because the dashboard runs centrally rather than locally, changes are not deployed by simply running the updated code. Instead, they pass through three locations: changes are made and tested locally, pushed to the repository, copied across to a shared network drive as an intermediary step, and finally copied from the shared drive onto the server itself before the dashboard process is restarted. Each chart loads independently, so a broken chart pushed in error will show an error box in its place without taking down the rest of the dashboard — though everything is still tested locally first as a matter of course.

To make sure the dashboard remains maintainable beyond my time on the desk, I documented this entire setup and deployment process, along with the full data pipeline and instructions for adding new charts, queries, and traders, in a standalone README for whoever inherits the project next.

Once the server is running, the dashboard is accessible to anyone on the internal network at the configured server address on port 5000, presenting the interface shown in Figure 2. From here, the analysis start and end dates can be selected using the date widgets, and the aggregation frequency can be toggled between Daily, Weekly, and Monthly. Clicking `Load Data` queries the database and loads all data into the global state, with a loading indicator displayed while the data is being fetched; once loaded, clicking `Refresh Charts` re-renders all charts for the currently active tab without re-querying the database.

The dashboard is organised into six tabs — DAM, RFIT, SOL, All Groups, Trader Performance, and Market Analytics — each containing a set of charts and KPI widgets relevant to that asset group or analysis type. Daily aggregation gives the most granular view of performance, while Monthly aggregation smoothens out short-term noise and can be more useful for identifying longer-term trends.

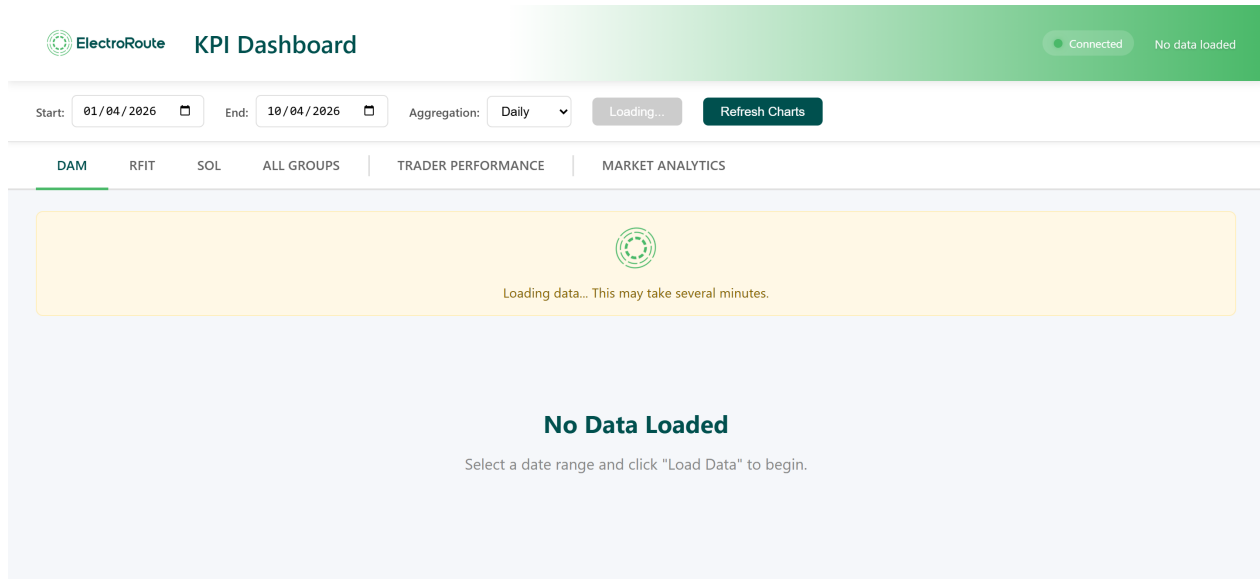


Figure 2: KPI dashboard interface with date range selector and aggregation toggle, loading DAM data.

3.3 Key Metrics

The following metrics recur throughout the dashboard and are defined here for reference, rather than repeated at each occurrence.

- **Capacity Factor (%)**: Production as a proportion of maximum possible output given installed capacity.
- **Dispatch Down Rate (%)**: Share of available generation not dispatched, split into curtailment (grid operator) and constraints (network).
- **DA Position Cost / Actual Balancing Cost (€/MWh)**: Blended day-ahead sale price, and the realised cost per MWh after all DA and IDA trading.
- **100/90/110% DA Forecast Bal Cost**: Hypothetical balancing cost had the portfolio traded exactly 100%, 90%, or 110% of forecast volume, used as sensitivity benchmarks.
- **Ex-Ante / Imbalance / Net Income**: Auction trading revenue, residual imbalance revenue or cost, and their sum.
- **IDA Net Gain vs. Imbalance**: Value added by trading a position intraday rather than leaving it to settle at the imbalance price.
- **Trade All Auctions (Simulation)**: Counterfactual comparing actual cost against trading the full forecast–position gap at every IDA auction.
- **System NIV**: Net volume the system operator must procure via the balancing mechanism; its sign determines whether imbalance is rewarded or penalised.
- **MAE / RMSE / Bias / MAPE**: Standard forecast accuracy measures applied to each stage (DA, IDA1–3).

Start: 13/07/2026

End: 19/07/2026

Aggregation: Daily

Load Data

DAM

RFIT

SOL

ALL GROUPS

TRADER PERFORMANCE

MARKET ANALYTICS

Portfolio Performance Summary

2026-07-13 to 2026-07-19

ACTUAL TRADED BALANCING COST

€-4.22/MWh

100% DA Forecast: €-5.09/MWh

vs Forecast: ▲ €0.87/MWh (Underperforming)

PRODUCTION

31,037 MWh

True Generation: 38,258 MWh

NET INCOME

€4,671,432

ExA: € Imb: €

Client Payment: €

CAPACITY FACTOR

17.6%

DISPATCH DOWN

18.1%

Curt: 6.5% Const: 11.6%

TOTAL BALANCING COST

€ (€-4.22/MWh)

Total DA Position Cost: € (€-5.09/MWh)

DISPATCH DOWN

6,911 MWh

Curt: 2,490 MWh Const: 4,420 MWh

Group Breakdown

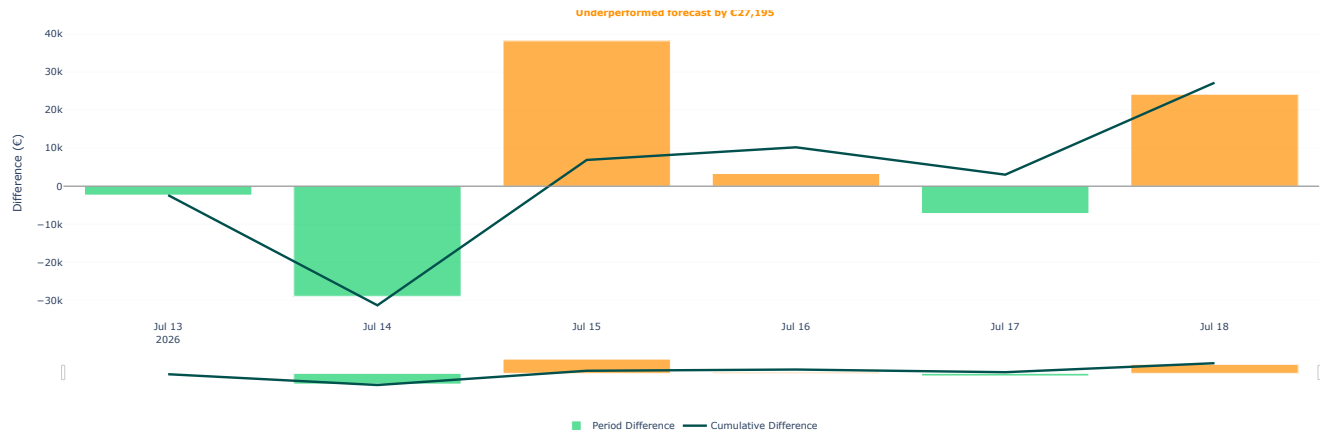
Group	Net Income	Production	Cost/MWh	Cap Factor	Dispatch Down
DAM		5,473 MWh	€-4.08	14.2%	22.0%
RFIT		5,753 MWh	€-2.54	11.7%	9.8%
SOL		19,810 MWh	€-4.74	20.2%	19.1%

Balancing Cost Scenarios

Balancing Cost Scenarios

Compares actual balancing costs against forecast scenarios (100%, 90%, 110% DA)





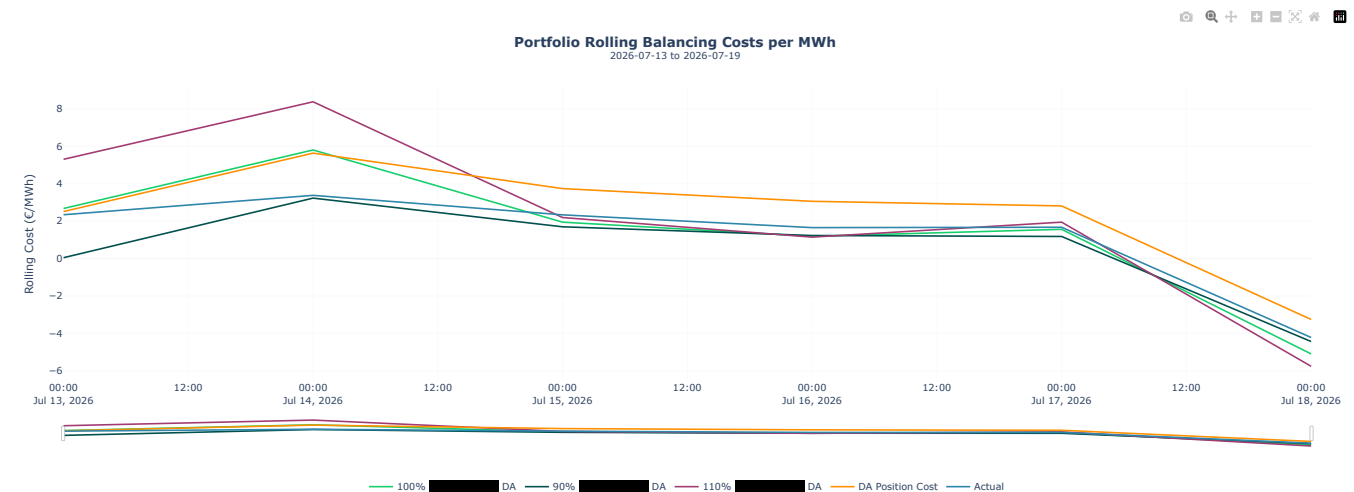
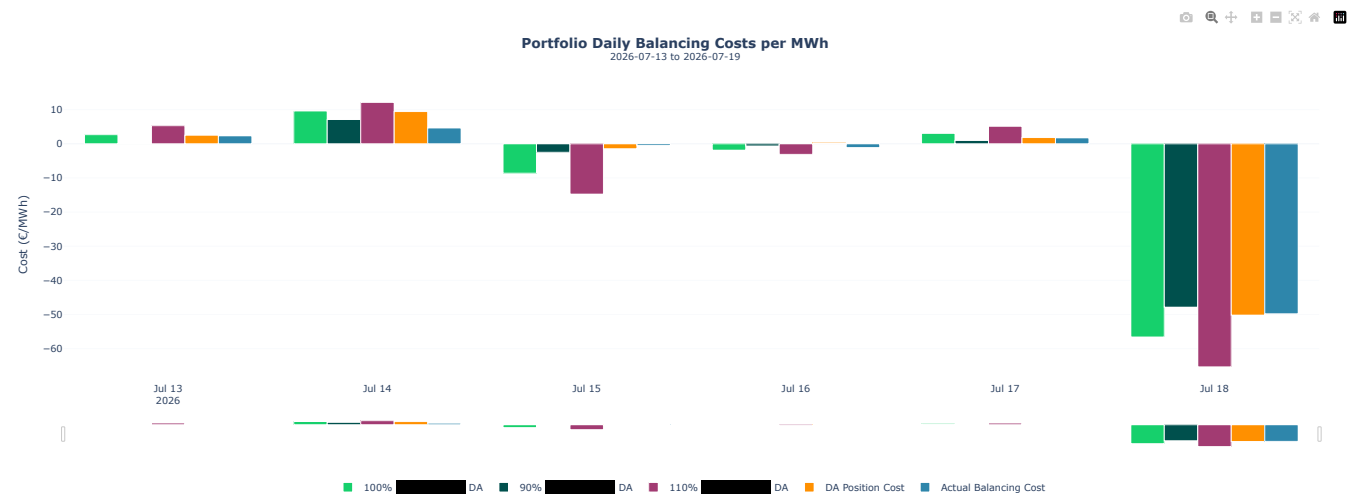
Portfolio Balancing Cost Summary (2026-07-13 to 2026-07-19)

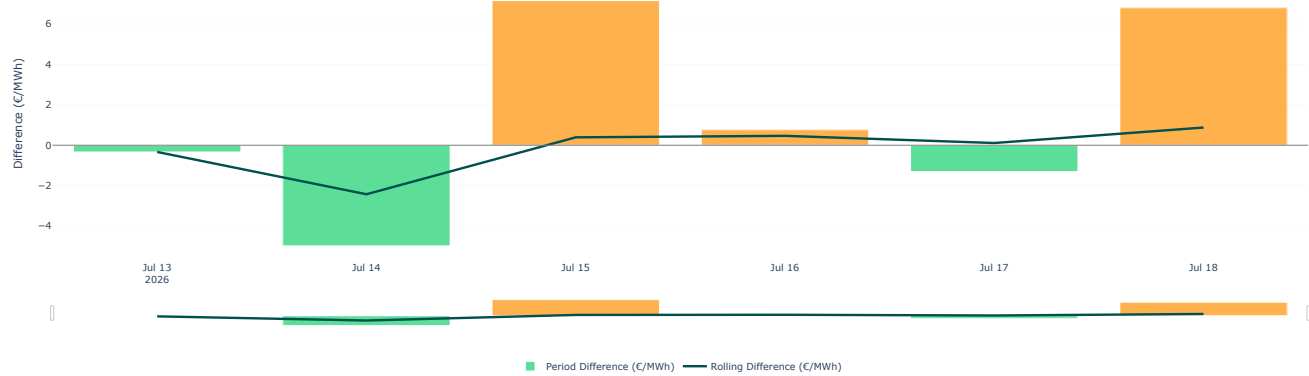
Scenario	Total (€)	vs Actual (€)
100% DA Forecast	€-158,099.72	€27,195.39
90% DA Forecast	€-137,567.29	€6,662.96
110% DA Forecast	€-178,632.15	€47,727.83
DA Position Cost	€-100,986.03	€-29,918.30
Actual Balancing Cost	€-130,904.33	—

Negative vs Actual = Actual cost less than scenario (outperformed) | Positive vs Actual = Actual cost more than scenario (underperformed)

Balancing Cost Scenarios (€/MWh)

Compares actual balancing costs per MWh against forecast scenarios (100%, 90%, 110% DA)





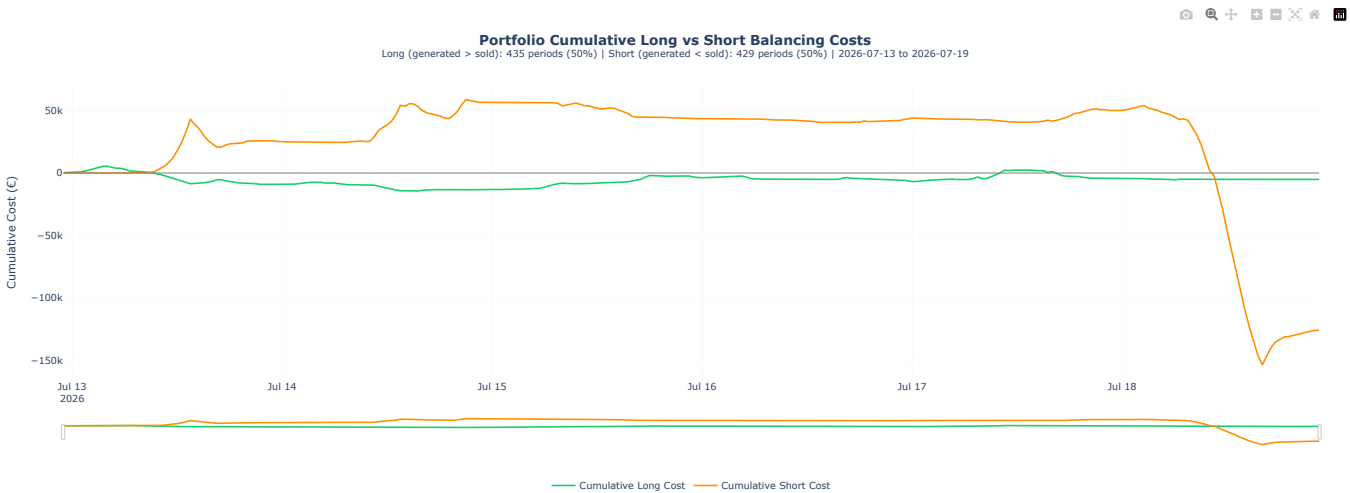
Portfolio Balancing Cost per MWh Summary (2026-07-13 to 2026-07-19)
Total Production: 31,036.93 MWh

Scenario	Cost (€/MWh)	vs Actual (€/MWh)
100% DA Forecast	€-5.09	€0.88
90% DA Forecast	€-4.43	€0.21
110% DA Forecast	€-5.76	€1.54
DA Position Cost	€-3.25	€-0.96
Actual Balancing Cost	€-4.22	—

Negative vs Actual = Actual cost per MWh less than scenario (outperformed) | Positive vs Actual = Actual cost per MWh more than scenario (underperformed)

Cumulative Long vs Short Balancing Costs

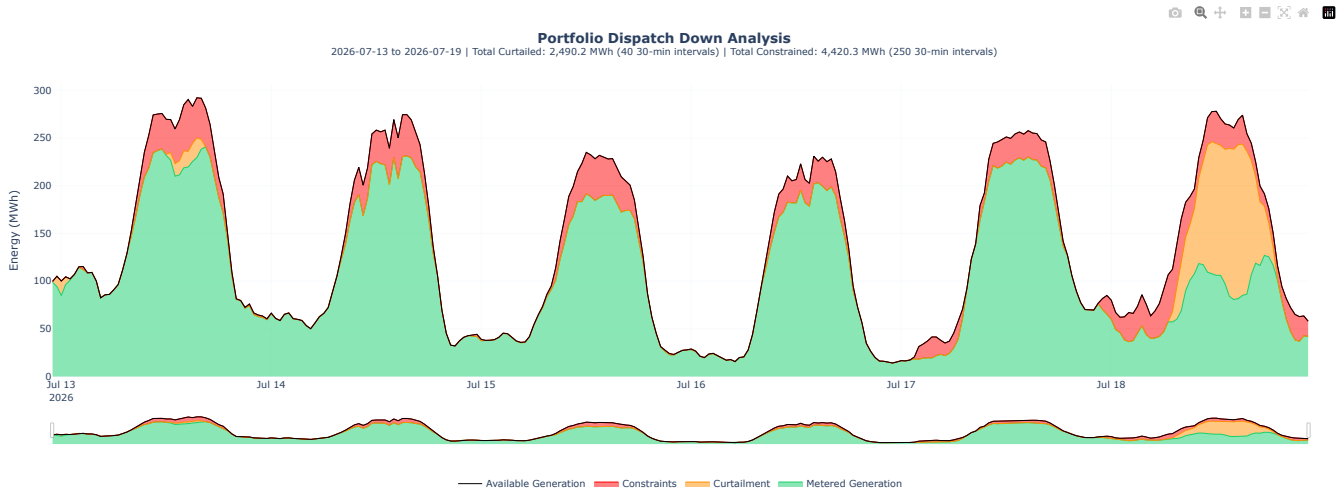
Compares cumulative balancing costs when long (generated > sold) vs short (generated < sold)

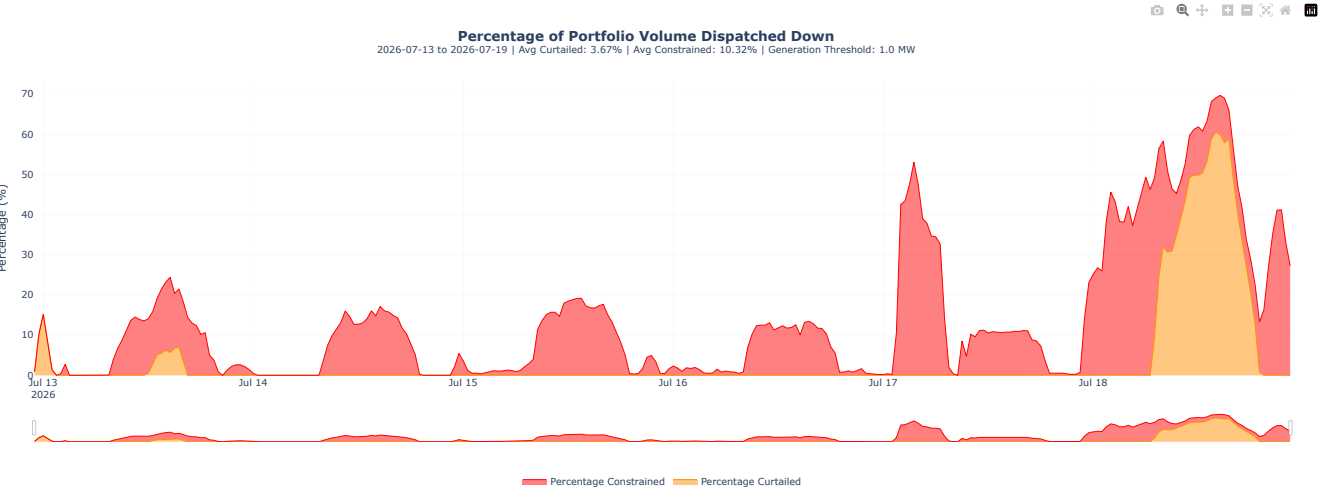


Curtailment

Dispatch Down Analysis

Volume and percentage analysis of curtailment and constraints on generation





Portfolio Dispatch Down Summary (2026-07-13 to 2026-07-19)

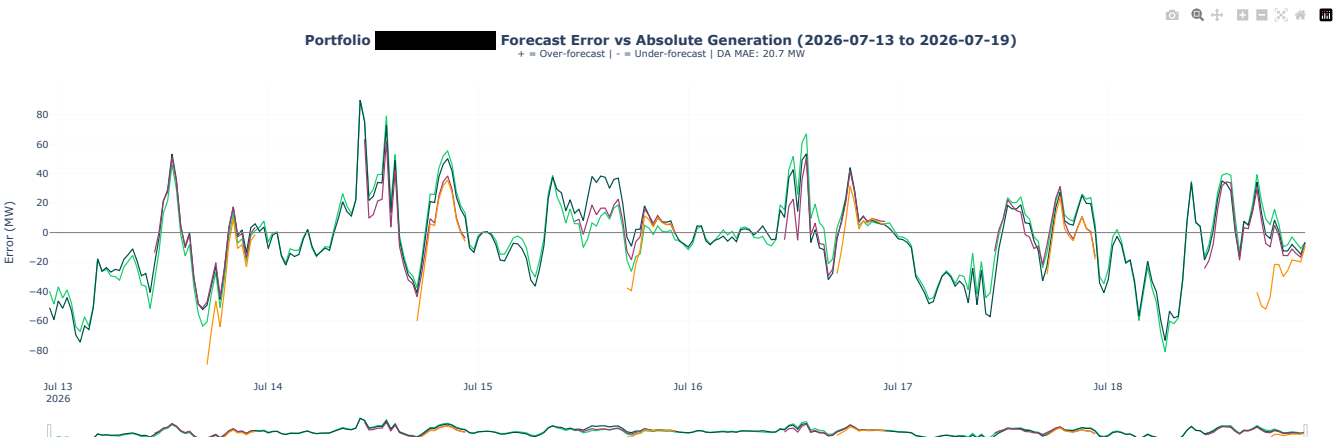
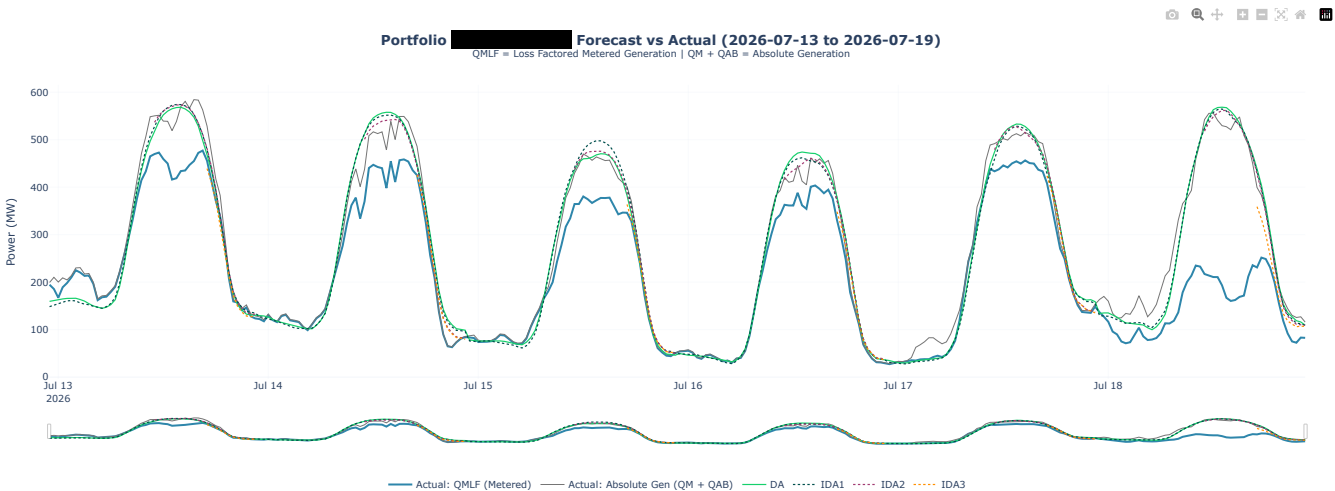
Metric	Curtailment	Constraints	Total
Volume (MWh)	2,490.2	4,420.3	6,910.5
% of Available Generation	6.51%	11.55%	18.06%
Avg % (when generating)	3.67%	10.32%	13.98%
Periods Affected	40 30-min intervals	250 30-min intervals	-
% of Periods Affected	13.9%	86.8%	-
Metered Generation	31,347.2 MWh		
Available Generation	38,257.7 MWh		
Periods with Generation (> 1.0 MW)	288 / 288 (100.0%)		

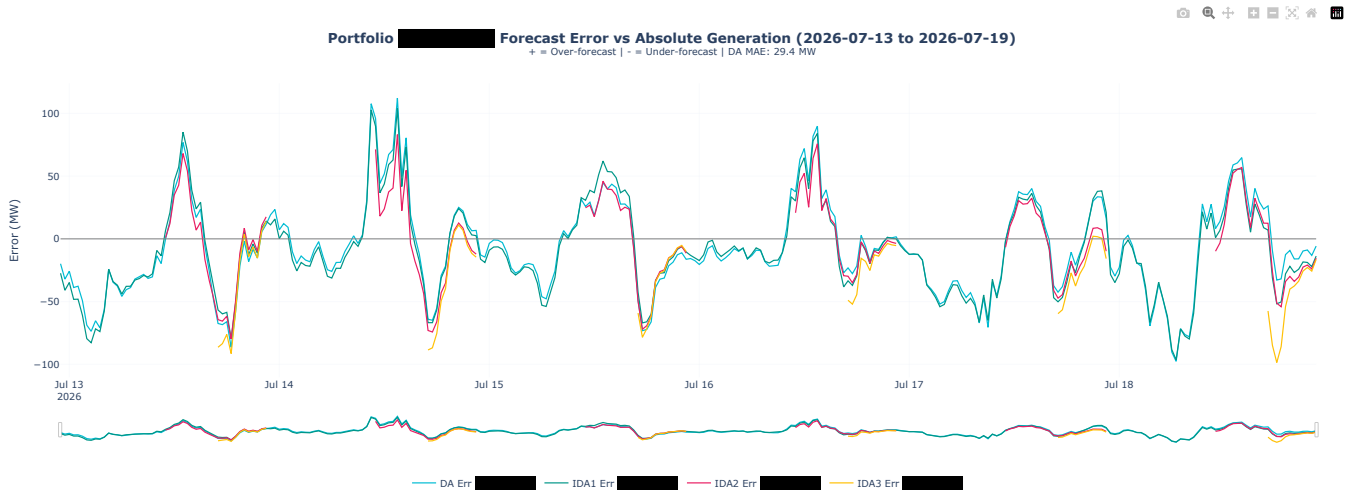
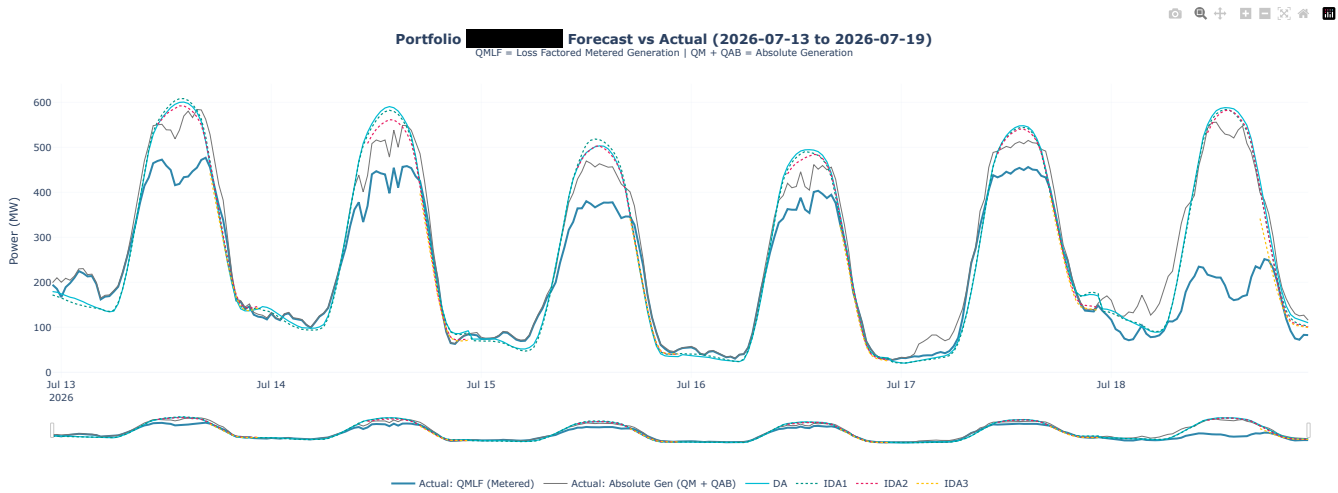
Note: Percentage calculations exclude periods where generation is below 1.0 MW threshold (filters meter noise/standby).

Forecasts & Accuracy

Forecast vs Actual Generation

Compares DA, IDA1, IDA2, IDA3 forecasts from [redacted] and [redacted] against metered generation (QMLF) and absolute generation (QM + QAB). Statistics compared against absolute generation only.





Portfolio [redacted] Forecast Accuracy (2026-07-13 to 2026-07-19)

Forecast	MAE (MW)	RMSE (MW)	Bias (MW)	MAPE (%)
DA	20.67	27.50	-4.12	10.9%
IDA1	21.15	27.61	-4.54	11.5%
IDA2	16.12	21.11	+3.43	6.6%
IDA3	19.56	26.74	-9.57	10.8%

All metrics calculated vs Absolute Generation (QM + QAB) | MAE: Mean Absolute Error | RMSE: Root Mean Square Error | Bias: Mean Error (+ = over-forecast, - = under-forecast) | MAPE: Mean Absolute Percentage Error

Portfolio [redacted] Forecast Accuracy (2026-07-13 to 2026-07-19)

Forecast	MAE (MW)	RMSE (MW)	Bias (MW)	MAPE (%)
DA	29.41	36.96	-7.78	15.5%
IDA1	29.99	37.32	-9.62	15.8%
IDA2	27.85	34.39	-1.28	9.5%
IDA3	33.65	43.83	-32.22	15.7%

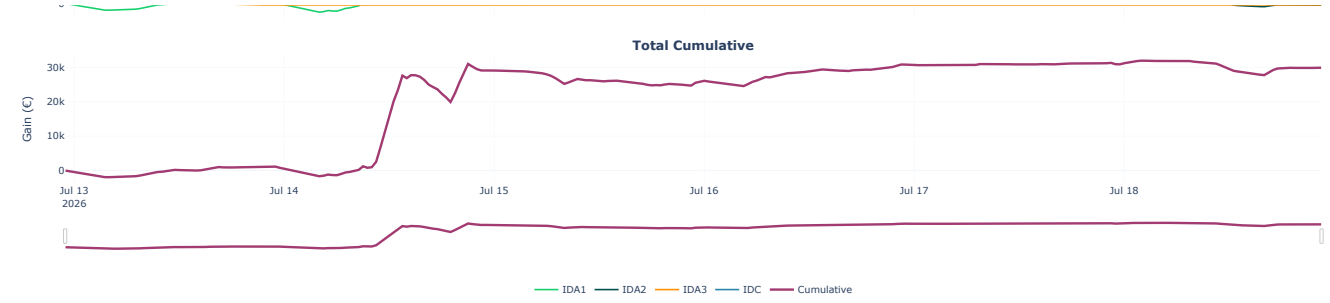
All metrics calculated vs Absolute Generation (QM + QAB) | MAE: Mean Absolute Error | RMSE: Root Mean Square Error | Bias: Mean Error (+ = over-forecast, - = under-forecast) | MAPE: Mean Absolute Percentage Error

IDA Trading Performance

IDA Net Gain vs. Imbalance

Compares cumulative gain/loss from IDA trading versus leaving positions to imbalance



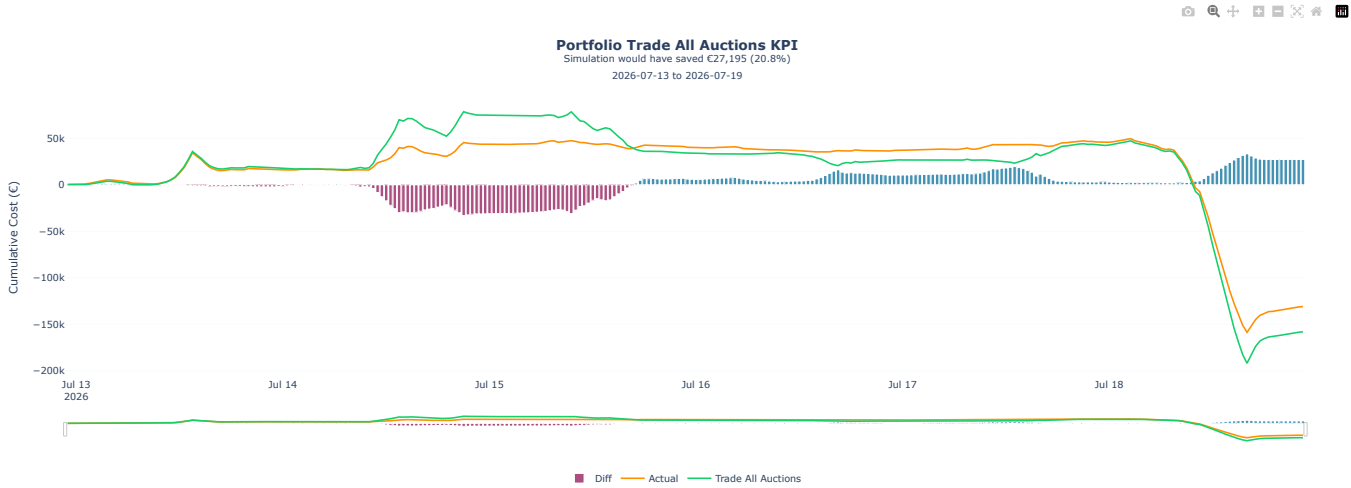


Portfolio IDA Net Gain Summary (2026-07-13 to 2026-07-19)

Auction	Total Gain (€)	% of Total
IDA1	€30,907.88	103.3%
IDA2	€-517.28	-1.7%
IDA3	€-455.87	-1.5%
IDC	€0.00	0.0%
Total	€29,934.73	100%

Trade All Auctions Simulation

Compares actual balancing costs vs simulated costs if trading all IDA auctions to forecast, limited to 30% of available volume to reflect realistic trading constraints



Overview

Summary Statistics (Exportable)

Detailed summary tables for data export. Use the Plotly toolbar to download as image or data.

Portfolio Summary (2026-07-13 to 2026-07-19)

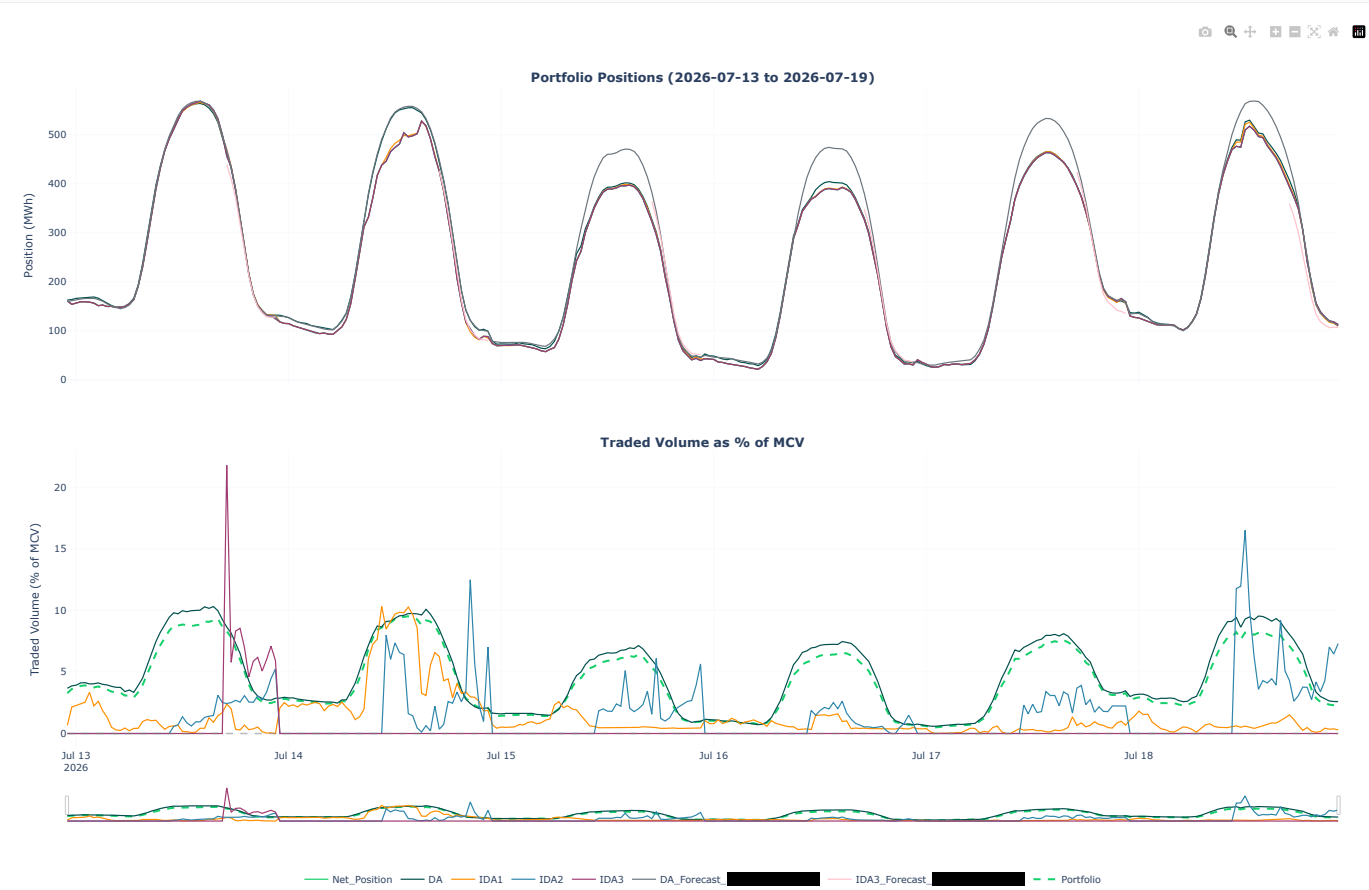
Metric	Portfolio Total
Ex-Ante Income (€)	€██████████
Imbalance Income (€)	€██████████
Net Income (€)	€4,671,432.14
Client Payment (€)	€██████████
Production (MWh)	31,036.93
Capacity Factor (% , weighted avg)	17.57
Dispatch Down Rate (%)	18.06
- Curtailment Rate (%)	6.51
- Constraint Rate (%)	11.55
DA Position Cost (€/MWh)	€-3.25
100% DA Forecast Bal Cost (€/MWh)	€-5.09
90% DA Forecast Bal Cost (€/MWh)	€-4.43
110% DA Forecast Bal Cost (€/MWh)	€-5.76
Actual Balancing Cost (€/MWh)	€-4.22
Total Balancing Cost (€)	€██████████

Group Breakdown (2026-07-13 to 2026-07-19)

Group	Net Income	Production (MWh)	Cost (€/MWh)	Cap Factor (%)	Dispatch Down (%)	Total Cost
DAM	€853,608.22	5,473.41	€-4.08	14.21	22.02	€██████████
RFIT	€848,822.84	5,753.31	€-2.54	11.74	9.81	€██████████

Trading Positions

Shows trading positions across DA, IDA1, IDA2, IDA3 with forecast and market clearing volume comparison



Portfolio Position Summary (2026-07-13 to 2026-07-19)

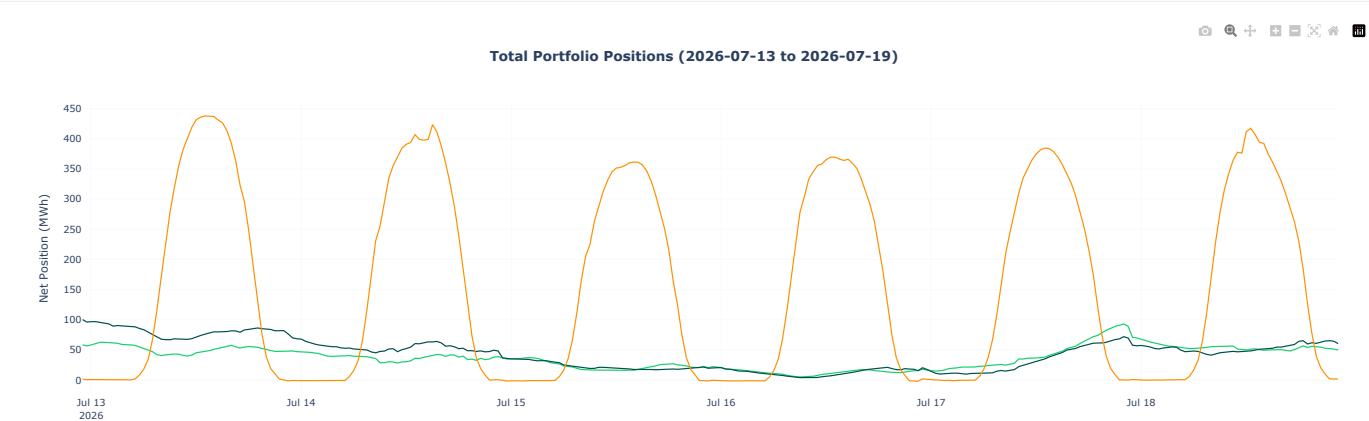
Position	Mean (MWh)	Min (MWh)	Max (MWh)	Std Dev
Net_Position	239.20	21.40	568.00	164.79
DA	246.89	25.20	563.50	167.92
IDA1	239.69	21.40	565.60	165.42
IDA2	239.17	21.40	568.00	164.84
IDA3	239.20	21.40	568.00	164.79
DA_Forecast	261.56	29.79	568.38	181.76
IDA3_Forecast	195.17	35.98	439.11	115.91

Traded Volume as % of MCV

Auction	Mean (% MCV)	Min (% MCV)	Max (% MCV)
Portfolio	4.31%	0.50%	9.53%
DA	4.74%	0.59%	10.31%
IDA1	1.31%	0.00%	10.32%
IDA2	1.49%	0.00%	16.51%
IDA3	0.32%	0.00%	21.79%

Portfolio Positions

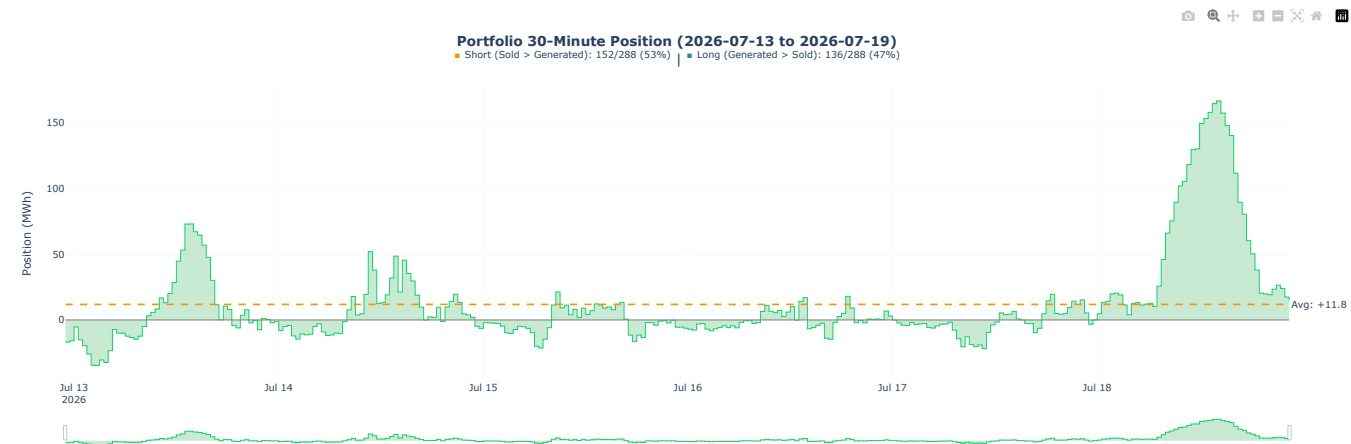
Shows combined portfolio positions across all groups



DAM RFIT SOL

Mean Long/Short Position

Shows average position direction over time. Positive = Short (sold > generated), Negative = Long (generated > sold). Identifies systematic trading biases.



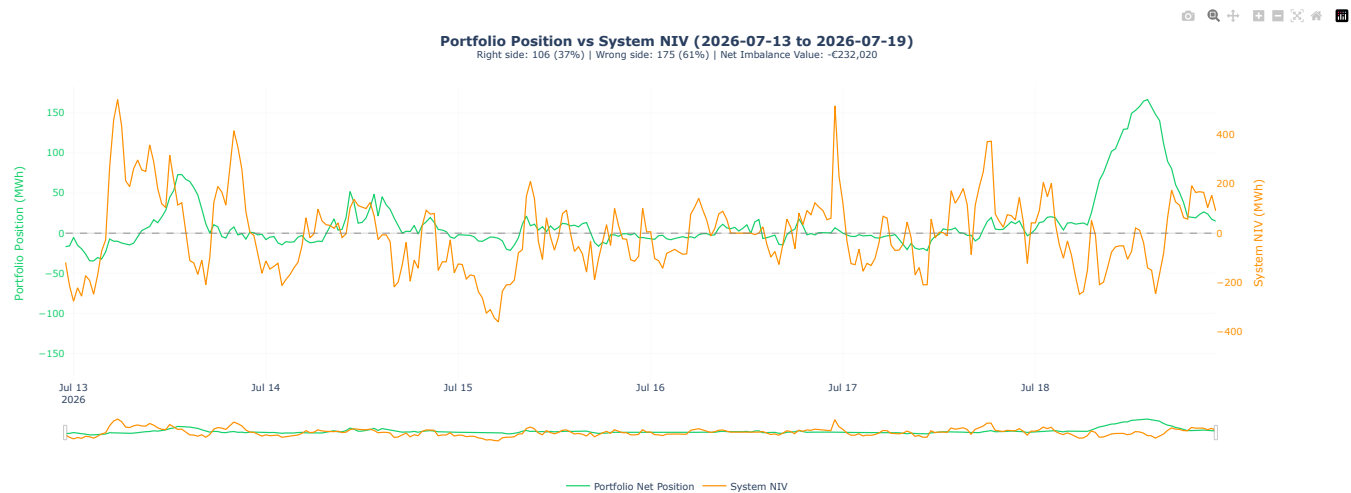
Portfolio Position Summary (2026-07-13 to 2026-07-19)

Metric	Value	Percentage
Short Periods (QEX > QMLF)	152 periods	52.8%
Long Periods (QMLF > QEX)	136 periods	47.2%
Total Periods	288 periods	100%
Average Position	+11.83 MWh	Short Bias

Trading Bias: Tendency to sell more than generated

Portfolio Position vs System NIV

Compares portfolio net position against system imbalance



Start: 13/07/2026

End: 19/07/2026

Aggregation: Daily

Load Data

DAM

RFIT

SOL

ALL GROUPS

TRADER PERFORMANCE

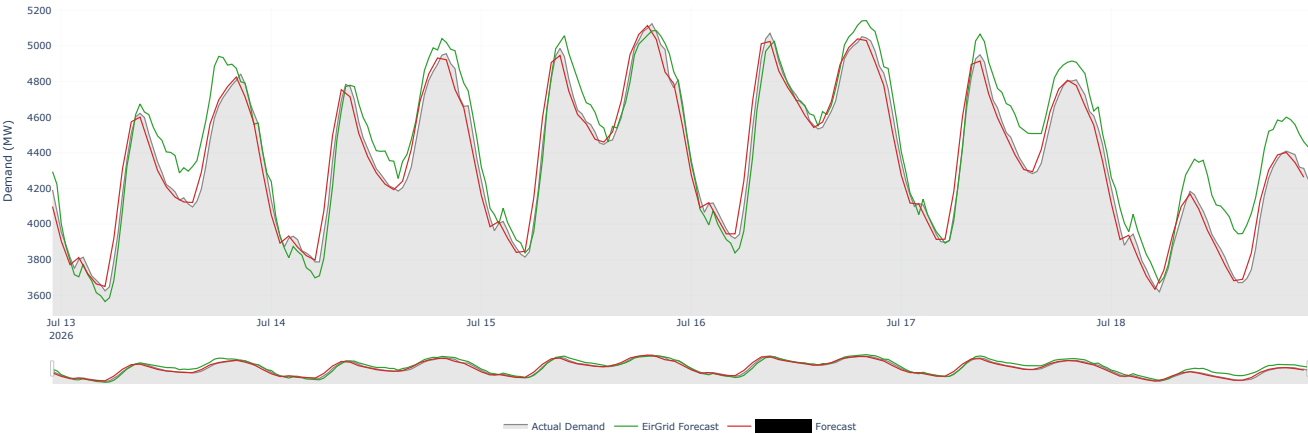
MARKET ANALYTICS

Market Overview

Demand Overview

Shows actual demand alongside EirGrid and demand forecasts

IE Demand (2026-07-13 to 2026-07-19)



Actual Demand

EirGrid Forecast

Forecast

Demand Statistics (2026-07-13 to 2026-07-19)

Series	Mean (MW)	Min (MW)	Max (MW)	Std Dev (MW)
Actual Demand	4365	3620	5124	397
EirGrid Forecast	4435	3565	5142	410
Forecast	4365	3633	5114	394

Forecast Error vs Actual

EirGrid Forecast

MAE99 MW

RMSE123 MW

Forecast

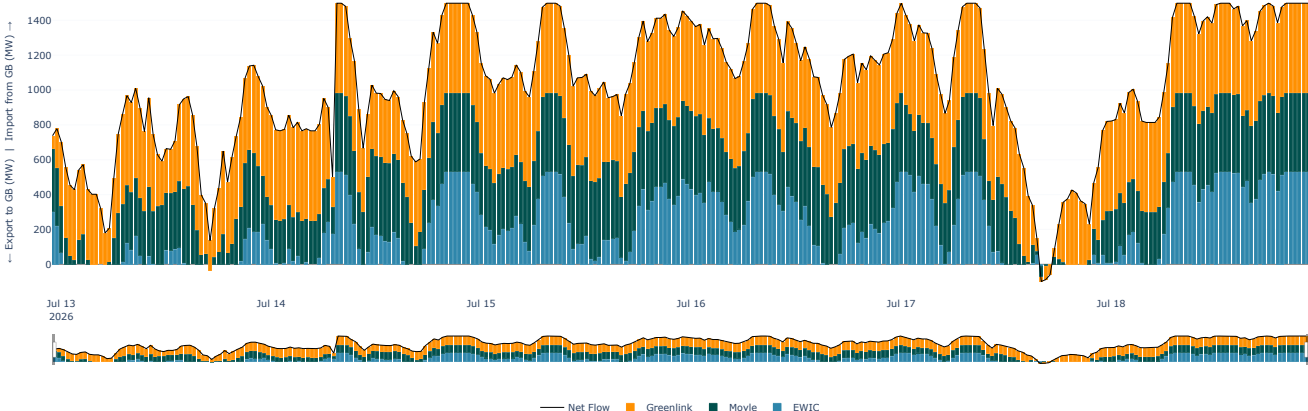
MAE53 MW

RMSE65 MW

IE-GB Interconnector Flows

Looks at the flows between IE and GB across the Greenlink, EWIC, and Moyle interconnectors

IE-GB Interconnector Flows (2026-07-13 to 2026-07-19)



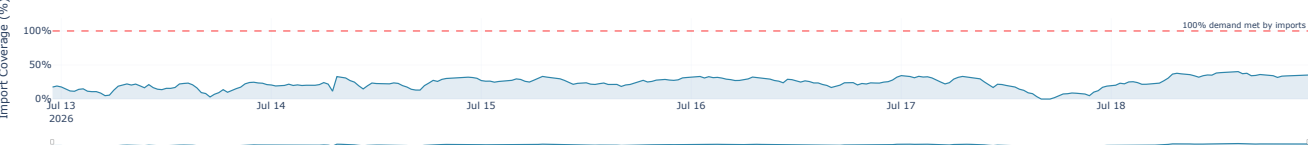
Net Flow

Greenlink

Moyle

EWIC

Import Coverage of Irish Demand (2026-07-13 to 2026-07-19)



100% demand met by imports

Interconnector Flow Statistics (2026-07-13 to 2026-07-19)

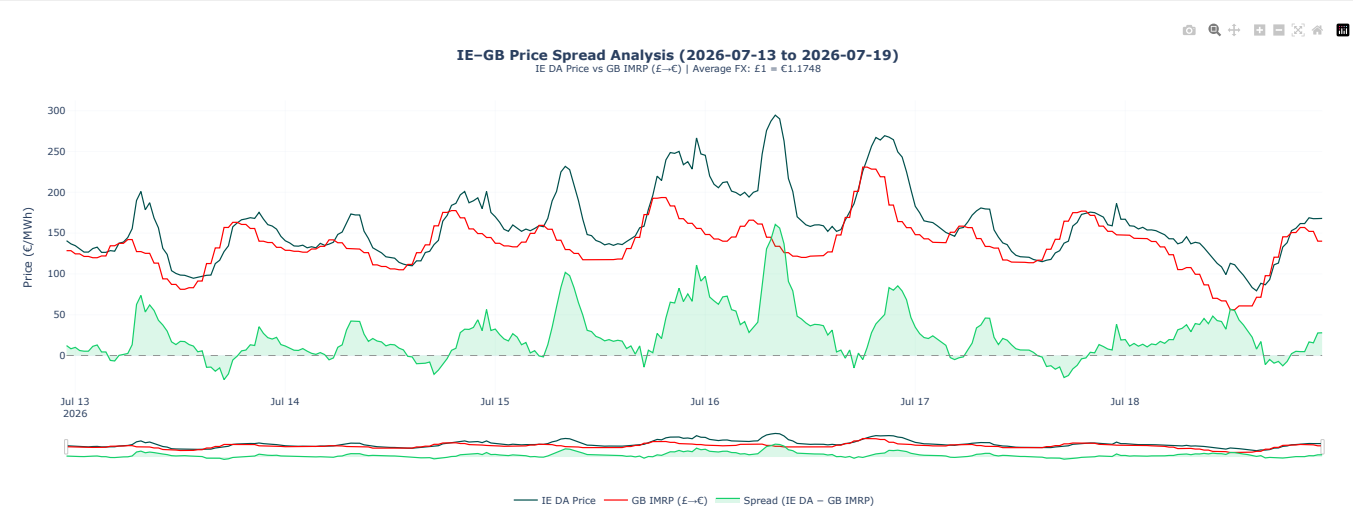
Interconnector	Mean (MW)	Min (MW)	Max (MW)	Std Dev (MW)
EWIC	222.6	-14.0	532.0	199.1
Moyle	344.0	-58.0	452.0	137.7
Greenlink	465.0	-78.0	514.0	102.1

Import Coverage Summary

Average Import Coverage	Peak Import Coverage	% of Time Imports > 25% Demand
23.7%	40.3%	44.4%

IE-GB Price Spread

Shows IE DA price vs GB IMRP (£→€ converted) and the spread between them



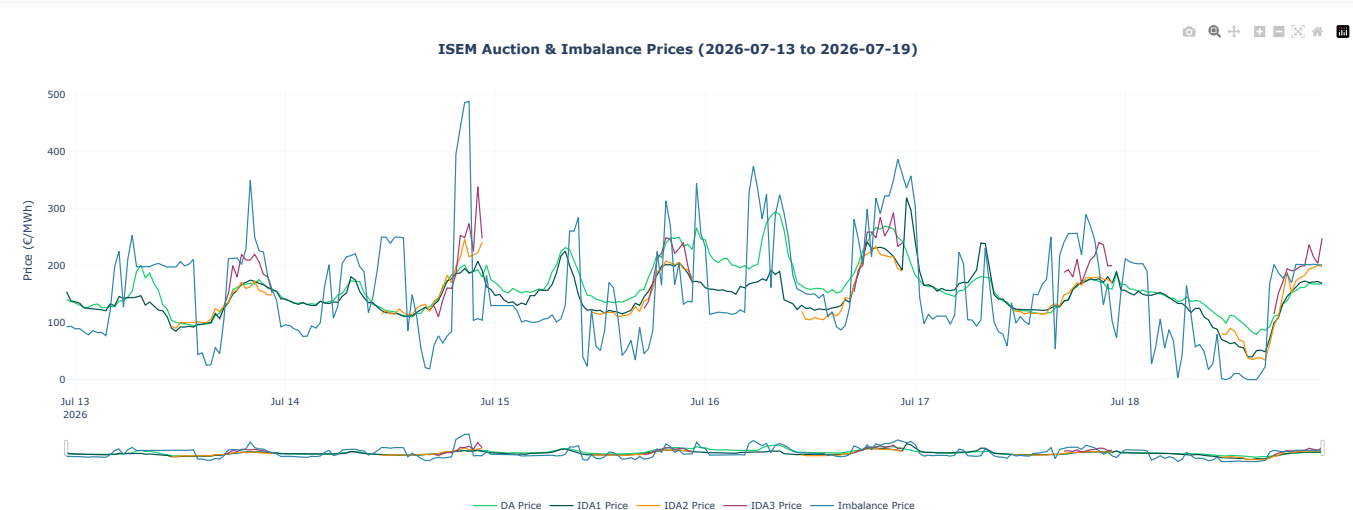
IE-GB Price Spread Statistics (2026-07-13 to 2026-07-19)

Metric	IE DA (€/MWh)	GB IMRP (€/MWh)	Spread (€/MWh)
Mean	€161.01	€136.60	€24.41
Min	€79.31	€56.14	€-29.54
Max	€294.50	€230.85	€160.66
Std Dev	€42.31	€29.84	€30.81
IE > GB (periods)	238 of 288 (82.6%)		
GB > IE (periods)	50 of 288 (17.4%)		
FX Rate Used	£1 = €1.1748 (avg over period)		

ISEM Market Prices

ISEM Auction Prices

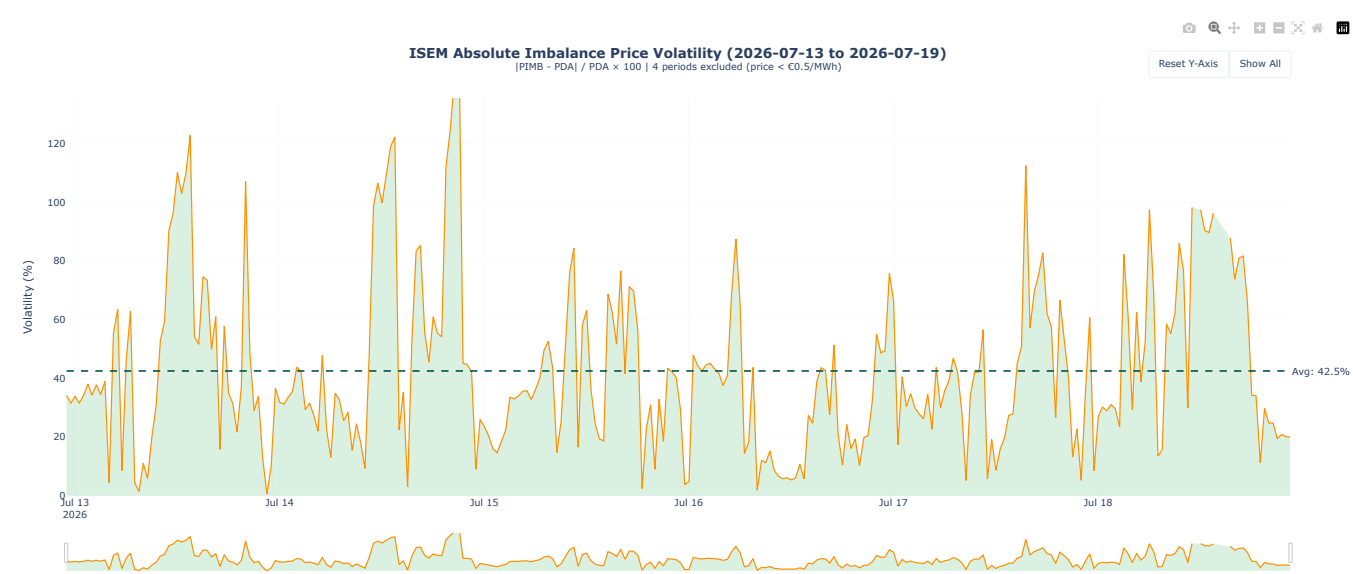
Shows ISEM DA, IDA1, IDA2, IDA3 and Imbalance prices over time



Price	Mean (€/MWh)	Min (€/MWh)	Max (€/MWh)	Std Dev (€/MWh)
DA Price	€161.01	€79.31	€294.50	€42.31
IDA1 Price	€147.07	€40.00	€319.38	€38.40
IDA2 Price	€143.20	€33.96	€246.08	€46.23
IDA3 Price	€205.31	€110.00	€338.65	€44.20
Imbalance Price	€155.43	€0.00	€488.32	€91.72

ISEM Imbalance Price Volatility

Shows ISEM imbalance price spread and volatility analysis



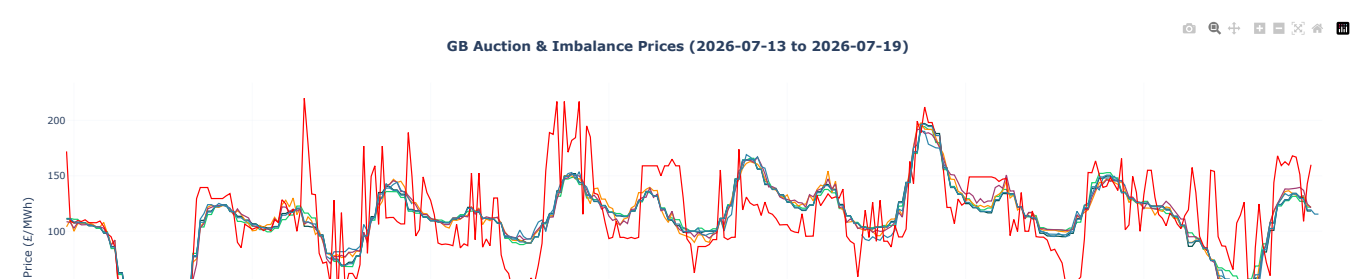
Imbalance Volatility Statistics (2026-07-13 to 2026-07-19)

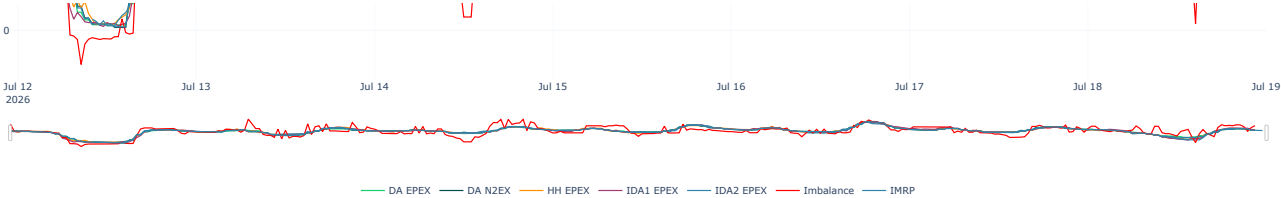
Metric	Value
Average Volatility	42.48%
Maximum Volatility	160.88%
Minimum Volatility	0.50%
Standard Deviation	28.75%
Excluded Periods	4 of 208 (price < €0.5/MWh)

GB Market Prices

GB Auction Prices

Shows GB DA, HH, IDA1, IDA2, Imbalance and IMRP prices over time



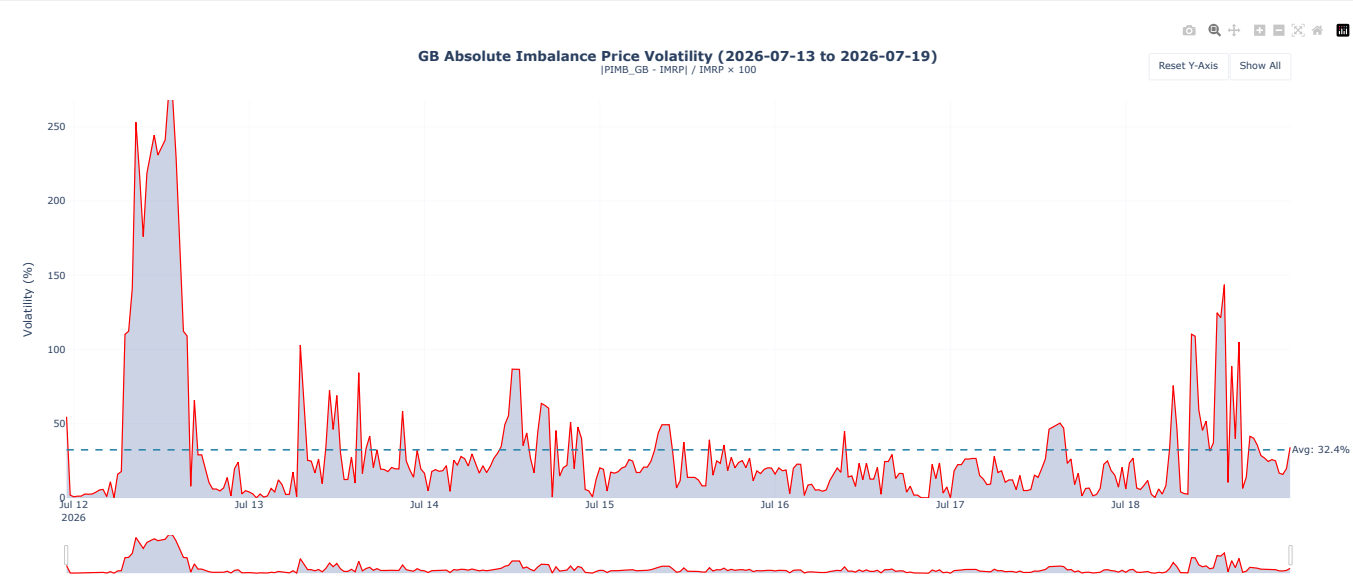


GB Price Statistics (2026-07-13 to 2026-07-19)

Price	Mean (£/MWh)	Min (£/MWh)	Max (£/MWh)	Std Dev (£/MWh)
DA EPEX	£109.93	£4.43	£194.34	£33.01
DA NZEX	£109.60	£2.49	£197.17	£33.67
HH EPEX	£110.12	£5.06	£197.64	£34.19
IDA1 EPEX	£109.96	£3.53	£192.00	£35.47
IDA2 EPEX	£107.52	£3.99	£190.20	£42.10
Imbalance	£109.65	£-30.84	£220.00	£45.92
IMRP	£109.74	£3.14	£196.51	£33.38

GB Imbalance Price Volatility

Shows GB imbalance price spread vs IMRP (Weighted DA reference price)



GB Imbalance Volatility Statistics (2026-07-13 to 2026-07-19)

Metric	Value
Average Volatility	32.40%
Maximum Volatility	273.95%
Minimum Volatility	0.01%
Standard Deviation	47.06%

Start: 13/07/2026

End: 19/07/2026

Aggregation: Daily

Load Data

DAM

RFIT

SOL

ALL GROUPS

TRADER PERFORMANCE

MARKET ANALYTICS

Trader Performance Summary

2026-07-13 to 2026-07-19

TEAM TOTAL PROFIT

€4,776

IDA: €4,776 IDC: €0

Efficiency: €8.37/MWh

TOP PERFORMER

€7,020 profit

TOTAL VOLUME TRADED

571 MWh

IDA1 PNL

€3,640

IDA2 PNL

€1,320

IDA3 PNL

€-184

IDC PROFIT

€0

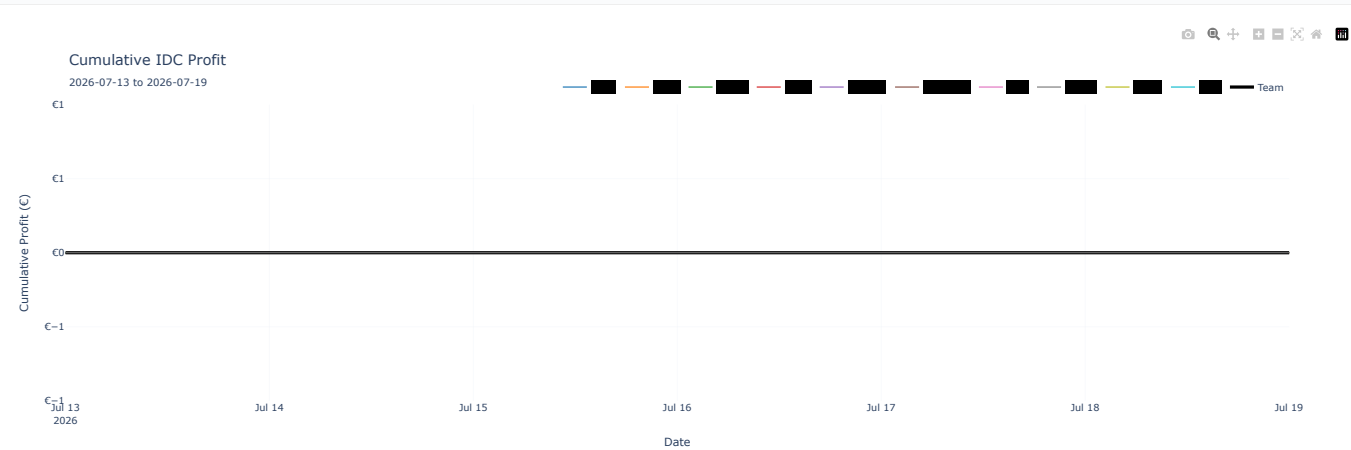
Trader Breakdown

Trader	IDA1 €	IDA2 €	IDA3 €	IDC €	Total €	MWh	€/MWh
	5,270	1,935	-184	0	7,020	406	17.31
	0	0	0	0	0	0	0.00
	0	0	0	0	0	0	0.00
	0	0	0	0	0	0	0.00
	0	0	0	0	0	0	0.00
	0	0	0	0	0	0	0.00
	0	0	0	0	0	0	0.00
	0	0	0	0	0	0	0.00
	-706	0	0	0	-706	40	-17.81
	-923	-615	0	0	-1,538	125	-12.27

Trading Decisions Analysis

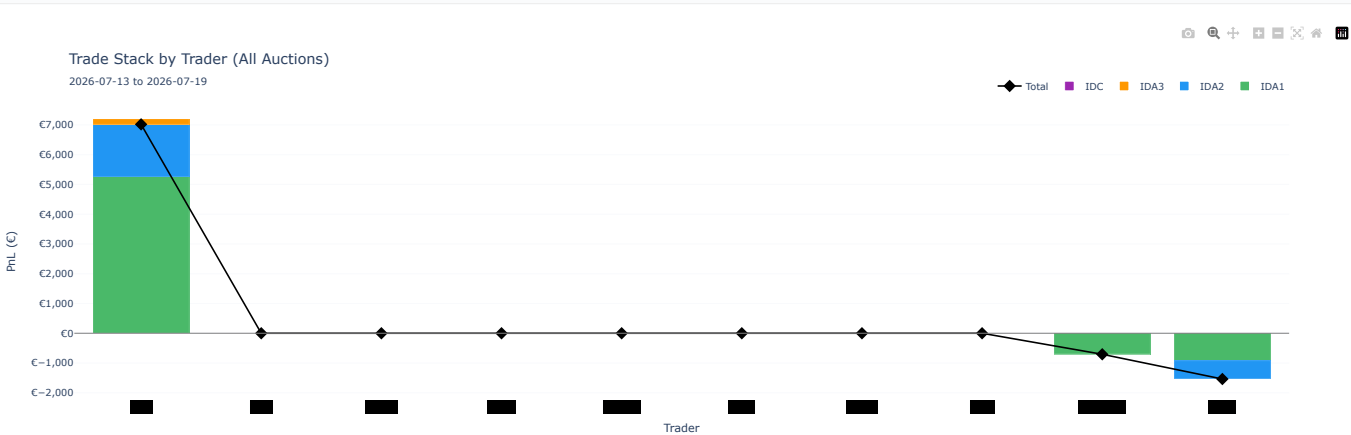
Cumulative IDC Profit

Cumulative IDC trading profit over time by trader

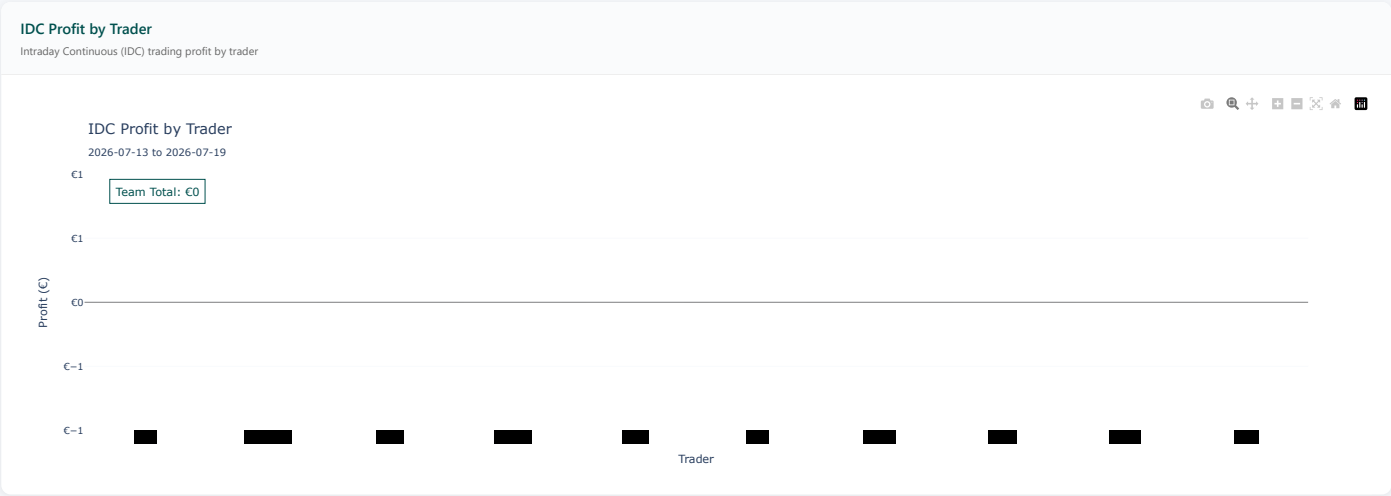
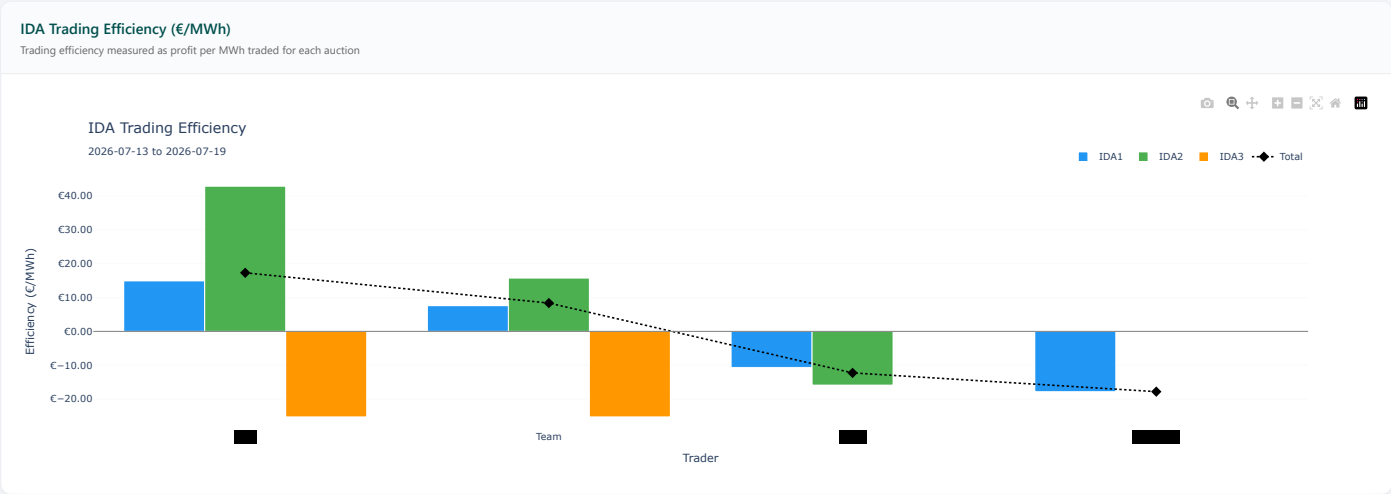
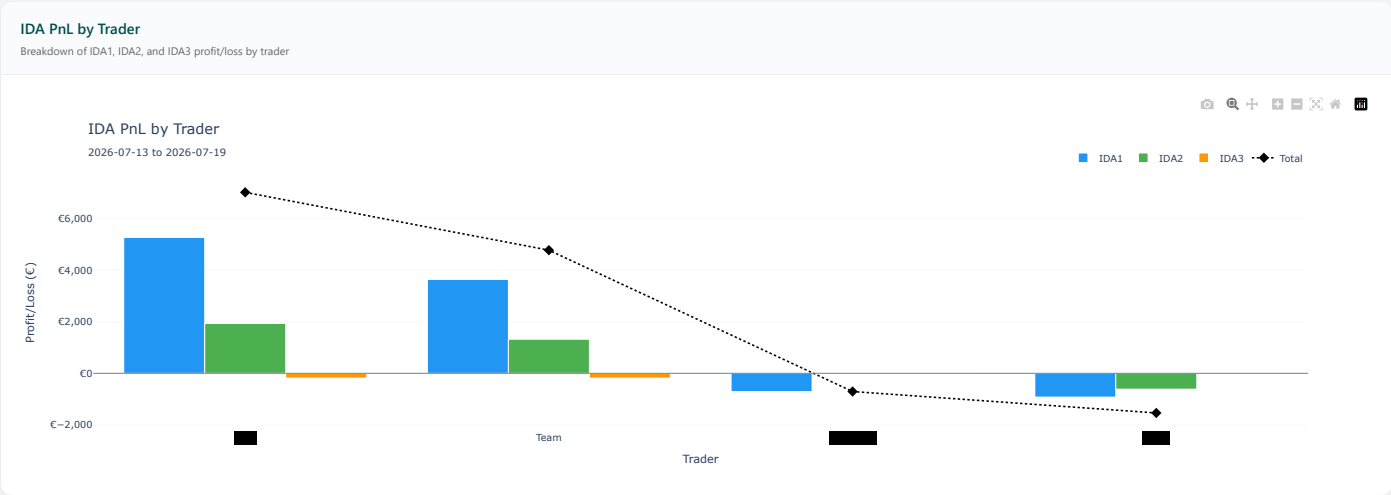


Trade Stack by Trader

Individual trade PnL stacked by trader across all auctions (IDA1, IDA2, IDA3, IDC)



Trader Overview



Performance Over Time



3.4 Skills and Learning Outcomes

The KPI dashboard project developed a broad range of technical skills across full-stack web development, data engineering, and visualisation. Designing and building the Flask backend provided hands-on experience with REST API architecture, server-side routing, and managing application state, while constructing the frontend in HTML, CSS, and JavaScript developed an understanding of how data is consumed and rendered in a browser environment. Working with Plotly for chart generation strengthened my ability to produce clear, interactive visualisations that communicate complex data effectively to both technical and non-technical audiences.

From a data engineering perspective, the project required writing extensive SQL queries to extract and join data from multiple database tables, and building a robust data processing pipeline in Python using Pandas to clean, aggregate, and transform raw market data into the metrics and structures required by each chart. Designing the modular chart architecture — where each chart is a self-contained module that can be added or removed without modifying the core application — developed an appreciation for writing maintainable, scalable code that others can work with after handover.

Presenting the completed dashboard to the CEO and wider team was a significant professional milestone. Preparing for this presentation required careful consideration of how to frame technical work for a non-technical audience, focusing on the business value delivered rather than implementation detail. The experience reinforced the importance of clear communication and the ability to tailor the depth and language of a presentation to the audience.

Finally, producing comprehensive documentation on Confluence to support the dashboard's ongoing maintenance after the internship developed technical writing skills and reinforced the importance of knowledge transfer in a professional environment. Overall, this project provided the most sustained and technically diverse learning experience of the internship, spanning backend development, frontend design, database querying, data processing, visualisation, presentation, and documentation.

4 Solar Trading Strategy Development

This section presents analysis conducted in collaboration with a senior trader to explore more effective approaches to trading the company's solar assets. The current strategy of selling the majority of solar positions at the day-ahead auction may not always be optimal, and this work investigates whether dynamically allocating volume across DA, IDA1, IDA2, and IDA3 auctions based on forecast accuracy and price signals could improve revenue outcomes. Strategies were back-tested across the last 12 months to identify the key drivers that influence which auction stage yields the best result.

The longer term goal is to develop a predictive model that determines the optimal volume split across auctions, giving traders a data-driven edge when allocating volume. The analysis was conducted in Python and SQL, with findings presented to the wider team on Thursday 30th April.



Objective:

- The overall objective of this project is to investigate how we could trade our solar positions better across different auctions.
- Currently, we have a tendency to just sell most of our solar positions at the day-ahead auction and either modify positions at later auctions or just settle our positions in the balancing market.
- This analysis investigates if this is the best strategy, and if there are other more strategic ways to manage our positions in order to maximise our revenue.
- Analysis and strategies were back tested across the last 12 months to see if we could have done things differently – and give a framework for how we could possibly change how we do things going forward.
- The first section of the slides focuses on forecasting accuracy analyses, followed by strategy.

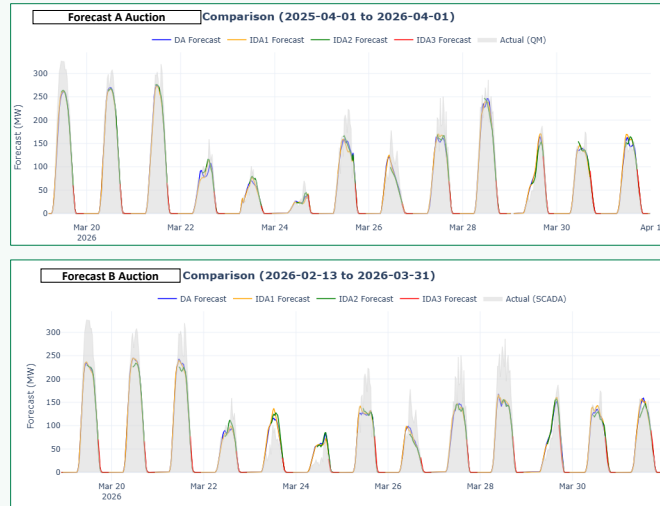


In order to once again protect any sensitive data, some of the details within the presentation were redacted. The first slide gave an introduction into what the following analysis would entail. Some of the key statistical measurements were defined for anyone who wasn't too familiar with them. The error was also defined to show the sign convention.

The total market clearing volumes for each auction were also compared with our traded activity to explore how much influence we have on the market.

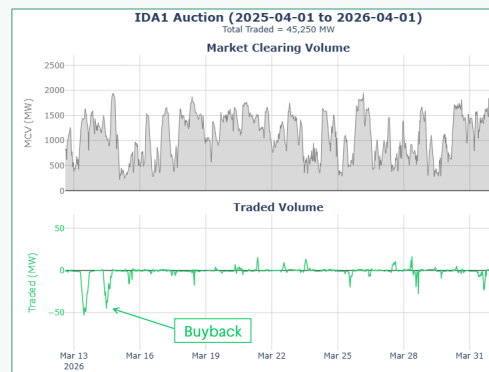
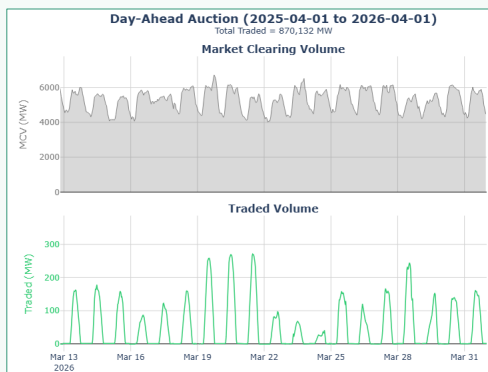
Solar Forecast Accuracy

- The following slides look at the performance and comparison of the two forecast providers, Forecast A vs Forecast B.
- Dispatch curtailment and constraint volumes were ignored as the generation forecasts don't take dispatch down into consideration.
- Some of the following key indicators are used to quantify forecast accuracy:
 - ME/MBE = mean error/bias
 - MAE = mean absolute error
 - RMSE = root mean squared error
 - MAPE = mean absolute percentage error
- Error Sign Convention:**
- Error = Forecast – Metered Generation**
 - Negative = Long/Under-forecasting
 - Positive = Short/Over-forecasting



Current Auction Trading Volumes

Auction	MCV (MW)	Traded (MW)	Sold (MW)	Bought (MW)	% of MCV	Buyback %
Day-Ahead	44,459,418.4	870,131.6	867,190.8	2,940.8	1.96	0.34
IDA1	20,077,865.5	45,250.5	14,503.0	30,747.5	0.23	67.95
IDA2	2,225,002.5	27,731.1	10,083.2	17,647.9	1.25	63.64
IDA3	515,106.0	5,762.9	1,995.8	3,767.1	1.12	65.37



4.1 Filtering the Data

Before conducting the analysis, the forecast data required filtering as it included many periods of zero generation such as night time periods. Initially a static time-based mask was applied, defining fixed windows for each auction stage during which solar generation was expected to occur. However, due to the solar path shifting throughout the year, the periods of active generation vary significantly across seasons, making a fixed mask insufficient.

To address this, a dynamic mask was introduced based on the solar ramp, defined as the derivative of the solar curve or the change in generation between two consecutive 30-minute periods. A ramp threshold of 0.2 MW was applied, meaning only periods where the ramp indicated active generation was occurring were included in the analysis. This ensured that error statistics were calculated only over meaningful trading periods, yielding more accurate and representative results across all auction stages.

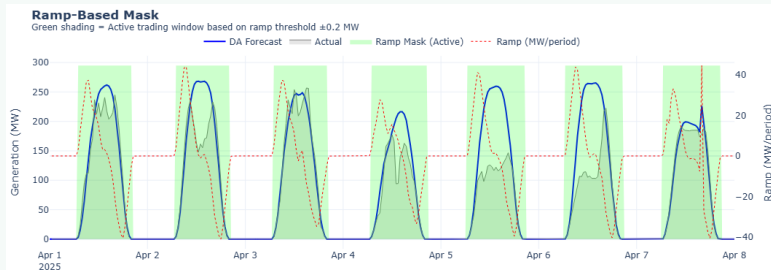
Filtering the Forecasts

- At first, all periods were being analysed (including night), which caused in the error statistics to be inaccurate.
- To improve accuracy, a static mask was used to try and filter out the periods of no solar generation.
- However, due to the solar path changing throughout the year, the periods where solar generation equals 0 MW varies.
- In order to handle this, a dynamic mask approach was used.
- The solar ramp was defined as being the difference in generation between two consecutive 30-min periods (or the derivative of the solar curve).
- A ramp threshold was set to 0.2 MW.
- This meant that only periods where solar generation was occurring were to be analysed, ultimately yielding more accurate results.

Auction:	Static Mask Definition:
DA & IDA1 Mask:	06:00 – 23:00
IDA2 Mask	11:00 – 23:00
IDA3 Mask	17:00 – 23:00

$$\text{Ramp} = \frac{d}{dt}(\text{Solar Curve})$$

Auction:	Dynamic Mask Definition:
DA & IDA1 Mask:	[0.2 MW < Ramp < 0.2 MW]
IDA2 Mask	11:00 – [Ramp < 0.2 MW]
IDA3 Mask	17:00 – [Ramp < 0.2 MW]



Forecast accuracy metrics for Forecast A were calculated across each auction stage using three masking approaches — no filter, a static time-based mask, and a dynamic ramp-based mask — to illustrate the impact of each on the reported statistics. The ramp-based mask, which restricted analysis to periods of active generation, produced notably improved results and was more representative of true forecast performance.

Forecast A Accuracy

- The expected outcome for the error distribution would logically be DA to IDA3, least to most accurate as we get closer to delivery.
- However this was not the case, as seen in the *Error (All Hours)* table. The error data was being flattened by the 0 and NaN periods within the data outside of the solar hours.
- When the static mask was applied, the distribution became more aligned with what was expected.
- When the dynamic mask was applied, the results became even more accurate. This led to the adjusted error results being much more in-line with expectation.
- A positive mean error across DA, IDA1, and IDA2 suggest we are usually long in these auctions, compared to being slightly short in IDA3.
- IDA3 was by far the most accurate forecast, despite having a high MAPE (likely caused by small division, yielding high numbers).
- IDA2 had the highest MAE and RMSE, likely due to it overcompensating for volatility.

Error (All Hours)

Forecast	ME (MW)	MAE (MW)	RMSE (MW)	MAPE
DA	2.74	10.82	22.97	71.1%
IDA1	2.34	10.6	22.65	69.7%
IDA2	2.83	15.5	27.55	62.0%
IDA3	-0.42	4	10.08	96.2%

Adjusted Error (Static Mask)

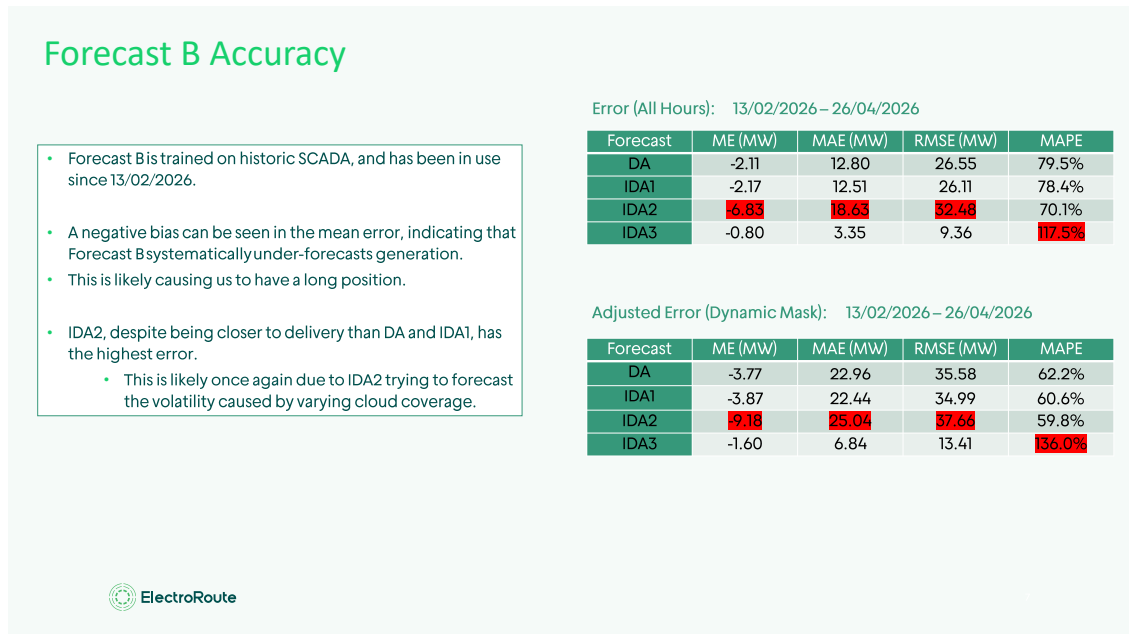
Forecast	ME (MW)	MAE (MW)	RMSE (MW)	MAPE
DA	3.86	15.25	27.29	63.6%
IDA1	3.31	14.95	26.91	61.7%
IDA2	2.83	15.50	27.55	62.0%
IDA3	-0.02	3.10	8.20	99.0%

Adjusted Error (Dynamic Mask)

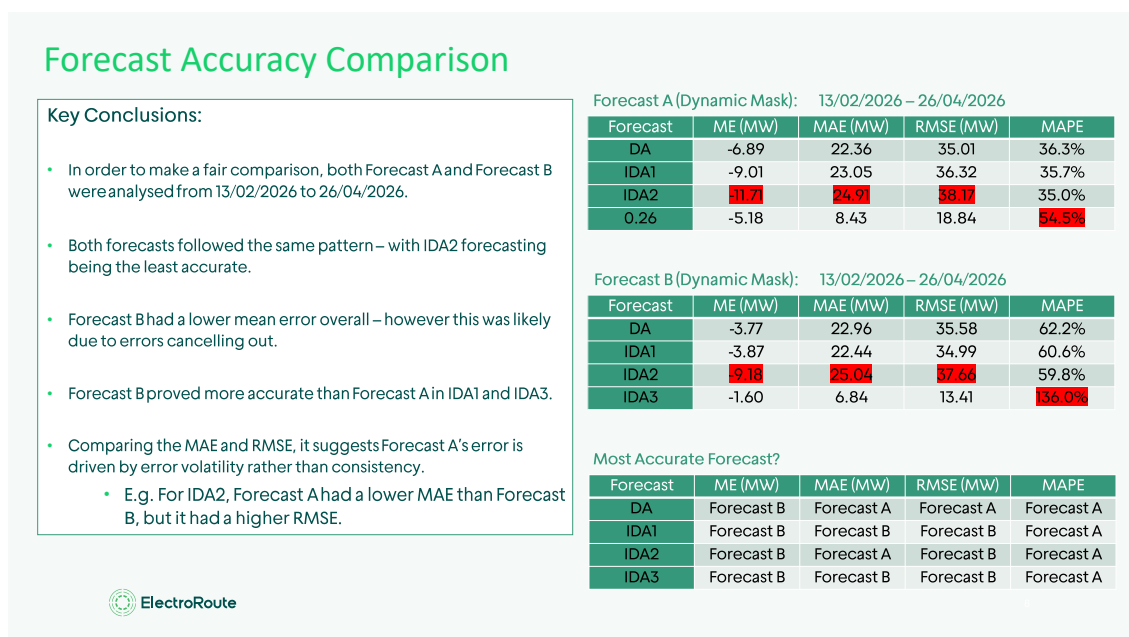
Forecast	ME (MW)	MAE (MW)	RMSE (MW)	MAPE
DA	5.07	19.95	31.21	57.5%
IDA1	4.34	19.55	30.77	55.4%
IDA2	3.85	21.08	32.13	53.5%
IDA3	-0.88	8.47	14.68	93.4%

4.2 Forecast Performance Analysis

Similarly, the same metrics were used for the analysis of the second forecast provider. This forecast has been in use since 13/02/2026 therefore the analysis took place over a smaller window.

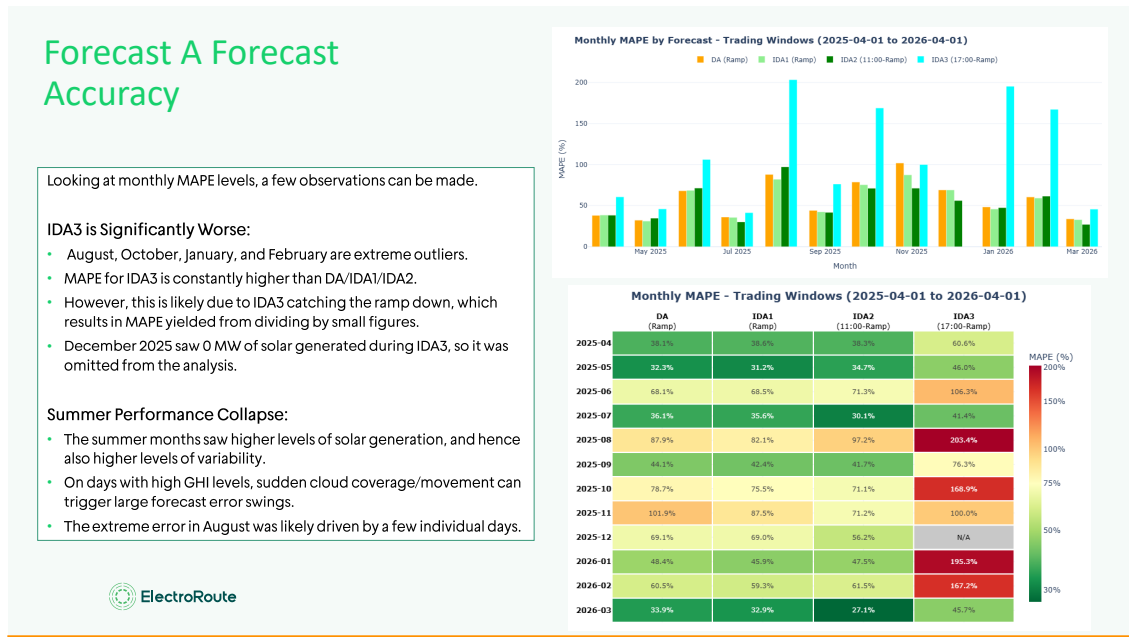


To ensure a fair comparison, both forecasts were evaluated over the same time period using the dynamic mask. Both followed the same pattern, with IDA2 being the least accurate auction stage. While Forecast B exhibited a lower mean error overall, this was likely due to errors cancelling out, with Forecast A proving more accurate across several key metrics.

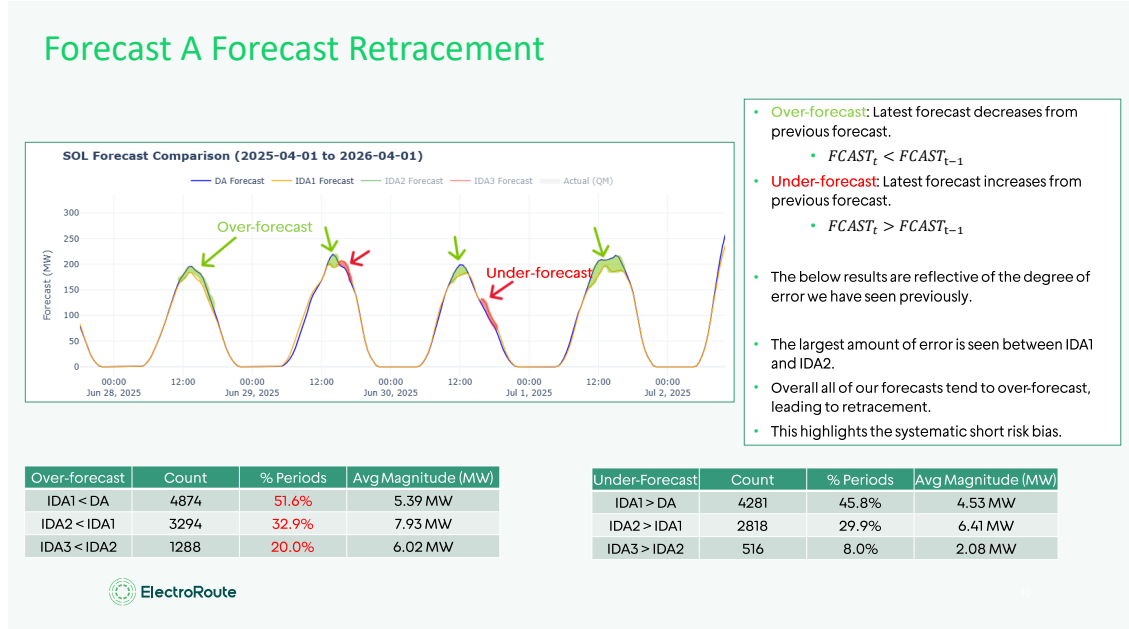


This slide presents the monthly MAPE breakdown for Forecast A across each auction stage, displayed as a bar chart and heatmap covering April 2025 to April 2026. The dynamic ramp-based mask is applied throughout, with December 2025 omitted from IDA3 as no solar generation was recorded during that period. The heatmap allows patterns in forecast accuracy to be identified across months and auction stages, with

colour scaling used to highlight periods of particularly high error.



In order to determine whether our forecasts were causing us to take short or long positions, the forecast retracement was analysed. Overall, the analysis suggested that our forecasts were over-forecasting, possibly driving the short risk bias across the portfolio.



4.3 Individual Solar Site Performance Analysis

The individual performance of each solar farm was analysed to try and identify if the errors were being driven by any particular sites. The site names and capacities were kept hidden. Scatter plots and Python's pvlb library helped us identify a systematic shortness and higher error probability during high solar days. We also identified some outliers that were skewing the results slightly.

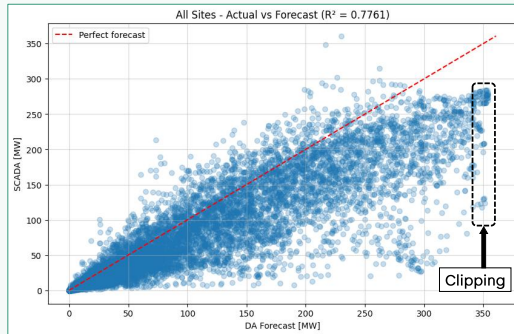
Forecast A Site Forecast Accuracy

This section of analysis looks at the SCADA / DA forecast relationship at each solar site in our portfolio.

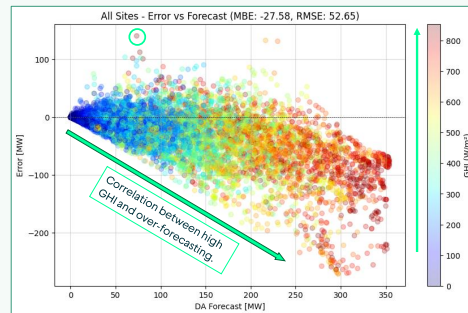
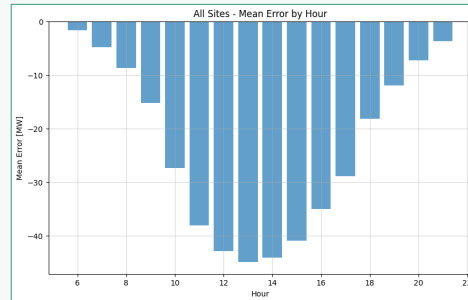
Python's pvlb library was used to model clear sky GHI output at an aggregated coordinate of all our sites.

A mask was applied to try and filter out inconsistencies:

06:30 – 22:00, excluding any periods where either SCADA, forecast, or both are NaN/0.



It is worth noting that the portfolio's maximum export capacity (MEC) is 356.4MW, which leads to a clipping effect as outlined in the plot above.



Forecast A Site Forecast Accuracy

'n' represents the amount of samples that were used in the analysis, with the red cells indicating significant data exclusion (mostly due to consistent SCADA outages).

Portfolio Metric	Value	%
ME	-27.58 MW	-7.64%
MAE	33.53 MW	9.28%
RMSE	52.65 MW	14.58%
R²	0.7761	-
Over-Forecast	-	80.4%
Under-Forecast	-	19.6%

Per-Site Forecast Error Statistics (sorted by nMAE)

Site	Capacity (MW)	n	ME (MW)	MAE (MW)	RMSE (MW)	nME (%)	nMAE (%)	nRMSE (%)	R²	Over (%)	Under (%)
		3029	0.02	0.44	0.72	0.25	4.86	8.03	0.68	49.72	50.28
		8414	-0.12	0.63	0.95	-1.43	7.6	11.53	0.74	56.23	43.77
		3289	0.12	0.4	0.61	2.47	8.13	12.24	0.82	44.6	55.4
		7873	-1.58	5.13	8.12	-2.64	8.55	13.53	0.77	59.75	40.25
		8496	-0.15	0.43	0.67	-3.07	9.11	14.12	0.71	59.52	40.35
		3294	-0.05	0.45	0.72	-0.99	9.16	14.45	0.71	55.49	44.47
		3296	0	0.46	0.69	0.01	9.25	13.85	0.72	52.12	47.88
		7450	-0.49	2.12	3.23	-2.26	9.73	14.86	0.68	56.27	43.73
		3307	0.1	0.57	0.88	2	11.46	17.85	0.53	44.84	55.16
		6321	-2.21	10.37	15.9	-2.54	11.91	18.28	0.66	56.97	42.97
		8262	-7.58	14.31	21.85	-6.37	12.02	18.36	0.65	64.43	35.57
		5498	-3.25	4.61	7.35	-10.29	14.59	23.25	0.38	71.12	28.88

Forecast A Site Accuracy

Looking at each Irish solar site individually, we can see how each one contributes to the overall forecast error.

The site error distribution is skewed negatively with the larger sites positioned toward the upper tail. However there are some outliers such as [SITE NAME] which is a relatively large site with decent accuracy.

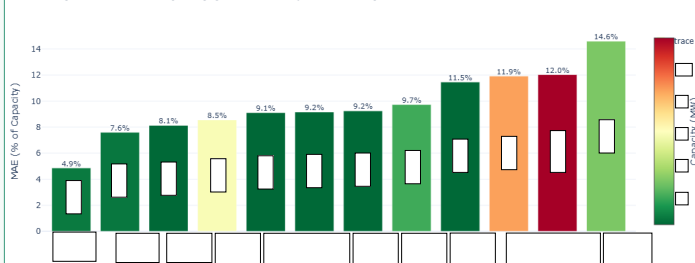
For values where the MBE is negative, the SCADA is less than the forecasted volume (site over-forecasting). This means that we are likely to be short if trading to forecast.

On average, the sites over-forecasted at all but two sites, meaning our solar book was more likely to be short across the last year. Therefore there appears to be a systematic short risk across our portfolio.

It is worth mentioning that this analysis is based on SCADA, which excludes a lot of null periods. Further analysis will use metered generation.



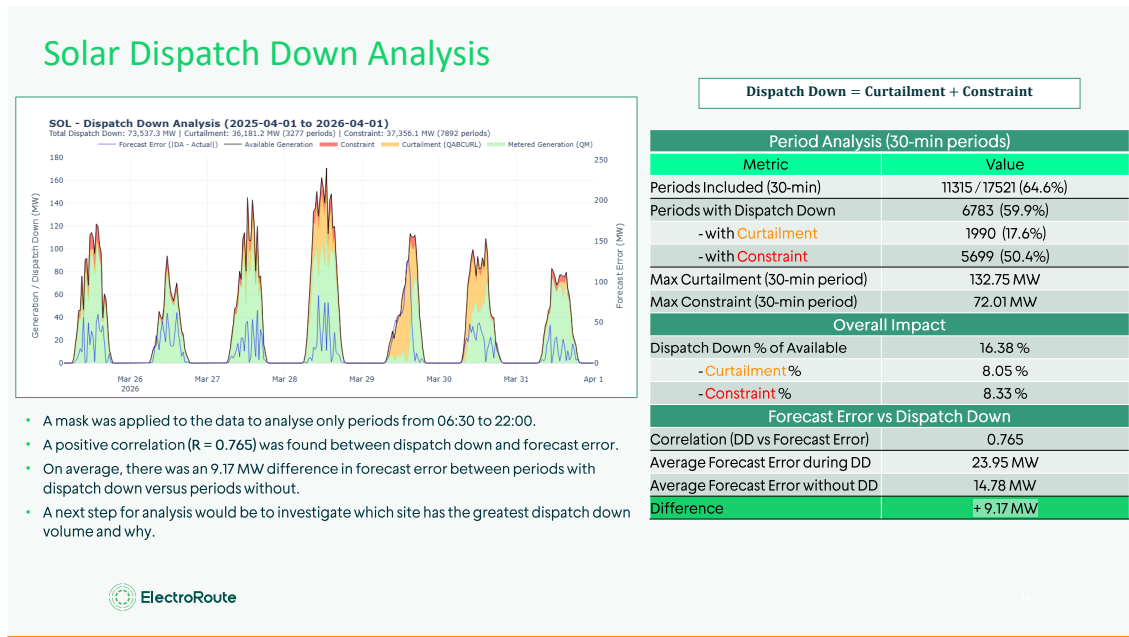
MAE by Site as % of Capacity (06:30-22:00, excl. zeros)



MBE by Site as % of Capacity (06:30-22:00, excl. zeros)

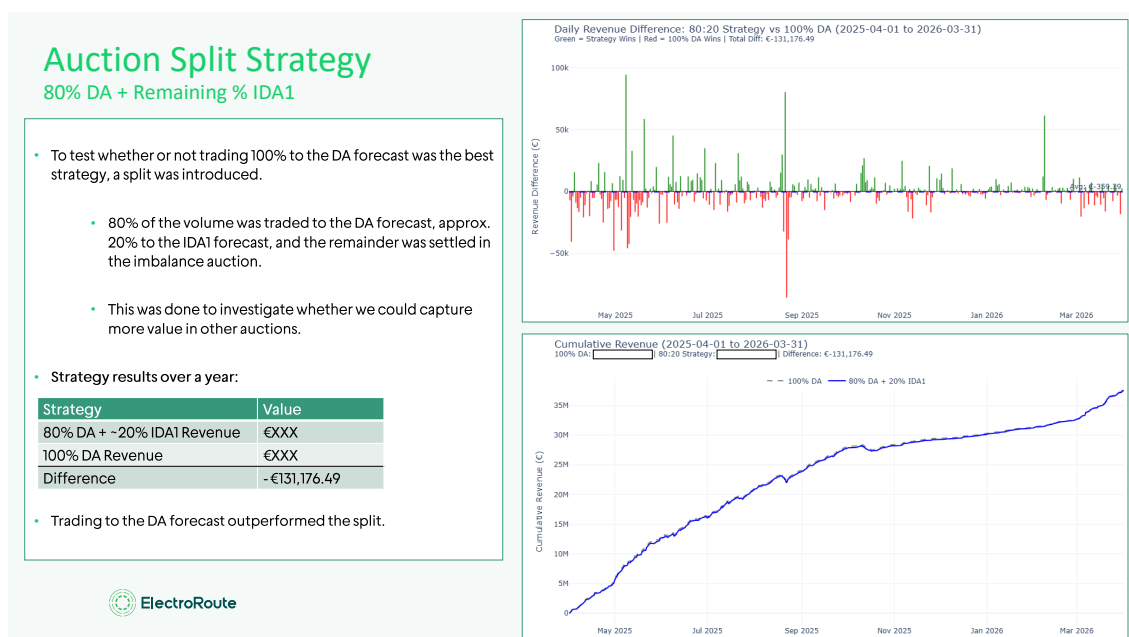


Analysis was also done to investigate whether or not there was a correlation between dispatch down and forecast error. A positive correlation was found between the two, with an overall difference of 9.17 MW between error during dispatch down and without.



4.4 Auction Split Strategy Backtest

The following slides look at some backtested strategies. This was less of a data-driven analysis, but rather just an investigation into whether modifying how we traded our volume would have an effect on performance, examining an 80:20 auction split. Rather than trading 100% of volume to the DA forecast, 80% was traded to DA, approximately 20% to IDA1 or IDA2, with the remainder settled in the imbalance auction. Revenue and consistency were compared with a 100% baseline.

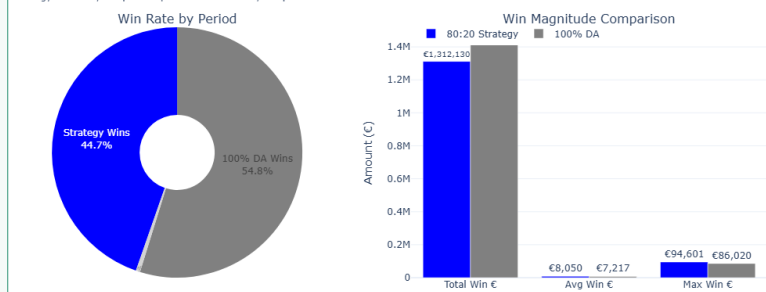


Split Analysis

80% DA + Remaining % IDA1

- A comparison of each individual traded period can tell us whether the DA strategy won through consistent smaller wins or occasional large wins.
- 100% DA had a better win rate.
- 100% DA had a greater win total.
- 80/20% IDA1 had a greater average win.
- 80/20% IDA1 also had a greater max win.
- This suggests that the 80/20% split is more volatile but is not as consistent as 100% DA.
- Overall trading 100% DA was better.

Big Wins vs Consistency Analysis (2025-04-01 to 2026-03-31)
Strategy wins 163/365 periods | 100% DA wins 200/365 periods



Metric	100% DA	80/20%
Win Rate	54.8%	44.7%
Total Win	€1,443,306	€1,312,130
Average Win	€7,217	€8,050
Maximum Win	€86,020	€94,601



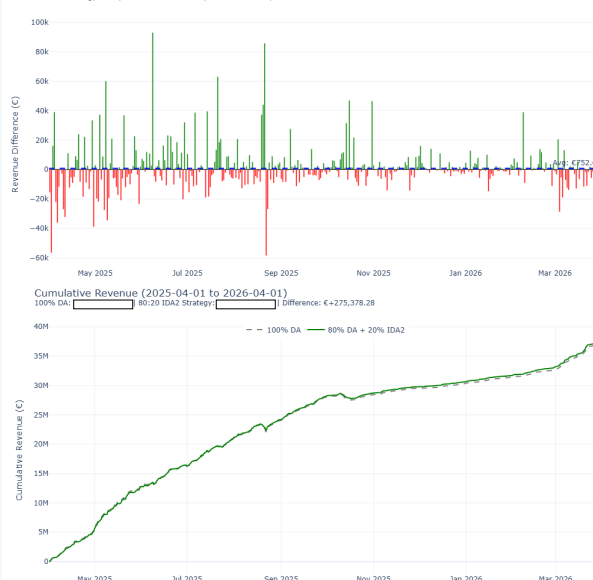
Split Analysis

80% DA + Remaining % IDA2

- The same test was done with IDA2 instead of IDA1.
 - 80% of the volume was traded to the DA forecast, approx. 20% to the IDA2 forecast, and the remainder was settled in the imbalance auction.
 - This was done to investigate whether we could capture more value in other auctions.
- Strategy results over a year:

Strategy	Value
80% DA + ~20% IDA2 Revenue	€XXX
100% DA Revenue	€XXX
Difference	+ €275,378.28
- The IDA2 split outperformed trading 100% to DA.

Daily Revenue Difference: 80:20 IDA2 Strategy vs 100% DA (2025-04-01 to 2026-04-01)
Green = Strategy Wins | Red = 100% DA Wins | Total Diff: €+275,378.28

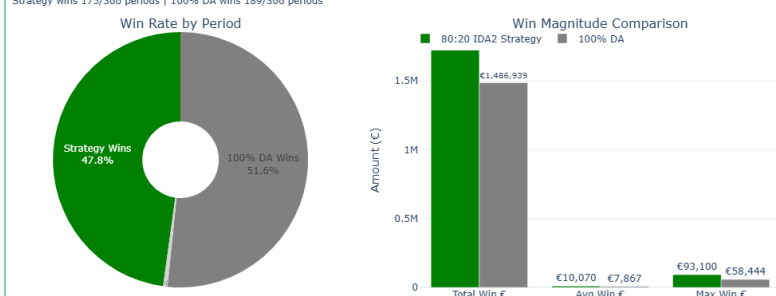


Split Analysis

80% DA + Remaining % IDA2

- 100% DA had a better win rate.
- 80/20% IDA2 had a greater win total.
- 80/20% IDA2 had a greater average win.
- 80/20% IDA2 also had a greater max win.
- This suggests that the 80/20% split is more volatile but is not as consistent as 100% DA.
- The IDA2 split performed better than 100% DA.
- The IDA2 split outperformed the IDA1 split.

Big Wins vs Consistency Analysis (2025-04-01 to 2026-04-01)
Strategy wins 175/366 periods | 100% DA wins 189/366 periods

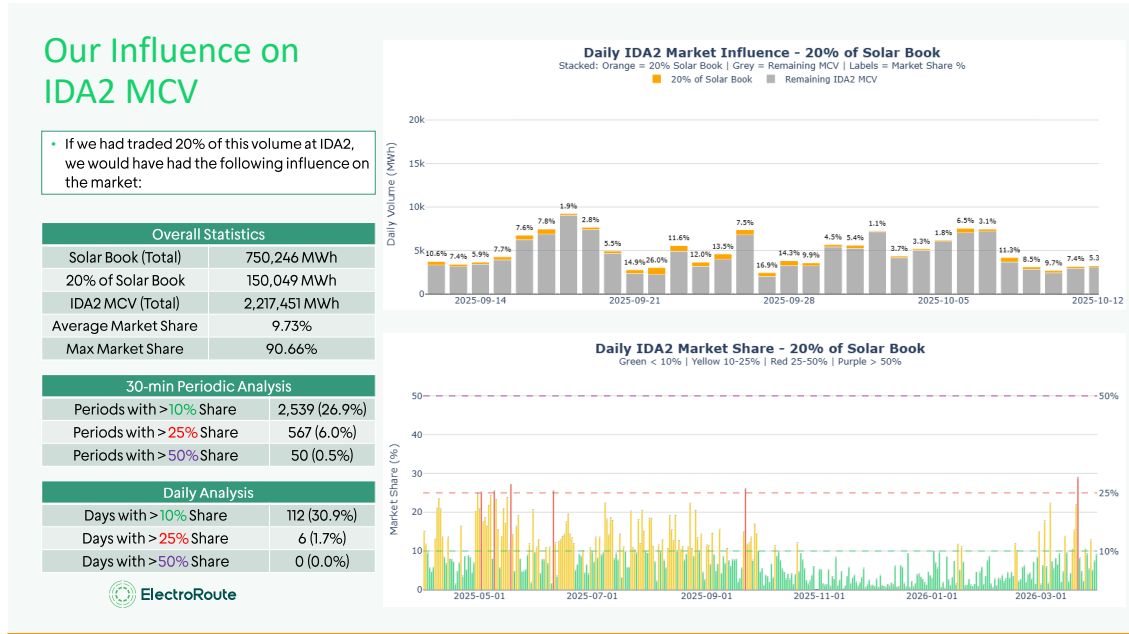


Metric	80/20% IDA1	80/20% IDA2
Total Revenue	€XXX	€XXX
Difference		+ €443,444.43

Metric	100% DA	80/20%
Win Rate	51.6%	47.8%
Total Win	€1,486,939	€1,762,317
Average Win	€7,867	€10,070
Maximum Win	€58,444	€93,100

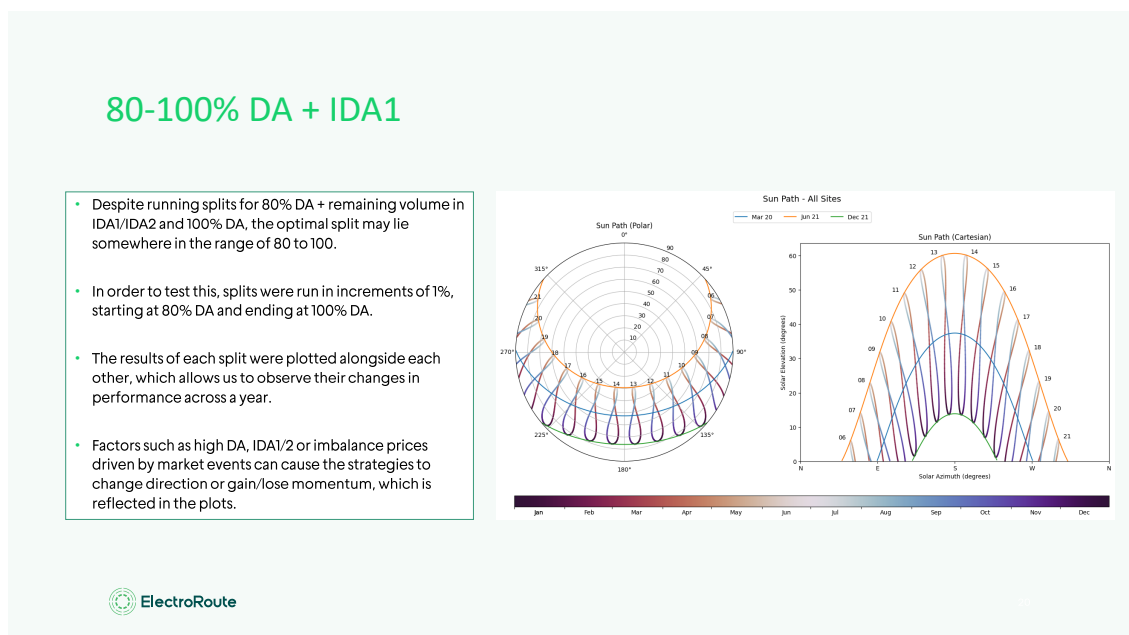


In order to determine if trading 20% of our volume in IDA2, we looked at the MCV and how we would influence it. Across all 30-minute trading periods, we held approximately 10% of the market clearing volume. This is not very desirable as it means our volume has the power to move prices and the system imbalance, which could lead to unexpected changes.



4.5 Finding the Optimal Split

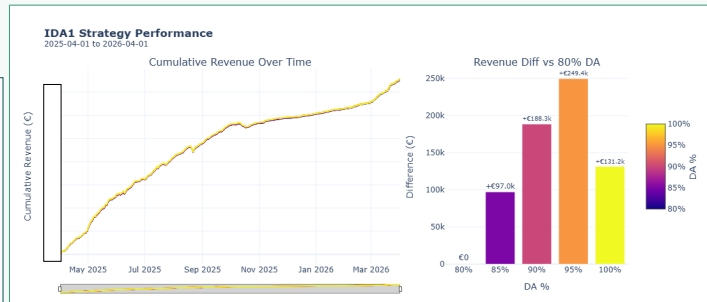
This section explores the range between the 80% DA and 100% DA strategies, acknowledging that the optimal split may lie somewhere between the two extremes. To investigate this, splits were run in increments of 1% from 80% DA to 100% DA, with the remaining volume traded in IDA1. The same approach was also applied for an IDA2 split, running the same incremental analysis with the remaining volume traded in IDA2 instead.



80-100% DA + IDA1

- Splitting over DA and IDA1 produced fairly consistent results. Across the year, lower splits (purple) performed worse.
- The higher splits consistently performed better with the upper-middle splits generating the most revenue from November 2025 onward.
- Overall, the best split was 98% DA.
- The results suggest that there is value in holding back some volume to trade in IDA1.

Metric	Strategy	Revenue
Best IDA1 Strategy	98% DA	€XXX
Baseline Strategy	100% DA	€XXX
Difference		€133,575.61

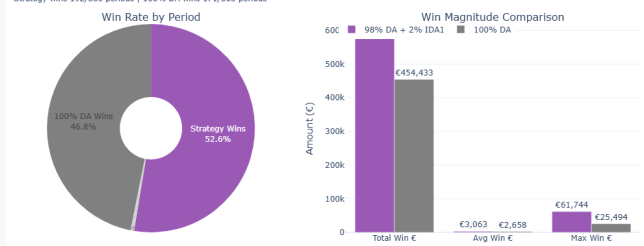


Optimal IDA1 Split 98% DA + Remaining % IDA1

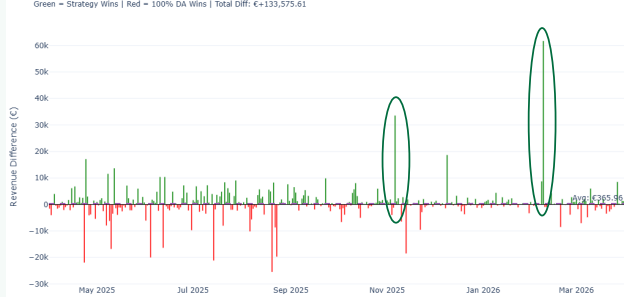
- The strategy won 52.6% of the time compared to 100% DA.
- It also had a greater total win, average win, and maximum win.
- However it did have two days of massive wins, outlined in the figure to the right, that could have driven the result.
- But overall it was more consistent too.

Metric	100% DA	98/2%
Win Rate	46.8%	52.6%
Total Win	€454,433	€588,008
Average Win	€2,658	€3,063
Maximum Win	€25,494	€61,744

Big Wins vs Consistency Analysis (2025-04-01 to 2026-03-31)



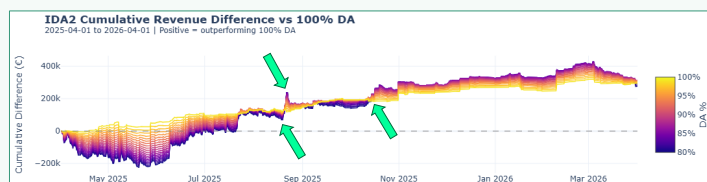
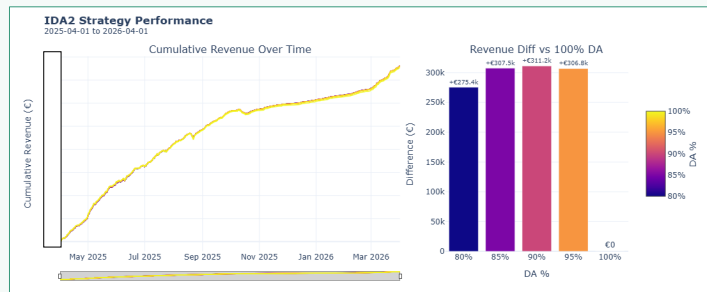
Daily Revenue Difference: 98% DA + 2% IDA1 Strategy vs 100% DA (2025-04-01 to 2026-03-31)



80-100% DA + IDA2

- The expected price volatility of IDA2 can be seen here, especially on August 21st and 22nd where high forecast error caused significant gains and losses.
- This volatility also resulted in multiple inflection points to be generated along the curve, highlighted by the arrows below.
- Overall, the IDA2 strategy outperformed the IDA1 strategy, with both the best and worst IDA2 strats outperforming the best and worst IDA1 strats.
- However, the IDA2 strategy yielded greater dispersion, which once again reflects this higher volatility and greater risk.

Metric	Strategy	Revenue
Best IDA2 Strategy	90% DA	€XXX
Baseline Strategy	100% DA	€XXX
Difference		€311,207.81

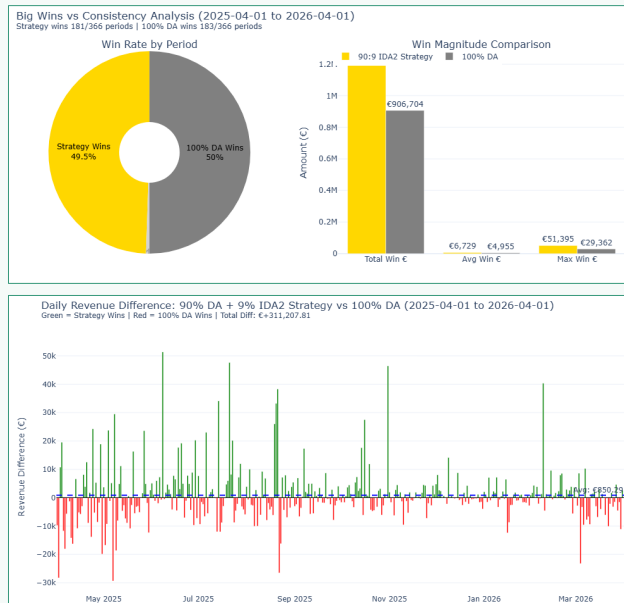


Optimal IDA2 Split

90% DA + Remaining % IDA2

- In terms of consistency, the 100% DA split outperformed the 90% split slightly.
- The 90% split had a greater total win, average win, and maximum win.
- From the graph below it looks like there were multiple high magnitude revenue days driving the performance.

Metric	100% DA	98/2%
Win Rate	50%	49.5%
Total Win	€906,704	€1,217,911
Average Win	€4,955	€6,729
Maximum Win	€29,362	€51,395



Trading 10% of the volume at IDA2 yielded the best returns. Therefore, the % of MCV was investigated. Overall it had a moderate but manageable influence on the MCV — making it unlikely to influence prices too much.

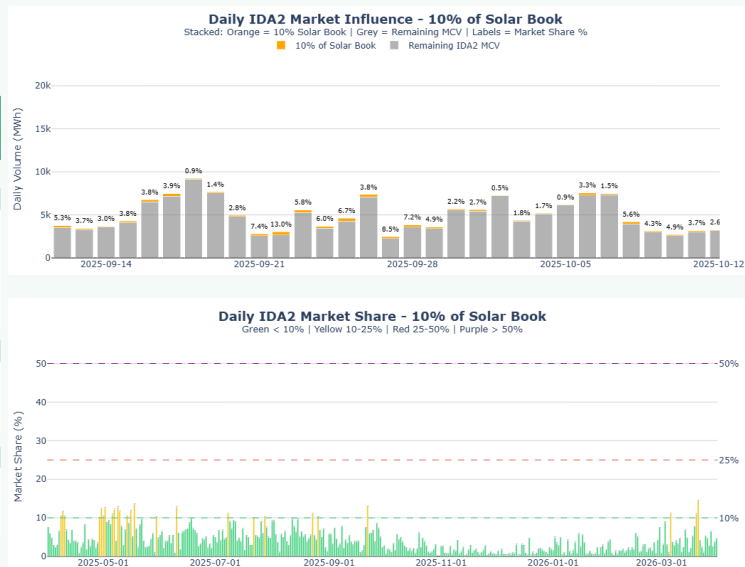
Our Influence on IDA2 MCV

- If we had traded 10% of this volume at IDA2, we would have had the following influence on the market:

Overall Statistics	
Solar Book (Total)	750,246 MWh
20% of Solar Book	75,025 MWh
IDA2 MCV (Total)	2,217,451 MWh
Average Market Share	4.86%
Max Market Share	45.33%

30-min Periodic Analysis	
Periods with >10% Share	949 (10.1%)
Periods with >25% Share	50 (0.5%)
Periods with >50% Share	0 (0.0%)

Daily Analysis	
Days with >10% Share	25 (6.9%)
Days with >25% Share	0 (0.0%)
Days with >50% Share	0 (0.0%)

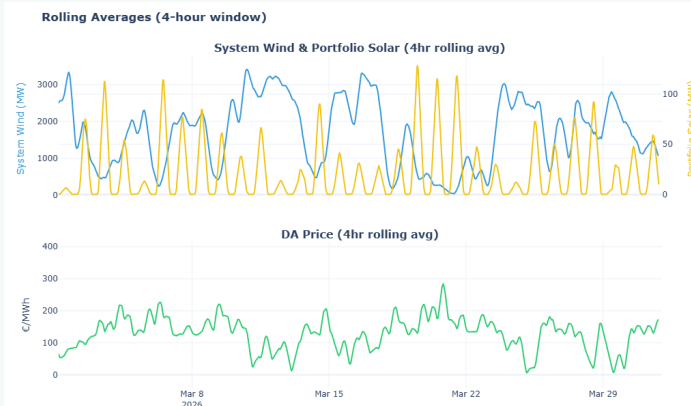


4.6 Influence of Wind and Dispatch Down

The relationship between wind generation and Day-Ahead (DA) prices was investigated, revealing a moderate negative correlation between the two, with high wind periods suppressing prices and impacting the profitability of solar trading. A case study of August 2025 was presented to illustrate this, with wind levels categorised into three groups to highlight the role of seasonality in trading performance. Finally, solar and wind curtailment were examined, with the vast majority of solar curtailment found to occur during periods of concurrent wind curtailment, suggesting wind curtailment is a strong predictor of solar curtailment.

Influence of Wind on Prices

- We know that high wind tends to push prices down, but here are some stats that back this up.



Key Statistics:

Wind vs DA Price Correlation **-0.454**

- Moderate negative relationship. As wind increases, DA price tends to decrease.

Wind-Price Sensitivity **-0.025 €/MW**

- For every 1 MW increase in system wind, the DA price drops by €0.025 (-€25.08 per 1000 MW).

R-Squared (R^2) **0.206**

- Roughly 21% of DA price variance is driven by wind alone. Other factors such as demand, solar, gas prices etc. drive the remaining 79%.

P-Value (P) **-0.00**

- The relationship is statistically significant and is not based on random chance.



Influence of Wind

Example: August 2025

- Here is plot for August 2025, a particularly high solar month where we had large balancing costs.
- Wind levels were low, solar levels were high, and prices remained relatively stable across DA, IDA1 & IDA2. However the imbalance prices were very volatile.
- When wind came up during these periods prices were pushed down, with imbalance prices being completely floored in some instances.
- The opposite is also true, low wind and high solar was extremely volatile in the BM.
- It is clear that seasonality plays a significant role in determining how effectively we can trade solar (i.e. large imbalances in the summer have greater impact).
- Further analysis is required to investigate why solar is being undersold on low wind days—specifically whether this is driven by wind forecast error or an expectedly low wind output.
- Based on percentages of the maximum, wind levels were categorised into 'High' (>66%), 'Medium' (33-66%), and 'Low' (<33%).

FY25/26

Wind	Periods	Wind (MW)	DA (€)	IDA1 (€)	IDA2 (€)	Imbalance (€)	DA vs Low Wind
Low	2940	344	140.1	138.95	138.36	152.11	-
Medium	2832	1150	104.5	101.94	98.91	100.39	-35.59
High	2581	2365	85.28	84.19	80.29	78.38	-54.81

Aug 25

Wind	Periods	Wind (MW)	DA (€)	IDA1 (€)	IDA2 (€)	Imbalance (€)	DA vs Low Wind
Low	256	181	135.39	128.48	134.99	181.99	-
Medium	308	792	96.63	96.82	91.31	109.51	-38.76
High	269	1,908	69.32	69.00	66.10	55.94	-66.08



Curtailment Analysis

- It is worth investigating whether there is any correlation between solar and wind curtailment.

Category	Count	Percentage
Total 30-min Periods	17,518	100%
Neither Curtailed	13,196	77.1%
Wind Only Curtailed	1,881	10.7%
Solar Only Curtailed	163	0.9%
Both Curtailed	1,963	11.2%

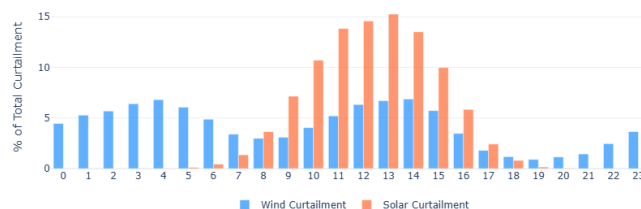
Metric	Value (MW)	Percentage
Total Wind Curtailment	68,386.4	65.4% of total
Total Solar Curtailment	36,181.2	34.6% of total
Wind During Concurrent	40,583.0	59.3% of wind
Solar During Concurrent	35,617.0	98.4% of solar

- Based on this analysis, it is fair to say that there is a relationship between solar and wind curtailment – and therefore forecasting wind curtailment could possibly help us trade solar more effectively.
- Further analysis could include studying the impact of over selling solar on high wind days.

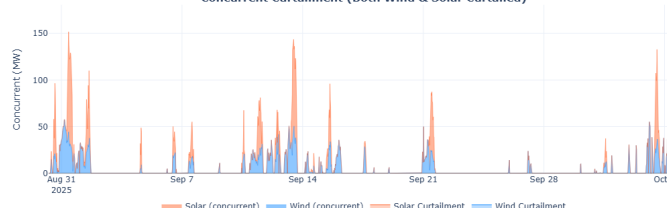


Curtailment Distribution by Hour of Day

% of total curtailment occurring in each hour



Concurrent Curtailment (Both Wind & Solar Curtailed)



The analysis concluded that there were multiple direct and indirect factors influencing solar book performance. Forecast accuracy was identified as the most influential factor, with a systematic over-forecasting bias causing consistent short positions. Curtailment created significant unavoidable exposure, particularly during peak solar hours, with further site-level analysis required to understand the magnitude of errors at individual sites. Wind was found to be the key driver of Irish electricity prices, with high wind periods correlating with both lower prices and increased curtailment risk, making wind forecasts essential for optimising the solar trading strategy. Finally, trading activity had the potential to influence prices and imbalance volumes depending on the auction in which volume was traded, which had to be carefully managed. Addressing forecast bias and understanding site-level performance variations represented the highest-impact opportunities for improving solar book profitability.

Conclusions

Based on this analysis, we can conclude that there are multiple different factors that direct and indirect influences on our solar book performance.

- **Forecast Accuracy** is without doubt the most influential factor on our performance. From this analysis, we have noticed a systematic over-forecasting bias – which consequently has caused us to be short.
- **Curtailment** creates significant unavoidable exposure particularly during peak solar hours.
 - Further analysis needs to be done on the individual sites such as [SITE NAME] and [SITE NAME] to determine why the magnitude of over-forecasting and error are so large.
- **Wind** is the key driver of Irish electricity prices. High wind periods correlate with both lower prices and increased solar curtailment risk, making wind forecasts essential for optimising our solar trading strategy.
- Depending on which auction we trade our volume in, we can have a significant influence on the **Market Clearing Volume**. This means that our trading activity has the potential to influence prices and imbalance volumes – which is something we need to be careful of.
 - A practical example of this was simulating 20% of our solar volume through IDA2, representing 10% of the total MCV.

Addressing forecast bias and understanding site-level performance variations represent the highest-impact opportunities for improving our solar book profitability.



4.7 Next Steps

The next steps outlined focused on strategy development and testing, with the aim of optimising volume in the IDAs to reduce balancing exposure. A scoring system was proposed to categorise the forecast on a given day based on volume levels, peak level and ramp rate, with seasonality accounted for using a Normalised Solar Index (NSI). The NSI was defined as the daily total forecasted volume divided by the 14-day rolling median, providing a smoother and more reliable measure of seasonality than raw forecasts alone.

To determine the optimal auction split for a given day, an ensemble model approach was proposed, whereby multiple models and their weights were learned automatically from a set of feature inputs. These inputs included solar and wind generation forecasts, system demand forecasts, historical forecast error metrics, auction price history, price spreads, historical price volatility, imbalance price flex, SNSP forecasts, curtailment events and expected ramp rates. The weighted ensemble would then output a recommended split for the following trading day. This was framed as a revenue maximisation problem, where predicting the optimal auction split for a given day would allow trading decisions to be made that maximise revenue. It was noted that this approach could be implemented with as little as one year of data, with model weights updated dynamically each trading day.

Next Steps

Strategy Development and Testing

- Strategy Development Aim – optimise volume in the IDA's to reduce balancing exposure.
- How - To optimise the volume between auctions, proposing to use a scoring system to categorise the forecast on a given day.
- The system looks to categorise the forecast based on volume levels, peak level and ramp rate. Seasonality is reduced using NSI (normalised solar index Forecast/14day rolling median - in winter you can extend the formula to include seasonal average using 2/3 years of data).
- Backtest the scores with associated % split to see the most profitable (if that is true at all)
- If it's true, shadow trade.
- Investigate underselling solar on low wind days and adding as a screen to the scoring

Date	Month	Forecast MWh	14d Median Expected	NSI	NSI Bucket	Peak MW	Peak	Peak Bucket	Max Ramp MW	Ramp	Ramp Bucket	Score	DA Sell %	IDA 2 Sell %
01-Apr	Apr	95	118	0.81	Medium	19	0.63	Medium	4	0.13	Low	4	95.0%	5.0%
02-Apr	Apr	140	120	1.17	High	27	0.90	Medium	8	0.27	Medium	4	95.0%	5.0%
03-Apr	Apr	155	122	1.27	High	29	0.97	High	6	0.20	Medium	5	97.5%	2.5%
04-Apr	Apr	50	121	0.41	Low	15	0.50	Low	5	0.17	Medium	1	85.0%	15.0%
05-Apr	Apr	180	125	1.44	High	30	1.00	High	9	0.30	High	4	95.0%	5.0%
06-Apr	Apr	125	128	0.98	Medium	24	0.80	Medium	5	0.17	Medium	3	92.5%	7.5%

NSI	NSI = Forecast Daily MWh / 14-day rolling median expected MWh.
Peak	Peak = Forecast Peak MW / Installed Capacity. For a 30 MW site, Peak = 1.00 means the forecast reaches full output.
Ramp	Ramp = Max half-hour MW change / Installed Capacity. For a 30 MW site, Ramp = 0.33 means the largest half-hour forecast step is 10 MW.
Scoring	This example uses a simple additive score: NSI Low/Medium/High = 0/1/2; Peak Low/Medium/High = 0/1/2; RampAdj Low/Medium/High = 2/1/0. Lower ramp gets a higher score because smoother days are typically easier forecast



Normalised Solar Index (NSI) - Example

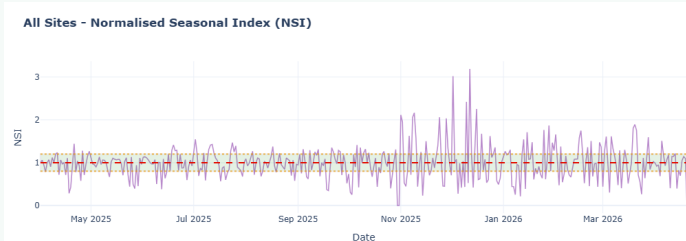
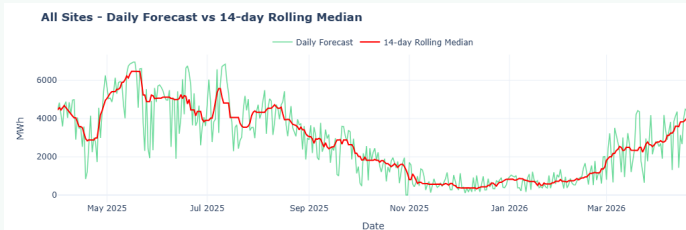
- A rolling median can also help us determine the seasonality of solar generation by creating a smoother curve.
- However this doesn't necessarily give us a good edge to predict the solar output due to its naturally high daily volatility – particularly in the summer months as seen on the right.
- A normalised solar index was defined as:

$$NSI = \frac{\text{Daily Total Forecasted Volume}}{14 - \text{Day Rolling Median}}$$

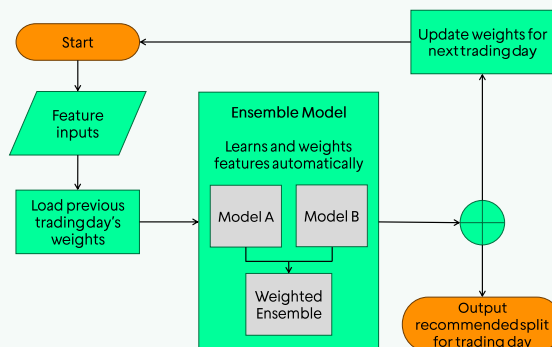
NSI Value	Meaning
NSI = 1.0	Exactly average day (matches rolling median)
NSI > 1.0	Above-average solar day (better than recent)
NSI < 1.0	Below-average solar day (worse than recent)
NSI > 1.2	Exceptionally good day (20% above recent)
NSI < 0.8	Exceptionally poor day (20% below recent)

- Results:

Metric	Value
Average NSI	0.997
NSI Standard Deviation	0.402
Days Above Normal (> 1.2)	93 (24.2%)
Days Below Normal (< 0.8)	107 (27.8%)
Days Normal (0.8 – 1.2)	185 (48.1%)



How Could This Be Done?



Example Feature Inputs	Weights
Solar Generation Forecasts (DA, IDA1, IDA2)	...
Wind Generation Forecasts (DA, IDA1, IDA2)	...
System Demand Forecast	...
Historical Forecast Error Metrics (ME, MAE, RMSE)	...
Forecast Bias History (systematic over/under-forecasting)	...
Auction Price History (DA, IDA1, IDA2, Imbalance)	...
Price Spreads (DA-IDA1, IDA1-IDA2, IDA2-Imbalance)	...
Historical Price Volatility by Auction	...
Imbalance Price Bias (long vs short penalty)	...
SNSP Forecast	...
Curtailment Events	...
Historical MCV %	...
Hour of Day & Day Type (Weekday/Weekend)	...
Season / Time of Year	...
Expected Ramp Rate (MW/hour)	...
Sum	1.00

- This is a revenue maximisation problem that can be resolved by predicting what the optimal auction split for a given day will be.
- By using some of the data features we have just analysed, alongside our forecasts, a recommendation can be made for the following trading day on what the best volume split would be to maximise revenue.
- This could be done by using an ensemble or some form of regression, where each feature is an input to the model and can be dynamically weighted from 0-1, depending on how much influence it would have on a particular day.

4.8 Skills and Learning Outcomes

Completing this analysis developed a range of both technical and professional skills. From a technical perspective, the project required extensive use of Python and SQL to extract, clean and process large volumes of market data across multiple auction stages. Writing complex queries to retrieve data from internal databases and manipulating it in Python using libraries such as Pandas, Plotly, and even learning new libraries such as pvlib, strengthened my ability to work efficiently with real-world datasets. The importance of ensuring data is cleaned effectively before analysing it was something that stood out to me. The analysis also provided valuable insight into the factors driving solar book performance, including forecast accuracy, curtailment and wind, which gave me a much stronger understanding of how renewable assets are traded in the Irish electricity market.

Presenting the findings to the full trading team was a particularly valuable experience. Communicating technical results clearly and concisely to a professional audience required careful consideration of how to structure and deliver the material. Fielding questions from the team further developed my ability to think critically under pressure and defend my methodology and conclusions. Overall, this project significantly strengthened my confidence in both conducting independent analysis and presenting findings in a professional setting.

5 IE-GB Interconnector Flow Prediction Model (XGBoost)

Net flow across the IE-GB interconnectors — EWIC, Moyle, and Greenlink — is a key driver of Irish market pricing and imbalance risk, yet no forecast for it existed on the desk beyond the interconnector nominations published close to real time. I built a model to forecast net interconnector flow at 30-minute resolution for the following delivery day, giving the desk earlier visibility of expected cross-border flow than was previously available. The model is designed to run once both the GB and Irish day-ahead auctions have cleared, so that the day-ahead price spread between the two markets — the primary driver of flow — is available as an input alongside wind, demand, and outage data.

5.1 Approach

XGBoost was chosen as the underlying model because interconnector flow is a non-linear function of several interacting variables — price spread, wind generation, and demand do not act independently, and tree-based methods capture these interactions without manual feature-crossing. Hyperparameters were tuned via Optuna using time-series cross-validation, and lag and rolling features were carefully shifted to avoid leaking future information into training. The full build, from data extraction through feature engineering, tuning, and evaluation, is reproduced below.

5.2 Key Concepts

- **Day-Ahead Price Spread:** GB day-ahead price less IE day-ahead price, used as the primary signal for the market incentive to flow power across the interconnectors.
- **Net Position:** Demand forecast minus wind and solar forecast for a market, proxying how much that market needs to import or can afford to export.
- **Cyclical Encoding:** Hour, month, and day-of-week represented as sine/cosine pairs so that boundary values (e.g. 23:00 into 00:00) are treated as continuous rather than discontinuous.

- **Lag and Rolling Features:** Past flow values and rolling statistics (24h/48h/168h lags, 24h rolling mean/std/max/min) added as predictors, with shifts applied before rolling to prevent leakage.
- **Data Leakage:** Use of information at prediction time that would not genuinely be available, such as an unshifted lag or a rolling window that includes the target period; guarded against throughout via explicit shifting and exclusion of the target's component columns from the feature set.
- **TimeSeriesSplit:** Cross-validation scheme that respects chronological order, training only on data prior to each validation fold rather than shuffling randomly.
- **Optuna:** Hyperparameter optimisation framework used to search the model's parameter space over multiple trials, minimising cross-validated RMSE.
- **MAE / RMSE / R^2 :** Evaluation metrics on the held-out test set — mean absolute error and root mean square error in MW, and R^2 as the proportion of variance in flow explained by the model.

IE-GB Interconnector Flow Prediction Model (XGBoost)

Net interconnector flow is primarily driven by the ISEM/GB day-ahead price spread, with wind generation imbalances and system demand acting as secondary drivers.

This model uses XGBoost to forecast net IC flow at 30-minute resolution for the following delivery day. It is designed to be run around or after 14:00 on D-1, once both the EPEX GB (approx. 09:00) and SEMPx (approx. 12:00) day-ahead auctions have cleared and their results have been loaded to our database.

Features combine day-ahead market prices, wind and demand forecasts, derived spread and position metrics, and lagged flow history. Hyperparameters are tuned via Optuna with time-series cross-validation to prevent lookahead bias.

Import libraries and functions:

```
In [1]: import pandas as pd
import numpy as np
import plotly.graph_objects as go
import xgboost as xgb
from xgboost import XGBRegressor
import optuna
from sklearn.model_selection import train_test_split
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from datetime import datetime as dt
from Common.DBconnect import dbConnect

# Define start and end dates:
start_date = '2025-04-18 23:00'
end_date = pd.Timestamp.today().normalize() + pd.Timedelta(days=2)
```

MySQL Database Query Functions:

Two helper functions are used to pull data from a database that stores market data.

`db_query` fetches a single time series for a given ID between a start and end date. Where multiple forecast versions exist for the same timestamp, the median value is taken.

`get_plant_outages` fetches the latest published outage records for the three IE-GB interconnectors - EWIC, Moyle, and Greenlink. It returns the most recently modified version of each published outage that overlaps the requested date window, along with installed capacity mapped from a lookup dictionary.

```
In [2]: # Fetches a single time series from the database for a particular ID.
def db_query(start_date, end_date, dim_id, quantity_label):
    query = """
    SELECT
        ts.TimestampUTC AS start_time_utc,
        ts.Value AS quantity
    FROM DataWarehouse.TimeSeries ts
    WHERE ts.SeriesID = '{0}'
        AND ts.TimestampUTC BETWEEN '{1}' AND '{2}'
    ORDER BY start_time_utc
    """.format(dim_id, start_date, end_date)

    df = pd.DataFrame(dbConnect.select(query, "DataWarehouse"),
                      columns=['Time (UTC)', quantity_label])
    df['Time (UTC)'] = pd.to_datetime(df['Time (UTC)'])

    # Cast values to float (SQL DECIMAL types are returned as Python decimal.Decimal)
    df[quantity_label] = pd.to_numeric(df[quantity_label], errors="coerce").astype(float)

    df = df.set_index('Time (UTC)')

    # Due to multiple updated forecast ticks, we take the median across all forecast versions for each timestamp
    df = df.groupby(level=0).median()

    return df

# Fetches the Latest published outage records for selected transmission assets.
def get_plant_outages(start_date, end_date):
    query = """
    SELECT assets.AssetName,
           outages.AvailableCapacity,
           outages.OutageStart,
           outages.OutageEnd
    FROM (
        -- Get the latest published version of each outage
        SELECT AssetID,
               PublicationDate,
               MAX(RevisionDate) RevisionDate
        FROM DataWarehouse.AssetOutages
        WHERE RecordType = 4
        GROUP BY AssetID, PublicationDate
    ) latest
    JOIN DataWarehouse.vw_AssetOutages outages
    USING (AssetID, PublicationDate, RevisionDate)
    JOIN DataWarehouse.vw_Assets assets
    ON assets.AssetID = outages.AssetID
    WHERE assets.AssetName IN ('INTERCONNECTOR_A',
                              'INTERCONNECTOR_B',
                              'INTERCONNECTOR_C')

    -- Include any outage that overlaps the requested window
    AND ((OutageStart >= '{0}' AND OutageStart < '{1}')
        OR (OutageEnd >= '{0}' AND OutageEnd < '{1}')
        OR (OutageStart < '{0}' AND OutageEnd >= '{0}'))
```

```
        AND (RetirementDate IS NULL OR RetirementDate > '{0}')
        ORDER BY AssetName, OutageStart, OutageEnd, OutageID;
"""format(start_date, end_date)

df = pd.DataFrame(
    dbConnect.select(query, "DataWarehouse"),
    columns=['Asset Name', 'Available (MW)', 'Start (UTC)', 'End (UTC)']
)

df['Installed Capacity (MW)'] = df['Asset Name'].map(installed_capacities)
df['Available (MW)'] = pd.to_numeric(df['Available (MW)'], errors='coerce')
df['Start (UTC)'] = pd.to_datetime(df['Start (UTC)'])
df['End (UTC)'] = pd.to_datetime(df['End (UTC)'])

return df[['Asset Name', 'Installed Capacity (MW)', 'Available (MW)', 'Start (UTC)', 'End (UTC)']]
```

Feature Data

Forecast data is pulled for wind, solar, demand and day-ahead prices across both IE and GB. GB prices are fetched in GBP and converted to EUR using a daily FX rate that is forward-filled to 30-minute resolution.

Actual flows are fetched for each of the three interconnectors (EWIC, Moyle, Greenlink) and flipped negative to align with the IE export convention, where positive values represent flow from GB to IE.

Plant outages are fetched for each interconnector using the installed capacities below.

Interconnector	Installed Capacity (MW)
EWIC	532
Moyle	452
Greenlink	514

```
In [3]: forecast_end = pd.Timestamp(end_date) + pd.Timedelta(days=1)

IE_wind_forecast = db_query(start_date, forecast_end, 9352, "IE Wind Forecast (MW)")
IE_solar_forecast = db_query(start_date, forecast_end, 17571, "IE Solar Forecast (MW)")
GB_wind_forecast = db_query(start_date, forecast_end, 4485, "GB Wind Forecast (MW)")
GB_solar_forecast = db_query(start_date, forecast_end, 4493, "GB Solar Forecast (MW)")
IE_demand_forecast = db_query(start_date, forecast_end, 9348, "IE Demand Forecast (MW)")
GB_demand_forecast = db_query(start_date, forecast_end, 3995, "GB Demand Forecast (MW)")

price_start = pd.Timestamp(start_date) - pd.Timedelta(days=1)

IE_DA_prices_EUR = db_query(price_start, forecast_end, 9408, "IE DA Prices (€)")
GB_DA_IMRP_prices_GBP = db_query(price_start, forecast_end, 18378, "GB DA Prices (£)")
IE_DA_prices_EUR = db_query(start_date, forecast_end, 9408, "IE DA Prices (€)")
GB_DA_IMRP_prices_GBP = db_query(start_date, forecast_end, 18378, "GB DA Prices (£)")

def get_GBP_EUR_fx_rate(start_date, end_date):
    adjusted_start = (pd.to_datetime(start_date) - pd.Timedelta(days=1)).strftime('%Y-%m-%d')

    query = """
    SELECT
        trade_date,
        exchange_rate
    FROM MarketData.fx_exchange_rates
    WHERE trade_date BETWEEN '{0}' AND '{1}'
        AND (base_currency = 'GBP' AND quote_currency = 'EUR')
    """format(adjusted_start, end_date)

    df = pd.DataFrame(
        dbConnect.select(query, "MarketData"),
        columns=['Time (UTC)', 'GBP-EUR FX Rate (€)']
    )

    df['Time (UTC)'] = pd.to_datetime(df['Time (UTC)'])
    df['GBP-EUR FX Rate (€)'] = df['GBP-EUR FX Rate (€)'].astype(float)

    df = df.drop_duplicates(subset='Time (UTC)')
    df = df.set_index('Time (UTC)')
    df = df.resample('30 min').ffill()
    df = df.astype(float)

    return df

GBP_EUR_fx_rate = get_GBP_EUR_fx_rate(start_date, end_date)

def get_GB_DA_prices_EUR(start_date, end_date):
    df = GB_DA_IMRP_prices_GBP.copy()
    df = df.astype(float)
    df['GB DA Prices (€)'] = df['GB DA Prices (£)'] * GBP_EUR_fx_rate['GBP-EUR FX Rate (€)']
    return df[['GB DA Prices (€)']]

GB_DA_prices_EUR = get_GB_DA_prices_EUR(start_date, end_date)

# Shift DA prices forward by 24h to align trade date with delivery date
IE_DA_prices_EUR.index = IE_DA_prices_EUR.index + pd.Timedelta(days=1)
GB_DA_prices_EUR.index = GB_DA_prices_EUR.index + pd.Timedelta(days=1)

ewic_flow_actual = -db_query(start_date, end_date, 4871, 'EWIC Actual Flow (MW)')
moyle_flow_actual = -db_query(start_date, end_date, 4872, 'Moyle Actual Flow (MW)')
greenlink_flow_actual = -db_query(start_date, end_date, 18349, 'Greenlink Actual Flow (MW)')

installed_capacities = {
    "IC_MOYLE" : 452.0,
```

```
"IC_EWIC" : 532.0
}

plant_outages = get_plant_outages(start_date, end_date)
```

Feature Engineering

All features are concatenated into a single dataframe at 30-minute resolution. Hourly forecast series are forward-filled to align with the half-hourly target. Day-ahead price series for Ireland and Great Britain are shifted forward by 24 hours so that the auction date aligns with the corresponding delivery day.

The target variable is the sum of actual flows across the three interconnectors.

Several derived features are calculated from the forecast and price data:

- **Net position** for IE and GB: demand minus wind and solar forecast, used as a proxy for how much each market needs to import or can afford to export
- **Wind penetration** for IE and GB: wind forecast as a share of demand forecast
- **Day-ahead price spread**: GB day-ahead price minus IE day-ahead price, used to capture the market incentive for cross-border flow
- **Interconnector capacity**: starts at installed capacity, with outage records applied on top for EWIC, Moyle, and Greenlink

Rows where the target or key price features are missing are dropped. This removes future timestamps where no actual flow data exists yet, as well as any timestamps where shifted day-ahead prices are unavailable.

```
In [4]: # Create a concatenated dataframe of all features
model_df = pd.concat([
    IE_wind_forecast,
    GB_wind_forecast,
    IE_solar_forecast,
    GB_solar_forecast,
    IE_demand_forecast,
    GB_demand_forecast,
    ewic_flow_actual,
    moyle_flow_actual,
    greenlink_flow_actual,
    IE_DA_prices_EUR,
    GB_DA_prices_EUR
], axis=1)

model_df.index = pd.to_datetime(model_df.index)
model_df = model_df.sort_index()

# Forward fill hourly forecasts to 30-min resolution
forecast_cols = [
    "IE Wind Forecast (MW)",
    "GB Wind Forecast (MW)",
    "IE Solar Forecast (MW)",
    "GB Solar Forecast (MW)",
    "IE Demand Forecast (MW)",
    "GB Demand Forecast (MW)",
]
model_df[forecast_cols] = model_df[forecast_cols].ffill()

# Target
model_df["Net IC Flows (MW)"] = model_df[[
    "EWIC Actual Flow (MW)",
    "Greenlink Actual Flow (MW)",
    "Moyle Actual Flow (MW)",
]].sum(axis=1)

# IE-GB Price Spread
model_df["DA Price Spread (€)"] = (
    model_df["GB DA Prices (€)"] - model_df["IE DA Prices (€)"]
)

# Forecast-derived features - no leakage, all from forecasts
model_df["GB Net Position (MW)"] = (
    model_df["GB Demand Forecast (MW)"]
    - model_df["GB Wind Forecast (MW)"]
    - model_df["GB Solar Forecast (MW)"]
)
model_df["IE Net Position (MW)"] = (
    model_df["IE Demand Forecast (MW)"]
    - model_df["IE Wind Forecast (MW)"]
    - model_df["IE Solar Forecast (MW)"]
)
model_df["IE Wind Penetration"] = model_df["IE Wind Forecast (MW)"] / model_df["IE Demand Forecast (MW)"]
model_df["GB Wind Penetration"] = model_df["GB Wind Forecast (MW)"] / model_df["GB Demand Forecast (MW)"]

# Interconnector capacities - start at installed capacity then apply outages
plant_col_map = {
    "IC_EWIC": "EWIC Capacity (MW)",
    "IC_GREENLINK": "Greenlink Capacity (MW)",
    "IC_MOYLE": "Moyle Capacity (MW)",
}
for plant, col in plant_col_map.items():
    model_df[col] = installed_capacities[plant]

for _, row in plant_outages.iterrows():
    col = plant_col_map.get(row["Plant Name"])
    if col:
        mask = (model_df.index >= row["Start (UTC)"]) & (model_df.index <= row["End (UTC)"])
        model_df.loc[mask, col] = row["Available (MW)"]

model_df["Total IC Capacity (MW)"] = (
    model_df["EWIC Capacity (MW)"]
    + model_df["Moyle Capacity (MW)"]
    + model_df["Greenlink Capacity (MW)"]
)
```

```
# Drop rows where target is NaN (future rows with no actual flow data)
model_df = model_df.dropna(subset=["Net IC Flows (MW)"])

print(f"Rows after cleaning: {len(model_df)}")
print(f"Date range: {model_df.index[0]} to {model_df.index[-1]}")
display(model_df.head())
```

Rows after cleaning: 22212
Date range: 2025-04-18 23:00:00 to 2026-07-27 00:00:00

	IE Wind Forecast (MW)	GB Wind Forecast (MW)	IE Solar Forecast (MW)	GB Solar Forecast (MW)	IE Demand Forecast (MW)	GB Demand Forecast (MW)	EWIC Actual Flow (MW)	Moyle Actual Flow (MW)	Greenlink Actual Flow (MW)	IE DA Prices (€)	...	Net IC Flows (MW)	DA Price Spread (€)	GB Net Position (MW)	IE Net Position (MW)	IE Wind Penetration	GB Wind Penetration	C
Time (UTC)																		
2025-04-18 23:00:00	1841.0	10017.5	0.0	0.0	4088.0	20736.0	-0.0	370.0	NaN	NaN	...	370.0	NaN	10718.5	2247.0	0.450342	0.483097	
2025-04-18 23:30:00	1841.0	10017.5	0.0	0.0	4088.0	20736.0	-0.0	350.0	514.0	NaN	...	864.0	NaN	10718.5	2247.0	0.450342	0.483097	
2025-04-19 00:00:00	1624.0	10761.0	0.0	0.0	3938.0	20422.0	-0.0	350.0	514.0	NaN	...	864.0	NaN	9661.0	2314.0	0.412392	0.526932	
2025-04-19 00:30:00	1624.0	10761.0	0.0	0.0	3938.0	20422.0	-0.0	350.0	514.0	NaN	...	864.0	NaN	9661.0	2314.0	0.412392	0.526932	
2025-04-19 01:00:00	1546.0	10262.0	0.0	0.0	3876.0	19995.0	-0.0	350.0	514.0	NaN	...	864.0	NaN	9733.0	2330.0	0.398865	0.513228	

5 rows × 21 columns



XGBoost Regression Model:

XGBoost (Extreme Gradient Boosting) is an ensemble method that builds a sequence of decision trees, where each tree is trained to correct the residual errors of the previous. The final prediction is the sum of all trees:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}$$

where $\mathcal{F}$ is the space of regression trees and K is the total number of trees. At each boosting round, a new tree f_t is added to minimise a regularised objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

where l is a differentiable loss function (RMSE here) and Ω penalises model complexity:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 + \alpha \sum_{j=1}^T |w_j|$$

T is the number of leaves, w_j are the leaf weights, and γ, λ, α control the degree of regularisation. The objective is approximated using a second-order Taylor expansion, allowing efficient closed-form computation of the optimal leaf weights.

XGBoost suits this problem for several reasons. Interconnector flow is a non-linear function of multiple interacting variables - price spreads, wind generation and demand do not act independently, and their relationships shift across seasons and market conditions. Tree-based methods capture these interactions naturally without requiring explicit feature engineering of cross terms. The regularisation terms guard against overfitting on the relatively noisy half-hourly flow data, and the model handles the mix of continuous forecast inputs and cyclically encoded time features without requiring normalisation.

```
In [5]: # Create a copy to protect the original dataframe
xgboost_df = model_df.copy()

# Force all data columns to numeric types, turning errors into NaNs
xgboost_df = xgboost_df.apply(pd.to_numeric, errors="coerce")

# Convert the dataframe index to datetime format
xgboost_df.index = pd.to_datetime(xgboost_df.index)

# Arrange the rows chronologically by their datetime index
xgboost_df = xgboost_df.sort_index()

display(xgboost_df.dtypes)
```

IE Wind Forecast (MW)	float64
GB Wind Forecast (MW)	float64
IE Solar Forecast (MW)	float64
GB Solar Forecast (MW)	float64
IE Demand Forecast (MW)	float64
GB Demand Forecast (MW)	float64
EWIC Actual Flow (MW)	float64
Moyle Actual Flow (MW)	float64
Greenlink Actual Flow (MW)	float64
IE DA Prices (€)	float64
GB DA Prices (€)	float64
Net IC Flows (MW)	float64
DA Price Spread (€)	float64
GB Net Position (MW)	float64
IE Net Position (MW)	float64
IE Wind Penetration	float64
GB Wind Penetration	float64
EWIC Capacity (MW)	float64
Greenlink Capacity (MW)	float64
Moyle Capacity (MW)	float64
Total IC Capacity (MW)	float64
dtype:	object

Time and Lag Features

Hour, month and day of week are encoded as sine/cosine pairs to preserve continuity across boundaries (e.g. 23:30 into 00:00). A weekend flag is also included.

Lagged flows are added at 24h, 48h and 168h, along with a direction flag for each. Rolling mean, std, max and min are computed over the 24 hours ending at t-24h. The shift is applied before the rolling window to prevent leakage.

Capacity utilisation is the absolute flow as a share of total available capacity, lagged 24 hours.

```
In [6]: # Time features - cyclical encoding preserves continuity across boundaries (e.g. 23:00 to 00:00)
xgboost_df["hour_sin"] = np.sin(2 * np.pi * xgboost_df.index.hour / 24)
xgboost_df["hour_cos"] = np.cos(2 * np.pi * xgboost_df.index.hour / 24)
xgboost_df["month_sin"] = np.sin(2 * np.pi * xgboost_df.index.month / 12)
xgboost_df["month_cos"] = np.cos(2 * np.pi * xgboost_df.index.month / 12)
xgboost_df["dow_sin"] = np.sin(2 * np.pi * xgboost_df.index.dayofweek / 7)
xgboost_df["dow_cos"] = np.cos(2 * np.pi * xgboost_df.index.dayofweek / 7)
xgboost_df["is_weekend"] = (xgboost_df.index.dayofweek >= 5).astype(int)

# Lagged flow features - shift(48)=24h, shift(96)=48h, shift(336)=168h
for lag, label in [(48, "24h"), (96, "48h"), (336, "168h")]:
    xgboost_df[f"Net IC Flow -{label}"] = xgboost_df["Net IC Flows (MW)"].shift(lag)
    xgboost_df[f"Flow Direction -{label}"] = (xgboost_df["Net IC Flows (MW)"].shift(lag) > 0).astype(int)

# Rolling stats over the 24h window ending at t-24h - shift(48) prevents Leakage
xgboost_df["IC Rolling Mean -24h"] = xgboost_df["Net IC Flows (MW)"].shift(48).rolling(48).mean()
xgboost_df["IC Rolling Std -24h"] = xgboost_df["Net IC Flows (MW)"].shift(48).rolling(48).std()
xgboost_df["IC Rolling Max -24h"] = xgboost_df["Net IC Flows (MW)"].shift(48).rolling(48).max()
xgboost_df["IC Rolling Min -24h"] = xgboost_df["Net IC Flows (MW)"].shift(48).rolling(48).min()

# Capacity utilisation - how close flow was to max available capacity, Lagged 24h
xgboost_df["Capacity Utilisation -24h"] = (
    xgboost_df["Net IC Flows (MW)"].shift(48).abs() / xgboost_df["Total IC Capacity (MW)"].shift(48)
)

print(f>Date range: {xgboost_df.index[0]} to {xgboost_df.index[-1]})

# Define the target variable
target = "Net IC Flows (MW)"

# Exclude the target and its components to prevent data Leakage
leak_cols = [
    "Net IC Flows (MW)",
    "EWIC Actual Flow (MW)",
    "Moyle Actual Flow (MW)",
    "Greenlink Actual Flow (MW)",
]

features = [col for col in xgboost_df.columns if col not in leak_cols]

print(f"Target: {target}")
print(f"Features: ({len(features)}):")
for f in features:
    print(f"    {f}")
```

Date range: 2025-04-18 23:00:00 to 2026-07-27 00:00:00

Target: Net IC Flows (MW)

Features (35):

- IE Wind Forecast (MW)
- GB Wind Forecast (MW)
- IE Solar Forecast (MW)
- GB Solar Forecast (MW)
- IE Demand Forecast (MW)
- GB Demand Forecast (MW)
- IE DA Prices (€)
- GB DA Prices (€)
- DA Price Spread (€)
- GB Net Position (MW)
- IE Net Position (MW)
- IE Wind Penetration
- GB Wind Penetration
- EWIC Capacity (MW)
- Greenlink Capacity (MW)
- Moyle Capacity (MW)
- Total IC Capacity (MW)
- hour_sin
- hour_cos
- month_sin
- month_cos
- dow_sin
- dow_cos
- is_weekend
- Net IC Flow -24h
- Flow Direction -24h
- Net IC Flow -48h
- Flow Direction -48h
- Net IC Flow -168h
- Flow Direction -168h
- IC Rolling Mean -24h
- IC Rolling Std -24h
- IC Rolling Max -24h
- IC Rolling Min -24h
- Capacity Utilisation -24h

Train/Test Split

The data is split 80/20 chronologically. No shuffling is applied as the model relies on time-based lag features and random shuffling would cause leakage.

```
In [7]: X = xgboost_df[features].astype(float)
y = xgboost_df[target].astype(float)

split_idx = int(len(X) * 0.8)

X_train = X.iloc[:split_idx]
X_test = X.iloc[split_idx:]
y_train = y.iloc[:split_idx]
y_test = y.iloc[split_idx:]

print("80% train, 20% split")
print(f"Train: {X_train.index[0]} to {X_train.index[-1]} ({len(X_train)} rows)")
print(f"Test: {X_test.index[0]} to {X_test.index[-1]} ({len(X_test)} rows)")
```

80% train, 20% split

Train: 2025-04-18 23:00:00 to 2026-04-24 03:00:00 (17769 rows)

Test: 2026-04-24 03:30:00 to 2026-07-27 00:00:00 (4443 rows)

Hyperparameter Tuning and Model Training

Optuna is used to search for the best hyperparameters over 50 trials, minimising mean RMSE across 5-fold time series cross-validation. Folds are ordered chronologically so no future data leaks into training.

Early stopping is applied within each fold to avoid overfitting. The test set is held out entirely during tuning.

Once the best parameters are found, the model is retrained on the full training set and evaluated on the held-out test set using MAE, RMSE and R^2 .

```
In [8]: optuna.logging.set_verbosity(optuna.logging.WARNING)

# 5-fold time series cross-validation - test set is never seen during tuning
tscv = TimeSeriesSplit(n_splits=5)

def objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 200, 2000), # number of boosting rounds
        "learning_rate": trial.suggest_float("learning_rate", 0.001, 0.1, log=True), # step size shrinkage to reduce overfitting
        "max_depth": trial.suggest_int("max_depth", 3, 8), # maximum depth of each tree
        "subsample": trial.suggest_float("subsample", 0.5, 1.0), # fraction of rows sampled per tree
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0), # fraction of features sampled per tree
        "colsample_bylevel": trial.suggest_float("colsample_bylevel", 0.5, 1.0), # fraction of features sampled per tree level
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 20), # minimum sum of instance weight in a leaf
        "gamma": trial.suggest_float("gamma", 0.0, 1.0), # minimum loss reduction required to split a node
        "reg_alpha": trial.suggest_float("reg_alpha", 0.0, 2.0), # L1 regularisation on leaf weights
        "reg_lambda": trial.suggest_float("reg_lambda", 0.0, 2.0), # L2 regularisation on leaf weights
        "random_state": 42,
        "eval_metric": "rmse",
        "n_jobs": -1,
    }

    fold_rmse = []
    for train_idx, val_idx in tscv.split(X_train):
        X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
        y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

        n = xgb.XGBRegressor(**params, early_stopping_rounds=50)
```

```

        preds = m.predict(X_val)
        fold_rmse.append(np.sqrt(mean_squared_error(y_val, preds)))

    return np.mean(fold_rmse)

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=10, show_progress_bar=True)

print(f"Best CV RMSE: {round(study.best_value, 2)} MW")
print(f"Best params: {study.best_params}")

# Retrain on full training set with best hyperparameters
model = xgb.XGBRegressor(
    **study.best_params,
    random_state=42,
    early_stopping_rounds=50,
    eval_metric="rmse",
    n_jobs=-1,
)

model.fit(
    X_train, y_train,
    eval_set=[(X_test, y_test)],
    verbose=100,
)

y_pred = model.predict(X_test)
mae = round(mean_absolute_error(y_test, y_pred), 2)
rmse = round(np.sqrt(mean_squared_error(y_test, y_pred)), 2)
r2 = round(r2_score(y_test, y_pred), 4)

print(f"MAE: {mae} MW")
print(f"RMSE: {rmse} MW")
print(f"R²: {r2}")

```

```

0%|          | 0/10 [00:00<?, ?it/s]

```

Best CV RMSE: 413.04 MW

Best params: {'n_estimators': 1729, 'learning_rate': 0.006858272283352329, 'max_depth': 4, 'subsample': 0.5485225754416276, 'colsample_bytree': 0.9465987491918076, 'colsample_bylevel': 0.5968940510937469, 'min_child_weight': 18, 'gamma': 0.8933300963763984, 'reg_alpha': 0.8441918950329723, 'reg_lambda': 0.7466338144377711}

```

[0]    validation_0-rmse:483.29565
[100]  validation_0-rmse:395.69905
[200]  validation_0-rmse:361.20149
[300]  validation_0-rmse:346.42233
[400]  validation_0-rmse:340.73005
[500]  validation_0-rmse:337.97475
[600]  validation_0-rmse:337.29888
[700]  validation_0-rmse:336.99663
[745]  validation_0-rmse:337.06760

```

MAE: 256.15 MW

RMSE: 336.89 MW

R²: 0.5167

Feature Importance:

```

In [9]: trained_features = model.get_booster().feature_names

importance = pd.Series(
    model.feature_importances_, index=trained_features
).sort_values(ascending=True)

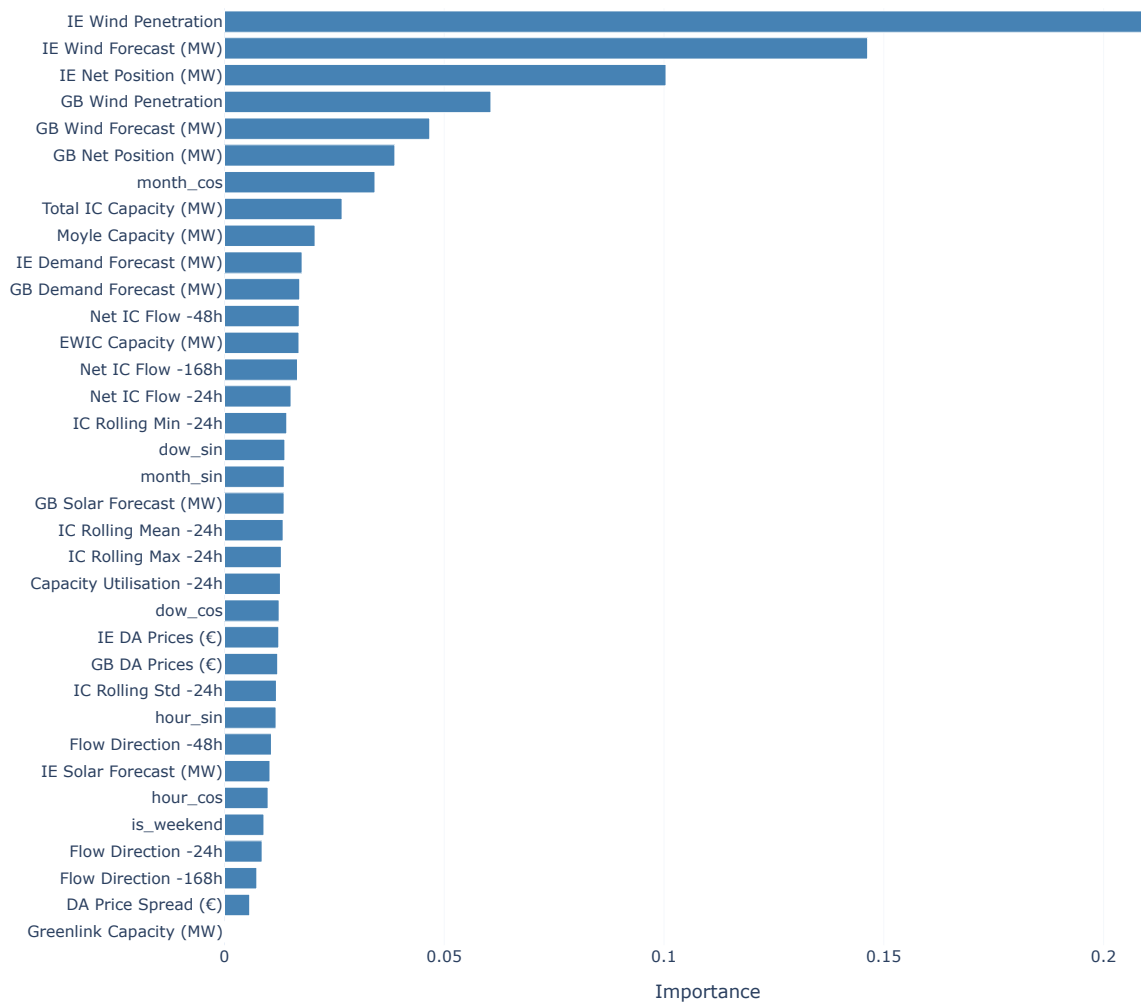
fig2 = go.Figure(go.Bar(
    x=importance.values,
    y=importance.index,
    orientation="h",
    marker_color="steelblue"
))

fig2.update_layout(
    title="XGBoost Feature Importance",
    xaxis_title="Importance",
    template="plotly_white",
    height=900,
    width=1000
)

fig2.show()

```

XGBoost Feature Importance



Model Results:

```
In [10]: now = pd.Timestamp.now()

pred_series = pd.Series(y_pred, index=X_test.index)
upcoming = pred_series[pred_series.index >= now]
upcoming_df = upcoming.reset_index()
upcoming_df.columns = ["Time (UTC)", "Predicted Net IC Flow (MW)"]
upcoming_df["Predicted Net IC Flow (MW)"] = upcoming_df["Predicted Net IC Flow (MW)"].round(1)
upcoming_df["Direction"] = upcoming_df["Predicted Net IC Flow (MW)"].apply(
    lambda x: "IE importing from GB" if x > 0 else "IE exporting to GB"
)
print(f"\nNext predicted IC flows from {now.strftime('%Y-%m-%d %H:%M')} UTC:\n")
display(upcoming_df)

# Plot of the full test set showing Actual vs Forecasted IC Flow
fig1 = go.Figure()

fig1.add_trace(go.Scatter(
    x=X_test.index, y=y_test,
    name="Actual", line=dict(color="black", width=1)
))
fig1.add_trace(go.Scatter(
    x=X_test.index, y=y_pred,
    name="Predicted", line=dict(color="blue", width=1), opacity=0.7
))
fig1.add_hline(y=0, line_dash="dash", line_color="red", opacity=0.5)
fig1.add_vline(
    x=now.timestamp() * 1000,
    line_dash="dot", line_color="grey", opacity=0.8,
    annotation_text="Now", annotation_position="top right"
)
fig1.update_layout(
    title="Actual vs Predicted Net IC Flows - Full Test Set",
    xaxis_title="Time (UTC)",
    yaxis_title="IC Flow (MW)",
    hovermode="x unified",
    template="plotly_white",
    height=600,
    width=1000,
    legend=dict(x=0, y=-0.2, orientation="h")
)
fig1.show()
```

```
window_end = X_test.index[-1]
window_start = window_end - pd.Timedelta(days=7)

mask = (X_test.index >= window_start)
x_zoom = X_test.index[mask]
y_actual = y_test[mask]
y_hat = pd.Series(y_pred, index=X_test.index)[mask]

fig2 = go.Figure()

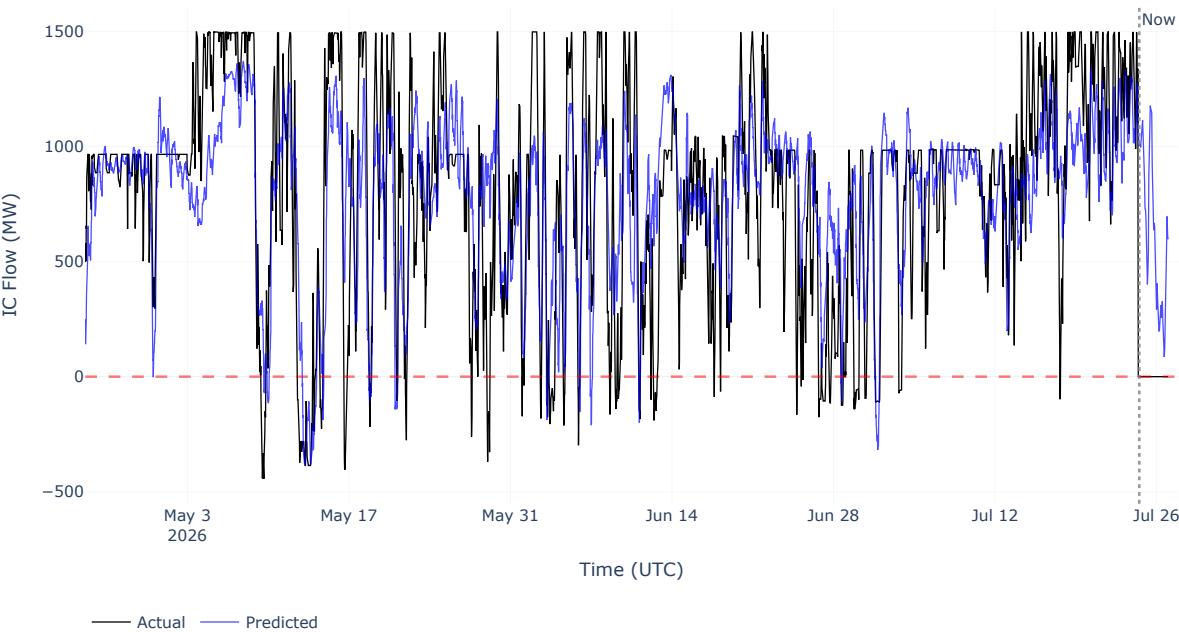
fig2.add_trace(go.Scatter(
    x=x_zoom, y=y_actual,
    name="Actual", line=dict(color="black", width=1.5)
))
fig2.add_trace(go.Scatter(
    x=x_zoom, y=y_hat,
    name="Predicted", line=dict(color="blue", width=1.5), opacity=0.7
))
fig2.add_hline(y=0, line_dash="dash", line_color="red", opacity=0.5)
fig2.add_vline(
    x=now.timestamp() * 1000,
    line_dash="dot", line_color="grey", opacity=0.8,
    annotation_text="Now", annotation_position="top right"
)
fig2.update_layout(
    title="Actual vs Predicted Net IC Flows - Last 7 and Next 2 Days of Test Set",
    xaxis_title="Time (UTC)",
    yaxis_title="IC Flow (MW)",
    hovermode="x unified",
    template="plotly_white",
    height=600,
    width=1000,
    legend=dict(x=0, y=-0.2, orientation="h")
)
fig2.show()
```

Next predicted IC flows from 2026-07-24 11:05 UTC:

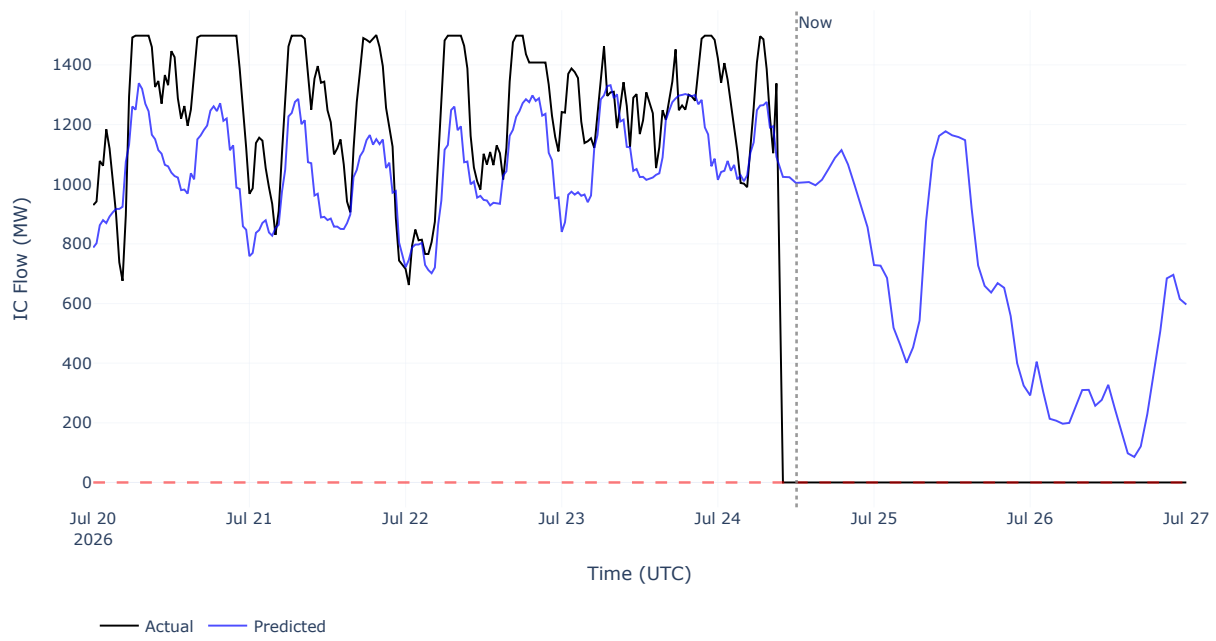
	Time (UTC)	Predicted Net IC Flow (MW)	Direction
0	2026-07-24 12:00:00	1004.299988	IE importing from GB
1	2026-07-24 13:00:00	1004.700012	IE importing from GB
2	2026-07-24 14:00:00	1007.099976	IE importing from GB
3	2026-07-24 15:00:00	996.400024	IE importing from GB
4	2026-07-24 16:00:00	1014.900024	IE importing from GB
...	...	...	...
56	2026-07-26 20:00:00	507.700012	IE importing from GB
57	2026-07-26 21:00:00	684.299988	IE importing from GB
58	2026-07-26 22:00:00	696.200012	IE importing from GB
59	2026-07-26 23:00:00	615.099976	IE importing from GB
60	2026-07-27 00:00:00	596.599976	IE importing from GB

61 rows × 3 columns

Actual vs Predicted Net IC Flows - Full Test Set



Actual vs Predicted Net IC Flows - Last 7 and Next 2 Days of Test Set



5.3 Future Improvements

A number of extensions were identified but not yet implemented, listed below in rough priority order.

1. **Per-interconnector ensemble:** Train a separate XGBoost model for each of Moyle, EWIC, and Greenlink, each with its own tuned hyperparameters, and sum the three predictions to form the total net flow forecast. This should better capture each cable's individual behaviour, constraints, and response to outages than a single pooled model.
2. **Capacity/MEC features:** Add a utilisation ratio ($|\text{flow}|/\text{MEC}$) and a binary flag for each interconnector approaching its Maximum Export Capacity (e.g. utilisation $> 95\%$), along with directional export/import constraint flags, so the model can learn when price signals can no longer move flow due to a physical ceiling.
3. **Flow ramp features:** Add short-horizon flow derivatives (30-min, 1hr, 3hr changes) and a ramp volatility measure, to help the model capture rapid flow reversals and physically realistic movement in response to price shocks.
4. **Price spread dynamics:** Add the rate of change in the day-ahead spread (30-min, 1hr, daily) alongside the absolute spread, since large movements in the spread may explain flow changes better than the level alone.
5. **Forecast revision features:** Add wind and demand forecast changes and forecast error terms, which should be particularly informative during high-wind or balancing-driven intraday flow movements.
6. **Outage/availability features:** Incorporate interconnector outage and availability data more directly as a feature rather than only as a capacity adjustment, to help the model anticipate flow behaviour around planned and unplanned outages.

The most promising of these is the per-interconnector ensemble, which would replace the current single-model architecture with three independently trained models summed at inference time:

$$\text{Total IC Flow} = \widehat{\text{Moyle}} + \widehat{\text{EWIC}} + \widehat{\text{Greenlink}}$$

This would also make the model's feature importances more interpretable, since drivers specific to one cable would no longer be averaged out across all three.

5.4 Skills and Learning Outcomes

This project was my first end-to-end machine learning build on the desk, and it developed skills across the full pipeline rather than just model fitting. Extracting and joining data from the market data warehouse via SQL required understanding how forecast versions, outages, and settlement data were structured across multiple tables, while building the feature set — cyclical time encoding, lagged flow history, and rolling statistics — reinforced how easily time series leakage can be introduced without careful shifting, and the discipline needed to guard against it.

Working with `Optuna` and `TimeSeriesSplit` deepened my understanding of hyperparameter tuning for time-dependent data, where standard random cross-validation is not valid and folds must respect chronological order to give a realistic estimate of out-of-sample performance. Evaluating the model honestly — accepting an R^2 around 0.52 rather than over-engineering the feature set to chase a better in-sample fit — was a useful lesson in resisting overfitting under pressure to produce a stronger headline number.

Finally, presenting a probabilistic, imperfect forecast to a trading audience required framing the model's outputs and limitations clearly, so that its predictions could be used to inform trading decisions without being mistaken for certainty. This reinforced the importance of communicating model uncertainty honestly, particularly where the output feeds directly into commercial decision-making.

6 Machine Learning Demand Forecasting Ensemble

Another project undertaken during my placement involved the development of a machine learning ensemble model to more accurately forecast system-wide electricity demand. ElectroRoute currently uses multiple demand forecasts, however these rarely agree with one another, creating uncertainty when making trading decisions.

The project began as a straightforward analysis requested by one of the traders, who wanted to investigate the difference in error between the two primary forecasts used by the team. The first is provided by the TSO, EirGrid, and the second by an external forecast provider, whose name has been redacted from this document for confidentiality reasons. Upon completing the initial analysis, it became clear that each forecast had distinct strengths and weaknesses. The EirGrid forecast consistently performed better at predicting the troughs of the demand curve, while the external provider was more accurate in capturing peak demand periods. Recognising this complementary behaviour, the decision was made to take the initiative and attempt to develop a blended ensemble model that could leverage the strengths of both forecasts, producing a more accurate and robust prediction of system demand.

6.1 Model Selection

For the ensemble model, ridge regression was selected as the most appropriate modelling approach. Ridge regression is a form of linear regression that introduces a regularisation term, penalising large coefficients and reducing the risk of overfitting. Rather than treating the two forecasts equally, the model learns an optimal weighting for each based on historical performance, allowing it to allocate more weight to whichever forecast has proven more accurate. Given the relatively small number of input features and the need for a transparent and explainable model that could be presented to the trading team, ridge regression struck the right balance between simplicity, robustness and predictive performance.

Standard linear regression uses a method called ordinary least squares (OLS), which finds the coefficients that minimise the sum of squared differences between predictions and actual values. However, when input features are highly correlated—as is the case with two forecasts predicting the same quantity—OLS can produce unstable coefficient estimates that fluctuate dramatically with small changes to the training data.

Ridge regression addresses this by adding a penalty term proportional to the squared magnitude of the coefficients. The model minimises the following loss function:

$$\mathcal{L}(\beta) = \sum_{i=1}^n \left(y_i - \mathbf{x}_i^\top \beta \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (1)$$

The first term measures how well the model fits the data (the standard regression objective), while the second term penalises large coefficient values. The parameter λ controls the strength of this penalty—larger values produce more conservative, stable estimates. This regularisation prevents the model from assigning extreme weights to either forecast, producing a more balanced and generalisable ensemble. Table 4 provides a summary of the notation used.

6.2 Implementation

The ridge regression model was implemented in Python using the `scikit-learn` library. The core function, `method_ml_forecast()`, takes the combined forecast dataset as input and returns both the blended forecast and a summary of the model weights and error metrics. The key steps of the implementation are outlined below.

Symbol	Description
$\mathcal{L}(\beta)$	Loss function (the value to be minimised)
n	Number of observations in the training set
y_i	Actual demand value for observation i
$\mathbf{x}_i$	Input features for observation i (the two forecasts)
β	Vector of coefficients (weights) to be learned
$\mathbf{x}_i^\top \beta$	Model prediction for observation i
p	Number of predictors (in this case, 2)
β_j	Coefficient for predictor j
λ	Regularisation parameter (set to 1.0 in this implementation)

Table 4: Notation used in the ridge regression loss function.

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
```

Data Preparation and Train/Test Split

```
X = df[['external_forecast', 'eirgrid']].values # Input features: the two
↪ forecasts
y = df['actual'].values # Target: actual system demand

# Split into 80% training and 20% test data
# Randomised split ensures the model sees a variety of conditions in both
↪ sets,
# producing a more robust test of generalisation
X_train, X_test, y_train, y_test, idx_train, idx_test = train_test_split(
    X, y, indices,
    test_size=0.2,
    random_state=42, # Ensures reproducibility
    shuffle=True
)
```

Fitting the Ridge Regression Model

```
# Fit the ridge regression model
# alpha=1.0 controls regularisation strength - prevents overfitting
# The model learns: actual = w1*external_forecast + w2*eirgrid + intercept
# where w1 and w2 are the optimal weights for each forecast
model = Ridge(alpha=1.0)
model.fit(X_train, y_train)

# Normalise the coefficients so that the weights sum to 1
# This makes the output interpretable as a weighted blend of the two forecasts
weights = model.coef_ / model.coef_.sum() if model.coef_.sum() != 0 else
↪ model.coef_
```

Generating the Blended Forecast and Evaluating Performance

```
# Use the learned weights to generate the blended forecast across all data
df['forecast'] = model.predict(X)
df['error'] = df['forecast'] - df['actual'] # Positive = over-forecast,
↪ Negative = under-forecast
```

```
# Evaluate model performance on both training and unseen test data
weights_info = {
    'external_forecast' : weights[0],
    'eirgrid'           : weights[1],
    'intercept'         : model.intercept_,
    'train_rmse'         : np.sqrt(((model.predict(X_train) - y_train) **
    ↪ 2).mean()),
    'test_rmse'          : np.sqrt(((model.predict(X_test) - y_test) **
    ↪ 2).mean()),
    'train_mae'          : np.abs(model.predict(X_train) - y_train).mean(),
    'test_mae'           : np.abs(model.predict(X_test) - y_test).mean(),
}
```

6.3 Model Output and Results

```
=====
ML RIDGE REGRESSION
=====

Split Type: Randomised
Random State: 42
Train/Test Split: 12982 / 3246 samples
Weights: External Forecast=0.625, EirGrid=0.375
Intercept: -11.808
Train RMSE: 132.94 MW | Train MAE: 101.85 MW
Test RMSE: 134.07 MW | Test MAE: 102.66 MW
=====
```

The model was backtested over a year from 01/04/2025 to 01/04/2026. Its output revealed several interesting findings. The ridge regression model assigned a weight of 0.625 to the External Forecast and 0.375 to the EirGrid forecast, meaning the blended prediction drew more heavily from the External Forecast. This was consistent with the findings from the initial error analysis, which suggested that the External Forecast tended to perform better across a broader range of conditions, particularly during peak demand periods. The small negative intercept of -11.808 MW was a minor bias correction term learned by the model, indicating a slight systematic over-prediction in the combined forecasts that the model adjusted for.

A key indicator of model quality was the comparison between the training and test error metrics. The training RMSE of 132.94 MW and test RMSE of 134.07 MW were very close to one another, as were the training MAE of 101.85 MW and test MAE of 102.66 MW. This small gap between training and test performance was a strong sign that the model generalised well to unseen data and had not overfit to the training set. Had the model overfit, one would have expected to see a significantly lower training error relative to the test error. The consistency between the two suggested that the regularisation applied by ridge regression had been effective.

The forecast accuracy comparison tables provided a more comprehensive picture of how the ML Ridge model performed relative to the two individual input forecasts. Looking first at the full dataset results, the ML Ridge model achieved the best performance across the majority of metrics, recording the lowest MAE of 101.85 MW, the lowest RMSE of 132.94 MW, and the lowest MAPE of 2.10%, as well as the highest correlation of 0.9859. The EirGrid forecast performed the weakest across most metrics, with an MAE of 123.19 MW and an RMSE of 164.30 MW, while the External Forecast sat between the two, performing reasonably well but still falling short of the blended model.

The test data results further reinforced these findings. On unseen data, the ML Ridge model again achieved the best MAE of 102.66 MW, the best RMSE of 134.07 MW, and the best MAPE of 2.13%, along with the highest correlation of 0.9859. Notably, the External Forecast achieved the lowest maximum error

Forecast Accuracy Comparison - ML Ridge (Full Data)
Green = Best performer for metric

Forecast	ME (Bias)	MAE	RMSE	MAPE (%)	Std Error	Max Error	Correlation
Forecast	-4.85	109.47	141.90	2.27%	141.83	1299.53	0.9843
EirGrid	-46.74	123.19	163.65	2.59%	156.84	1417.83	0.9815
ML Ridge	0.56	102.01	133.17	2.12%	133.17	1322.02	0.9861

Figure 3: Forecast accuracy comparison across all data (full dataset).

Forecast Accuracy Comparison - ML Ridge (Test Data Only)
Green = Best performer for metric

Forecast	ME (Bias)	MAE	RMSE	MAPE (%)	Std Error	Max Error	Correlation
Forecast	-3.05	109.67	142.55	2.27%	142.54	1282.33	0.9842
EirGrid	-43.84	123.19	164.30	2.57%	158.37	1413.67	0.9812
ML Ridge	2.78	102.66	134.07	2.13%	134.06	1309.73	0.9859

Figure 4: Forecast accuracy comparison on unseen test data only.

of 1282.33 MW on the test set, suggesting it handled extreme demand events slightly better in isolation. However, the ML Ridge model still outperformed both individual forecasts on average error metrics, demonstrating that the weighted blend was able to consistently produce more accurate predictions across the full range of demand conditions. The near-identical performance between the full dataset and test-only results further confirmed that the model had not overfit and was generalising effectively to new data.

In terms of absolute performance, an RMSE of approximately 134 MW and MAE of approximately 103 MW represented a meaningful improvement over using either individual forecast in isolation. The blended ensemble model was therefore able to produce a more stable and accurate prediction of system-wide power demand by combining the complementary strengths of both input forecasts.

Figure 5 illustrates the ML Ridge blended forecast against the actual outturn demand over a sample period. The black line represents the actual outturn, the green line the EirGrid forecast, and the red dashed line the ML Ridge blended forecast. The blended model tracks the outturn more closely than the EirGrid forecast throughout, with the middle panel confirming that the ML Ridge errors remain tightly centred around zero. The lower panel indicates the randomised train/test split, with blue and red dots representing training and test samples respectively.

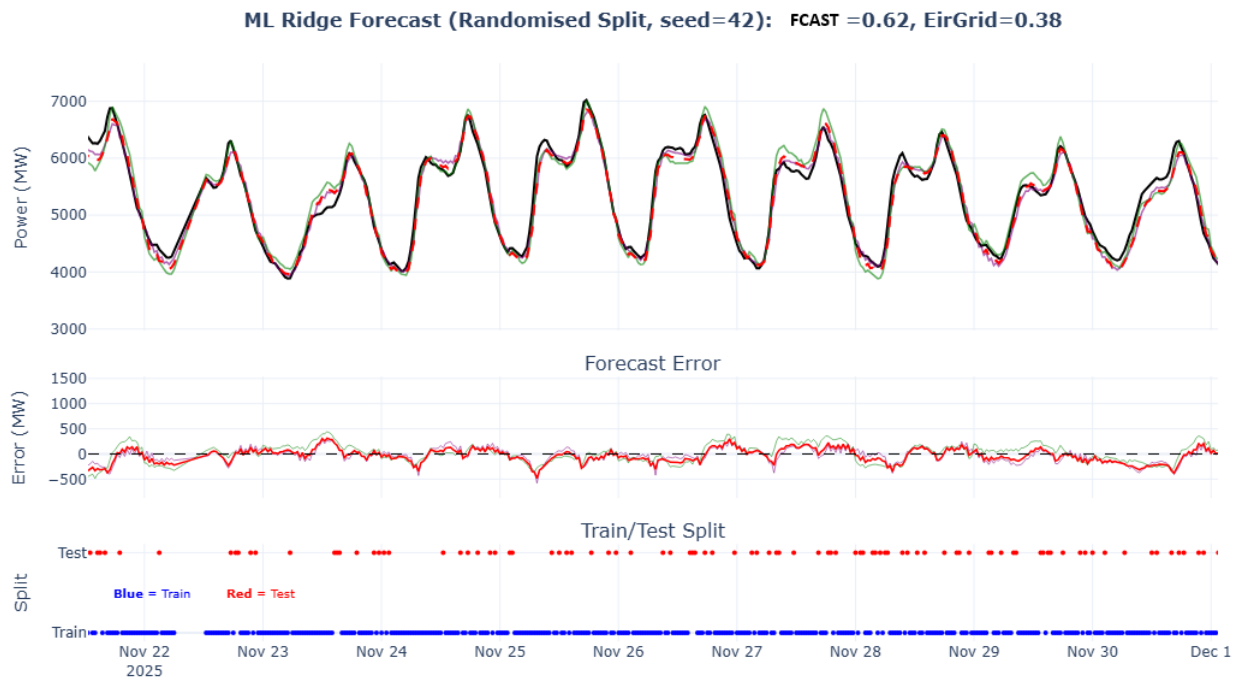


Figure 5: ML Ridge blended forecast vs. actual demand with error and train/test split.



Figure 6: Error distribution, cumulative error, scatter plot, and MAE/RMSE comparison.

6.4 Next Steps

While the ML Ridge blended model demonstrated a meaningful improvement over the individual input forecasts, there are several avenues through which the model could be further developed and refined.

The most immediate opportunity would be to incorporate additional input features beyond the two existing forecasts. Rooftop solar generation is a particularly relevant candidate, as it directly suppresses system demand during daylight hours and is not always fully captured in traditional demand forecasts. As rooftop solar capacity continues to grow across Ireland, its impact on outturn demand is likely to become increasingly significant, and failing to account for it could introduce a systematic bias into any demand forecast. Integrating rooftop solar generation estimates as an additional input feature could therefore improve forecast accuracy, particularly during summer months and periods of high irradiance. Beyond solar, other variables such as temperature, wind generation, and public holiday indicators could also be explored as additional inputs, given their known influence on electricity demand patterns. Extending the model to incorporate these features would likely require moving beyond ridge regression towards a more flexible modelling framework, such as gradient boosting or a neural network, which could better capture the non-linear relationships between demand and its drivers.

Finally, the model could be further developed to produce probabilistic forecasts rather than a single point estimate, providing the trading team with a measure of forecast uncertainty that could be directly incorporated into trading decisions. The model will continue to be worked on over the coming weeks, and hopefully by the end of my internship I will have contributed another valuable tool to the desk.

6.5 Skills and Learning Outcomes

Working with `scikit-learn` provided hands-on experience implementing machine learning pipelines in Python, including data preparation, model fitting, and performance evaluation. Applying ridge regression in a practical context developed a stronger understanding of how machine learning techniques can be applied to real-world forecasting problems. The project also reinforced broader statistical concepts around model validation and the interpretation of error metrics. Finally, analysing the drivers of forecast error developed a greater appreciation of the physical factors that influence electricity demand, including the growing impact of rooftop solar generation on system demand outturn.

7 Paper Trading and Trading Strategy

As part of the internship, paper trading was carried out on the REFIT asset portfolio in the Day-Ahead Market. Paper trading involves submitting simulated trades against live market data without actual financial exposure — the traded volumes do not enter the real auction and therefore do not influence clearing prices or volumes. While this removes the market impact element of real trading, it provides a valuable environment for developing trading intuition, testing strategies, and building a structured decision-making process against real market outcomes.

Over the course of the internship, a trading strategy framework was developed and iteratively refined based on the daily feedback loop that paper trading provided. Each morning, the available generation forecasts, weather data, system fundamentals, and market conditions were reviewed, a position was constructed, and after the auction cleared, the result was evaluated to identify what worked, what did not, and why. This section describes the individual variables that fed into that process and the effect each one tends to have on generation, price, and system position. No single variable is decisive on its own, and the same variable can point in different directions depending on what else is happening on a given day; the framework is a way of structuring judgement, not a fixed set of rules to apply mechanically. This is expanded on further in Section 7.5.

7.1 Underselling and Overselling

The most fundamental decision in renewable energy trading is how much of the forecast generation to sell at the Day-Ahead auction versus holding volume for later markets. This is framed in terms of underselling and overselling relative to the generation forecast, and the balance between the two shifts with the factors described throughout this section.

Underselling — selling less than the forecast — becomes more attractive as:

- **Forecast uncertainty rises:** selling 100% of an unreliable forecast risks being short and paying high imbalance prices to buy back the shortfall.
- **Imbalance penalties are asymmetric:** where the cost of being short exceeds the cost of being long, underselling provides a buffer.
- **Later prices are expected to be higher:** if IDA or imbalance prices are expected to exceed DA prices, holding back volume allows it to be sold later at better prices.
- **Flexibility has value:** selling only a portion at DA preserves optionality to adjust in intraday auctions as forecasts improve.

Overselling — selling more than the forecast — becomes more attractive as:

- **Forecasts run conservative:** if forecast providers consistently under-predict generation, selling above the forecast captures the expected upside.
- **DA prices are elevated:** if prices are high and expected to fall, locking in revenue early becomes more valuable.
- **The system is expected to be long:** if the grid is expected to be long and imbalance prices low, overselling and buying back cheaply in the Balancing Market can be profitable.

7.2 Weather-Driven Factors

Unlike conventional commodities, renewable energy generation is fundamentally weather-dependent. Ireland's position at the western edge of Europe exposes it to Atlantic weather systems that can change conditions dramatically within hours. The following factors were monitored daily as part of the paper trading workflow, each carrying its own implications for expected generation and price.

Wind Speed

Wind speed is the most critical variable for renewable generation in Ireland given the high penetration of wind power on the grid. Table 5 summarises the typical effect of different wind conditions on generation and price.

Condition	Effect on Generation	Effect on Price/Risk
Low (<5 m/s)	Under-generation likely	Prices tend to rise; shortfall risk if oversold
Moderate (8–15 m/s)	Good, stable generation	Forecasts tend to be reliable
High (15–25 m/s)	Full generation	Prices often suppressed or negative
Storm (>25 m/s)	Cut-out risk: output can fall from 100% to 0%	High uncertainty, large potential shortfall
Atlantic fronts	Rapid changes in generation	DA forecasts become unreliable

Table 5: Wind speed conditions and their typical effect on generation and price.

Wind turbines operate within specific thresholds: below the cut-in speed (typically 3–3.5 m/s) turbines do not generate, and above the cut-out speed (typically 25 m/s) turbines shut down for safety. Forecasts near these thresholds carry particular risk, as a small error can mean the difference between full output and zero generation.

Temperature, Cloud Cover, and Precipitation

Temperature is a primary driver of electricity demand in Ireland. Unlike continental Europe, Irish demand is predominantly driven by heating rather than cooling, so cold weather is the key factor behind demand-driven price increases; mild or warm conditions have the opposite effect, suppressing demand and easing pressure on price.

Cloud cover directly affects solar generation output. Ireland is typically overcast, meaning solar output is generally low. Clear skies and heavily overcast conditions are both relatively predictable, whereas broken or variable cloud introduces the greatest uncertainty in solar output. Global Horizontal Irradiance (GHI) correlates directly with solar panel output — high GHI (>800 W/m²) tends to produce reliable generation, while variable GHI introduces the same forecast uncertainty as broken cloud.

Rain and fog reduce solar irradiance, while heavy precipitation events are often associated with changing wind patterns. Atmospheric pressure serves as a useful indicator of weather stability more broadly: high pressure brings settled, predictable conditions, while low or falling pressure signals greater uncertainty across generation and demand alike.

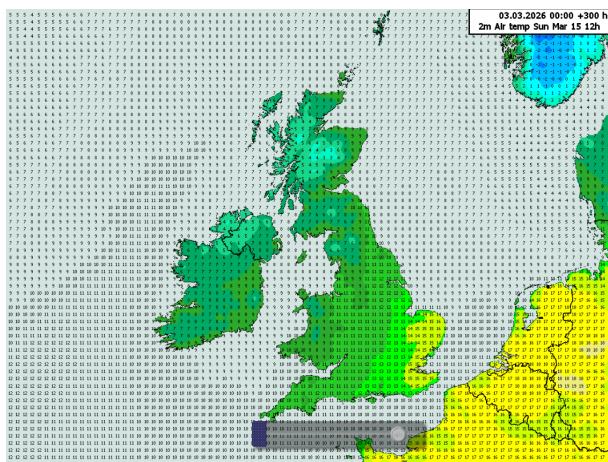


Figure 7: Forecasted temperatures.

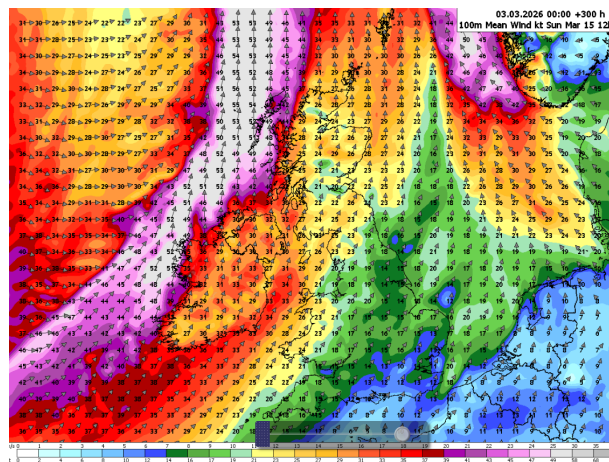


Figure 8: Forecasted wind speeds.

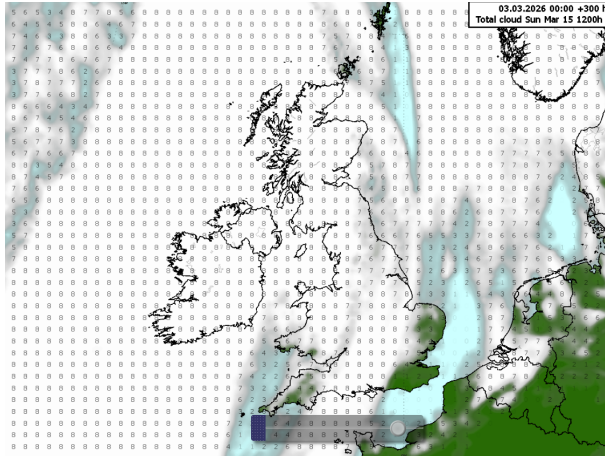


Figure 9: Forecasted cloud cover.

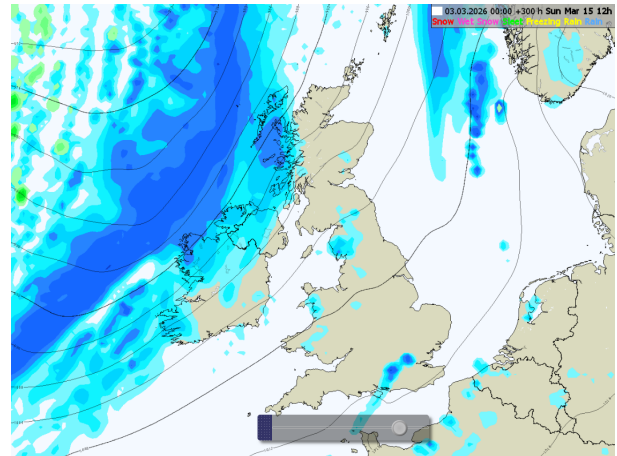


Figure 10: Forecasted precipitation.

7.3 System and Market Factors

Interconnector Flows

The EWIC and Moyle interconnectors link the I-SEM to the GB market, and flow direction has a direct effect on price. When Ireland is exporting to GB, additional demand is being drawn from the Irish system, which tends to support I-SEM prices; when importing from GB, the extra supply tends to compress them. Constrained interconnectors — where flow is limited relative to the price signal — introduce additional price volatility, as the market cannot arbitrage the spread away as freely.

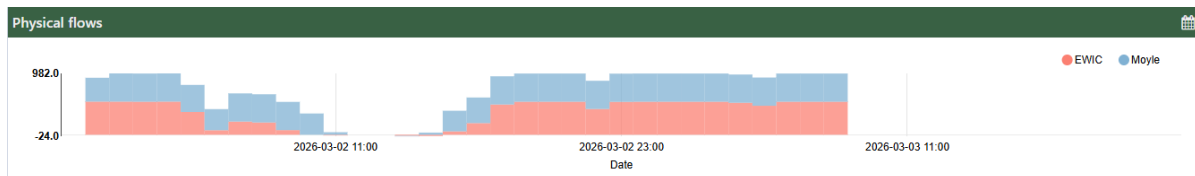


Figure 11: Power flows across the I-SEM/GB interconnectors. Positive values indicate an import.

Curtailment and SNSP

System Non-Synchronous Penetration (SNSP) measures the proportion of demand being met by non-synchronous generation. The current SNSP limit in Ireland is approximately 75%, and the closer the system sits to this limit, the greater the risk of curtailment. Below around 60%, curtailment is unlikely; between 60–70% it becomes a real possibility and TSO dispatch instructions are worth monitoring closely; above 75%, curtailment is highly likely, and since generators are not compensated for constrained volume, this materially changes the risk of holding a large sold position.

Net Imbalance Volume and System Position

The Net Imbalance Volume (NIV) indicates whether the system as a whole is over-generating (long) or under-generating (short) relative to scheduled positions, and it directly determines who is rewarded and who is penalised at settlement. Being on the opposite side of the system is favourable: being long when the system is short means receiving a high imbalance price for excess generation, while being short when the system is long means paying a low imbalance price to cover the shortfall. Being on the same side as the

system has the opposite effect — being short when the system is short means paying a high imbalance price, and being long when the system is long means receiving a low price for excess generation.

This gives NIV expectations a direct bearing on position sizing: a system expected to be long makes underselling more attractive, while a system expected to be short makes overselling more attractive, and genuine uncertainty about system direction is itself a reason to sell closer to forecast. An internal tool was used to estimate the probability of the system being long or short and the likely imbalance price, which fed into the daily assessment of position.

7.4 System-Wide Forecasts

Demand Forecasts

System-wide demand forecasts from multiple providers were reviewed daily, looking at three things: how forecast demand compared to normal levels, as an early read on whether the system was likely to be long or short; how DA and intraday forecast revisions compared, as a signal of whether prices were more likely rising or falling; and how closely the External and EirGrid forecasts agreed with each other, as a proxy for confidence in the read. Agreement between providers generally supported taking a more decisive view; divergence was a signal that the picture was still unclear.

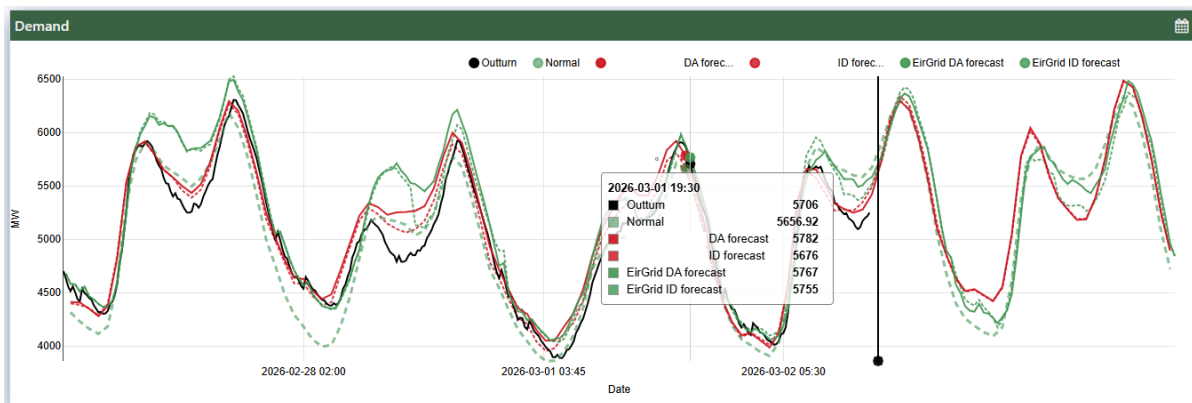


Figure 12: Actual and forecasted system demand from multiple providers.

Wind Generation Forecasts

Wind generation forecasts were assessed against a similar set of questions. Wind level was read alongside SNSP to gauge curtailment risk, since high wind combined with high SNSP points toward a system that is both long and constrained. The true (uncurtailed) wind figure was compared against the outturn forecast to estimate expected curtailment separately from raw generation. DA and intraday forecast revisions were also compared with each other: a rising intraday wind figure relative to DA tends to coincide with falling prices, while a falling figure tends to coincide with rising prices.

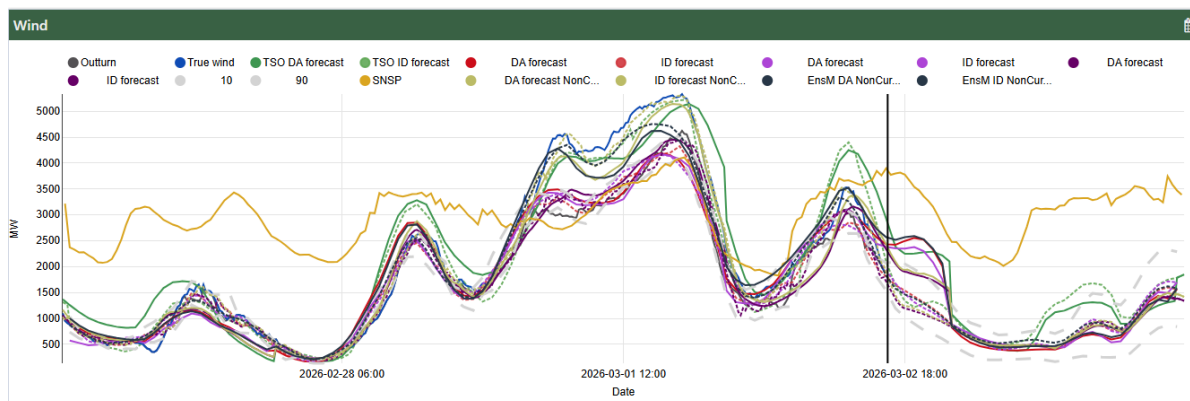


Figure 13: System-wide actual and forecasted wind generation.

Solar Generation Forecasts

Solar forecasts were assessed with particular attention to the P10/P90 spread as a measure of forecast confidence — a narrow spread relative to the median indicates a more reliable forecast, while a wide spread signals greater uncertainty in expected output. Agreement across forecast providers played a similar role to the demand and wind forecasts: close agreement supported more confidence in the figure, while divergence introduced doubt. Historical bias was tracked as well, so that a provider with a recent pattern of over- or under-forecasting in similar conditions could be weighted accordingly against the others.

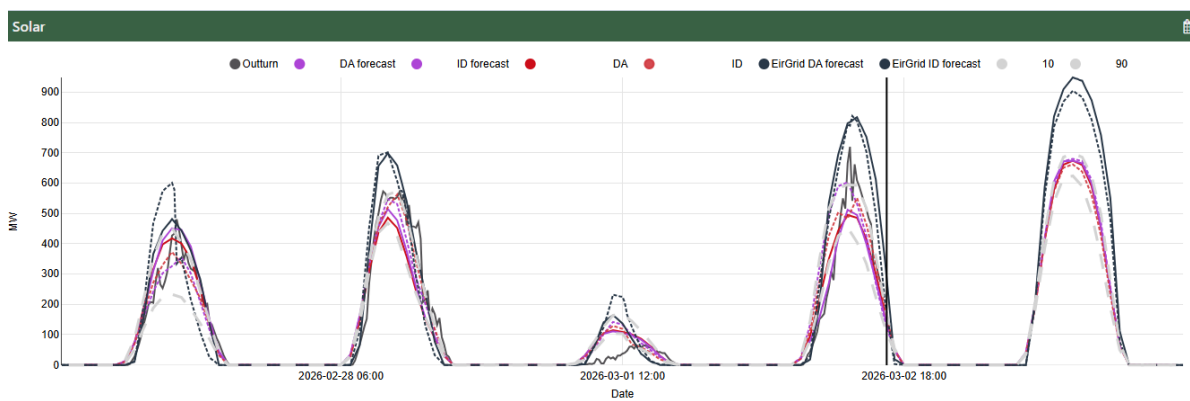


Figure 14: System-wide actual and forecasted solar generation.

7.5 Combined Assessment

In practice, no single factor determines the trading position. The variables described in the preceding subsections — weather conditions, system fundamentals, forecast confidence, interconnector flows, SNSP, NIV probability, and gas prices — all interact in complex ways, and their relative importance shifts depending on the specific conditions of the day. A high wind forecast, for example, would normally point toward confident selling, but if SNSP is already near its limit and curtailment is likely, the same high wind forecast could instead point toward a significant undersell. Similarly, a cold snap driving high demand might suggest strong DA prices, but if interconnector imports are flooding the market, that price signal may be muted.

The framework outlined in this section is therefore best understood as a set of variables and their typical effects, not a deterministic decision tool, and no two trading days looked alike in how these factors combined. Its value was in providing a structured way to work through the relevant variables and their likely interactions each morning, so that nothing was overlooked — the final position still required weighing conflicting signals,

judging which factors were dominant on a given day, and accepting the inherent uncertainty in the decision. A decision matrix was used during the morning trading workflow for exactly this purpose: mapping out each factor and its current status, not mechanically producing an answer from it.

Additional factors monitored daily included gas prices (particularly the NBP benchmark, as thermal units typically set the clearing price), planned outages of portfolio assets or interconnectors, day-of-week and seasonal demand patterns, and time-of-day effects such as the morning demand ramp, evening peak, and midday solar price suppression.

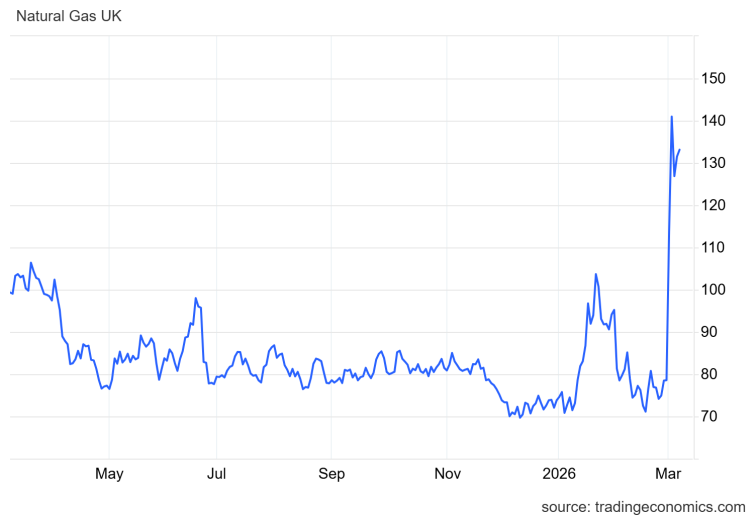


Figure 15: UK NBP Natural Gas futures prices as of 03/03/2025.

7.6 Paper Trading Results and Key Learnings

The results of each paper trading day were tracked and visualised using a custom plotting tool. Figure 16 shows an example result, with the upper panel displaying cumulative balancing costs over the trading day for both the paper-traded position and the Day-Ahead forecast baseline, and the lower panel showing the period-by-period breakdown of the difference.

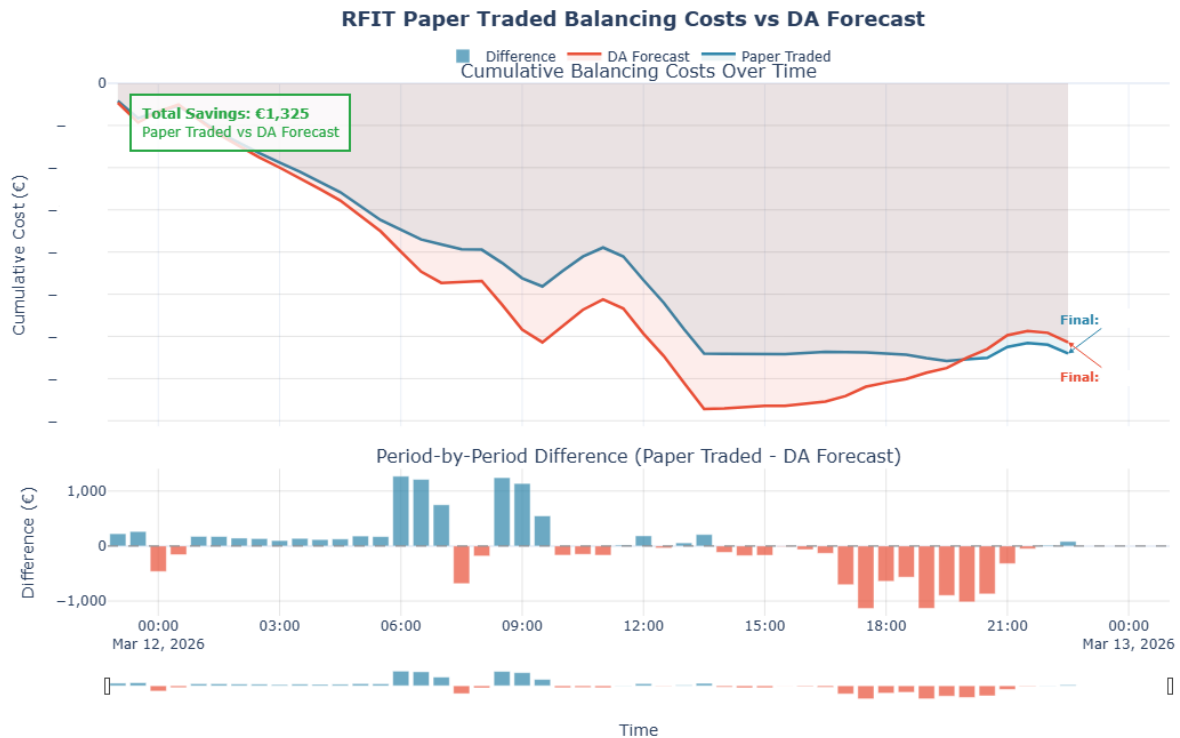


Figure 16: Example paper trading result for the REFIT portfolio, comparing cumulative balancing costs between the paper-traded position and the DA forecast baseline.

The paper trading exercise, combined with the iterative development of the trading strategy framework, reinforced several important lessons that would have been difficult to learn through analysis alone.

The most significant was developing an intuition for how forecast uncertainty translates into financial risk. Reviewing generation forecasts each morning and committing to a position forced a direct engagement with the question of how much to trust a forecast and how to hedge against the possibility of it being wrong. Over time, this built a practical understanding of the relationship between forecast confidence, traded volume, and exposure to imbalance prices.

The exercise also highlighted the importance of reading system fundamentals holistically rather than in isolation. On days with high wind penetration and low demand, for example, DA prices tended to be suppressed and the risk profile of holding volume into the Balancing Market changed accordingly. Learning to synthesise multiple signals — wind, demand, SNSP, interconnector flows, NIV probability — into a single coherent position was a skill that developed gradually through repetition.

Perhaps most importantly, the daily feedback loop provided a structured mechanism for learning from mistakes. Each trading day produced a clear, quantitative result that could be reviewed and reflected upon. This made it possible to identify patterns in decision-making — periods where positions were consistently too aggressive or too conservative — and to adjust the framework accordingly.

Although paper trading lacks the psychological pressure and market impact of live trading, it provided an essential foundation in the mechanics of energy trading and the discipline of structured decision-making under uncertainty. The trading strategy framework developed through this process represents a tangible, reusable output that makes use of the market knowledge accumulated over the course of the internship. Hopefully one day I will have the opportunity to swap the paper book for a real portfolio and become an energy trader.

8 Certifications and Professional Development

In addition to the project work described in this report, the internship included the completion of several professional development courses and certifications relevant to data science, energy markets, and compliance.

8.1 Dataquest: Data Scientist in Python

To strengthen the programming and data science skills required for the internship projects, the Dataquest *Data Scientist in Python* learning path was undertaken. This is a structured programme covering Python programming, data analysis, visualisation, data cleaning, SQL, machine learning, and advanced topics in data science. The complete learning path is available at <https://app.dataquest.io/learning/path/data-scientist/v2-7>. The courses completed to date are listed in Table 6.

Part	Course	Date Completed
1	Python Introduction	January 2025
2	Data Analysis and Visualisation	February 2025
3	Data Cleaning	April 2025
4	The Command Line	April 2025
5	Working with Data Sources Using SQL	In Progress
6	APIs and Web Scraping in Python	In Progress
7	Probability and Statistics	—
8	Machine Learning in Python	—
9	Deep Learning in Python	—
10	Advanced Topics in Data Science	—

Table 6: Dataquest *Data Scientist in Python* learning path progress. Completed course names link to their respective certificates.

The full *Data Scientist in Python* certification is pending completion of the remaining modules, which will be continued over the coming months alongside the extended internship contract.

8.2 Energy Markets Training

The *Energy Market Fundamentals* course offered by Energy Traders Europe is currently in progress. This course covers the structure and operation of European energy markets, including wholesale electricity and gas trading, market coupling, and the role of exchanges and clearing houses. Completing this course will complement the practical market knowledge gained through the internship with a broader understanding of European energy market frameworks.

8.3 Regulatory Compliance

The *Anti-Bribery Fundamentals* training course was completed as part of ElectroRoute’s mandatory compliance programme. This course covered the legal and ethical obligations relating to anti-bribery and corruption in a commercial trading environment, including the identification of bribery risks, reporting obligations, and the company’s internal policies and procedures.

9 Conclusion

This internship has been the most formative experience of my academic and professional career to date. Working within ElectroRoute's Trading Operations team for 7 months provided an immersion into the commercial realities of renewable energy trading that no classroom setting could replicate, and the chance to work on projects with genuine business impact — tools and analysis the team will continue to use after I leave — gave the work a sense of purpose and accountability that pushed me to produce my best work.

From a technical perspective, the placement transformed my ability to work with data. At the start of the internship, my experience with Python and SQL was largely self-motivated, with prior academic experience limited to C++ and MATLAB; by the end, I was writing production-level code, building full-stack applications, and applying machine learning techniques to real forecasting problems. Learning to work with large, messy, real-world datasets, and to extract meaningful insight from them, was a fundamentally different challenge to anything I had encountered in university, and one I found deeply rewarding.

Equally valuable was the commercial and market knowledge I developed. Understanding how the I-SEM operates, how renewable generators are traded, and how trading decisions are made under uncertainty gave me a perspective on the energy system that extends well beyond the engineering fundamentals covered in my degree. Sitting on the trading desk, attending market calls, and working alongside the traders allowed me to ask questions and learn from some of the most experienced individuals in the industry.

The experience also reinforced the importance of communication. Whether presenting my dashboard to the CEO or delivering analysis to the trading team, the ability to convey technical work clearly and concisely proved just as important as the technical work itself — something I will carry forward into every professional setting.

Looking ahead, this internship has firmly inspired me to pursue a career in energy trading once I graduate from my Masters in Engineering next year. The combination of quantitative problem-solving, real-time decision-making, and direct engagement with energy markets is something I find genuinely exciting. I leave this placement with a strong foundation in the skills and knowledge required to contribute in this space, and a clear ambition to build a career at the intersection of energy markets, data science, and trading.

For my final year, I would like to complete my Masters Research Project in an area that bridges energy markets and energy systems engineering. The energy sector is undergoing a period of rapid transformation, driven by increasing global demand, the integration of variable renewable generation, and the electrification of transport and heating. What was once considered a slow-moving industry has become one of the most dynamic, technology-driven spaces globally, creating new opportunities across markets, infrastructure, and innovation. It is an exciting time to be entering the field.

I am deeply grateful to the team at ElectroRoute for the opportunity, the trust, and the mentorship they provided throughout the year. This report covers the period up until mid-July, though my contract extends through to August; for the remaining time, I hope to continue contributing to the team and deepening my understanding of the energy industry, building a foundation I can carry into a graduate role.